

DÉDICACE

A
MA
FAMILLE
OUAFO

REMERCIEMENTS

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui, par leur soutien, leurs conseils et leur disponibilité, ont contribué à l'aboutissement de ce mémoire.

Avant tout, je remercie sincèrement le Seigneur Dieu Tout-Puissant pour toutes les orientations, la force, la persévérance et les multiples faveurs qu'il m'a accordées durant toutes ces années d'étude. Je lui exprime ma gratitude d'avoir rendu toutes choses possibles, car c'est grâce à Lui que j'ai pu aller jusqu'au bout.

- Au Pr. KAMMOGNE SOUP Alain, qui a accepté de superviser ce travail. Sa guidance éclairée et ses conseils avisés ont été cruciaux pour l'orientation et la progression de ce travail.
- À M. NDJE MAN Dieudonné, mon encadreur académique. Ma profonde gratitude va à sa disponibilité, sa compréhension et ses précieux encouragements qui ont été un appui constant et déterminant tout au long de cette période de rédaction.
- À M. FOPOSSE Léger, mon encadreur professionnel. Son enthousiasme, sa confiance et son soutien infaillible ont été essentiels pour mener cette étude à terme et lui donner une dimension pratique.
- Au les membres du Jury pour avoir accepté d'évaluer ce travail de recherche. Leurs échanges constructifs nous ont permis d'améliorer considérablement ce travail.
- À tout le corps professoral de l'Institut Universitaire de la Côte (IUC) et de l'Université de Dschang. Leurs enseignements de qualité ont consolidé notre savoir-faire depuis la première année.
- Tout le mérite revient à mes parents, mes sœurs et mon frère, qui m'ont toujours encouragé et soutenu moralement et financièrement jusqu'au terme de ce travail. Sans leur appui inconditionnel, ce projet n'aurait pu être mené à bien.

Je remercie également toutes les personnes, plus particulièrement les camarades de ma promotion, qui ont contribué directement ou indirectement à la réalisation de ce mémoire.

SOMMAIRE

DÉDICACE	i
REMERCIEMENTS.....	ii
SOMMAIRE	iii
LISTE DES FIGURES	iv
LISTE DES TABLEAUX.....	v
LISTE DES ABREVIATIONS.....	vi
RÉSUMÉ	vii
ABSTRACT.....	viii
INTRODUCTION GENERALE	1
CHAPITRE I : REVUE DE LA LITTÉRATURE	4
CHAPITRE II : MATÉRIELS ET MÉTHODES	1
CHAPITRE III : RÉSULTATS ET DISCUSSIONS.....	31
CONCLUSION GÉNÉRALE.....	45
RÉFÉRENCES BIBLIOGRAPHIQUES.....	47
ANNEXES.....	A
TABLE DES MATIERES	C

LISTE DES FIGURES

Figure 1: evolution des erp	10
Figure 2: Architecture Global de Formuloo OS	14
Figure 3: Diagramme de cas d'utilisation Auth/iam	16
Figure 4: Diagramme de cas d'utilisation module RH	19
Figure 6: Diagramme de cas d'utilisation du module Compta	21
Figure 7: Diagramme de cas d'utilisation du module CRM	23
Figure 8: Diagramme de classe du module Auth	25
Figure 9: Diagramme de classes du module Compta	26
Figure 10: Diagramme de classes du module CRM	26
Figure 8: Diagramme de séquence de l'authentification	27
Figure 9: Diagramme de séquence pour la génération de contrat	28
Figure 13: Diagramme d'activité pour le traitement d'une API	29
Figure 14: Diagramme d'état transition pour le cycle de vie d'un lead	30
Figure 15: Interface d'authentification	35
Figure 16: Pipeline validact docs montrant les status	32
Figure 17: Interface du dashboard grafana	36
Figure 18: interface swagger ui pour endpoints CRM	38
Figure 19: Interface Angular des CRM	38
Figure 20: Interface Angular pour le contact	39
Figure 21: Interface swagger ui pour le CRM contact	39
Figure 22: Interface pour l'observabilité grafana	40

LISTE DES TABLEAUX

Tableau 1: Comparaison des architectures logicielles pour les systèmes d'entreprise	13
Tableau 2: Comparaison des ERP disponibles sur le marché Africain	15
Tableau 3: Outils Matériels utilisés dans le projet	2
Tableau 4: Récapitulatifs des outils logiciels utilisés dans le projet	3
Tableau 5: Intervenants du Projet Formuloo OS	5
Tableau 6: Comparaison des méthodes d'estimation du coût logiciel	6
Tableau 7: Estimations des coûts des ressources matérielles	7
Tableau 8: Estimation des coûts des ressources logicielles	8
Tableau 9: Estimation des coûts des ressources humaines	8
Tableau 10: Coût total du projet Formuloo OS	8
Tableau 8: Comparaison des approches de développement	10
Tableau 9: Comparaison du Contract-first et du Code First	11
Tableau 13: Comparaison de Merise et UML 2.5	12
Tableau 14: Description textuelle du cas d'utilisation s'authentifier	17
Tableau 15: Description du cas d'utilisation gérer les Permissions RBAC	18
Tableau 16: Description du cas d'utilisation Générer la paie	20
Tableau 17: Description textuelle du cas d'utilisation saisie d'une écriture comptable	22
Tableau 18: Description textuelle pour gérer le pipeline lead	24
Tableau 19: Répartition des endpoints documentés par module et requêtes HTTP	33
Tableau 20: Résultats des tests d'isolation Multi-tenant	34
Tableau 21: Alerte Prometheus configurée dans Formuloo OS	36
Tableau 22: Évaluation de l'atteinte des objectifs spécifiques	41
Tableau 23: Planification des sprints scrum Formuloo	A

LISTE DES ABREVIATIONS

API : Application Programming Interface
ADR : Architectural Decision Record
CI/CD : Continuous Integration / Continuous Delivery
DRF : Django REST Framework
ERP : Enterprise Resource Planning (Progiciel de Gestion Intégré)
HTTP : HyperText Transfer Protocol
IAM : Identity and Access Management
JWT : JSON Web Token
JSON: JavaScript Object Notation
OAS : OpenAPI Specification
OIDC : OpenID Connect
PME : Petites et Moyennes Entreprises
REST : Representational State Transfer
SaaS : Software as a Service
SSO : Single Sign-On
TDD : Test-Driven Development
UML : Unified Modeling Language
YAML : YAML Ain't Markup Language

RÉSUMÉ

Les petites et moyennes entreprises africaines font face à une fragmentation chronique de leurs outils de gestion : tableurs non intégrés, logiciels disparates, processus manuels incompatibles entre eux. Cette fragmentation rend impossible le pilotage cohérent de l'activité et expose ces entreprises à des risques opérationnels et réglementaires importants. Si des solutions ERP existent sur le marché mondial Odoo, SAP, Sage X3 leur coût de déploiement, leur complexité d'adoption et leur dépendance à des intégrateurs certifiés les rendent financièrement inaccessibles à la grande majorité des PME africaines. C'est pour répondre à ce manque qu'a été lancé Formuloo OS, un ERP cloud-native conçu dès sa fondation pour les contraintes spécifiques des PME de la zone CFA. Ce mémoire présente la conception et la mise en œuvre de l'architecture backend orientée services de Formuloo OS, assortie d'un mécanisme de gouvernance contractuelle des API REST. Dans le contexte du développement agile de ce projet par plusieurs équipes travaillant en parallèle sur des modules distincts backend, frontend Angular, mobile l'absence de contrat d'interface formalisé générait des régressions silencieuses détectées tardivement, bloquant l'avancement du frontend et fragilisant la cohérence d'ensemble du système. Pour résoudre ce problème, nous avons adopté une approche Contract-First reposant sur trois piliers : drf-spectacular 0.27.2 pour la génération automatique des spécifications OpenAPI 3.1, CONVENTIONS.md et quatre Architecture Decision Records pour la formalisation des règles transverses, et un pipeline GitLab CI `validate_docs` pour la vérification automatique des artefacts contractuels à chaque merge request. La sécurité multi-tenant est assurée par `TenantJWTAuthentication`, qui extrait le `tenant_id` du JWT à chaque requête, et `TenantFilterMixin`, qui ajoute automatiquement la clause `WHERE tenant_id` sur chaque requête SQL. Les résultats obtenus sont concrets : 56 endpoints documentés et conformes à leurs contrats OpenAPI, quatre ADR validés en intégration continue, un pipeline opérationnel en 42 secondes (pipeline #230 PASSED), et une architecture modulaire déployée sous Docker Compose.

Mots-clés : Architecture orientée services, API REST, gouvernance contractuelle, OpenAPI, Contract-First, Django REST Framework, drf-spectacular.

ABSTRACT

African small and medium-sized enterprises face a chronic fragmentation of their management tools: non-integrated spreadsheets, disparate software packages, and manual processes that are mutually incompatible. This fragmentation makes it impossible to run the business coherently and exposes these companies to significant operational and regulatory risks. While ERP solutions exist on the global market — Odoo, SAP, Sage X3 — their deployment costs, complexity of adoption, and dependence on certified integrators make them financially inaccessible to the vast majority of African SMEs. It is to address this gap that Formuloo OS was launched: an cloud-native ERP designed from the ground up for the specific constraints of SMEs in the CFA zone. This thesis presents the design and implementation of the service-oriented backend architecture of Formuloo OS, together with a contractual governance mechanism for REST APIs. In the context of the agile development of this project by multiple teams working in parallel on distinct modules — backend, Angular frontend, mobile — the absence of a formalized interface contract generated silent regressions detected late, blocking frontend progress and undermining overall system consistency. To solve this problem, we adopted a Contract-First approach based on three pillars: drf-spectacular 0.27.2 for automatic OpenAPI 3.1 specification generation from Django code, CONVENTIONS.md and four Architecture Decision Records for cross-cutting rules formalization, and a GitLab CI `validate_docs` pipeline for automated contractual artifact verification at each merge request. Multi-tenant security is ensured by `TenantJWTAuthentication`, which extracts the `tenant_id` from the JWT at each request, and `TenantFilterMixin`, which automatically appends the `WHERE tenant_id` clause to every SQL query. The results achieved are concrete and verifiable: 56 endpoints documented and compliant with their OpenAPI contracts, four ADRs validated in continuous integration, an operational pipeline completing in 42 seconds (pipeline #230 PASSED), and a modular architecture deployed under Docker Compose.

Keywords: Service-oriented architecture, REST API, contractual governance, OpenAPI, Contract-First, Django REST Framework, drf-spectacular.

INTRODUCTION GENERALE

Les progiciels de gestion intégrés se sont imposés comme des outils incontournables pour structurer et automatiser les processus d'affaires des entreprises modernes. En centralisant dans une base de données unique les fonctions de comptabilité, de gestion des ressources humaines, de relation client et de gestion des stocks, ils offrent aux dirigeants une visibilité et un contrôle accrus sur l'ensemble de l'activité opérationnelle. Pourtant, malgré leur diffusion mondiale, ces outils n'ont pas encore atteint la grande majorité des entreprises africaines. Selon la Banque Africaine de Développement, les petites et moyennes entreprises représentent plus de 90 % du tissu entrepreneurial du continent et assurent entre 60 et 70 % de l'emploi formel, mais moins de 22 % d'entre elles avaient adopté des outils de gestion numérique au Cameroun à la date du dernier recensement général des entreprises. Cette réalité s'explique par un double obstacle structurel : d'un côté, les solutions ERP disponibles sur le marché Odoo, SAP, Sage X3 sont conçues pour des marchés occidentaux et imposent des coûts de déploiement estimés entre 300 000 et 2 000 000 FCFA selon les intégrateurs locaux consultés ; de l'autre, elles ne supportent pas nativement le Système Comptable de l'OHADA, référentiel comptable obligatoire dans dix-sept pays africains, qui impose des règles d'enregistrement et des formats de clôture spécifiques, différents des normes IFRS et US GAAP sur lesquelles ces produits ont été construits.

C'est pour répondre à ce manque que l'entreprise Formuloo a lancé le projet Formuloo OS, une suite ERP cloud-native et multi-tenant, pensée dès sa fondation pour les contraintes spécifiques des PME de la zone CFA : accessibilité sans intégrateur certifié, conformité native au SYSCOHADA, et capacité à mutualiser les ressources de plusieurs organisations clientes sur une même infrastructure. Nous avons effectué notre stage de Master 2 au sein de l'équipe de développement de cette entreprise, sous la supervision de Leger FOPOSSE. Au cours de notre prise de poste, nous avons rapidement constaté que le développement reposait sur une organisation multi-équipes travaillant simultanément sur des modules distincts : une équipe backend en charge des modules Auth/IAM, Ressources Humaines, Comptabilité, CRM, Stock, Projet, Analytic une équipe frontend Angular développant l'interface utilisateur, et une équipe mobile développant l'application native. Ces trois équipes partagent des interfaces de programmation les API REST dont elles dépendent mutuellement. Chaque fois qu'un développeur backend modifie le nom d'un champ, supprime un endpoint ou change le comportement d'une opération, il risque de provoquer une rupture silencieuse dans le frontend ou l'application mobile, détectée parfois plusieurs jours après son introduction, au moment des sessions d'intégration.

L'absence de contrat d'interface formalisé et de mécanisme de validation automatique exposait le projet à ce que la littérature en ingénierie logicielle nomme régressions silencieuses : des modifications du code d'un service qui passent inaperçues lors du développement et ne sont découvertes qu'au moment de l'intégration, au coût d'heures de débogage et de retards dans les livraisons. Ce double constat fragmentation des outils de gestion des PME africaines d'une part, absence de gouvernance des contrats d'interface dans un projet agile multi-équipes d'autre part conduit à la problématique centrale de ce mémoire : *dans le cadre du développement agile de Formuloo OS par plusieurs équipes travaillant en parallèle sur des modules distincts, comment concevoir et mettre en œuvre une architecture backend orientée services dotée d'un mécanisme de gouvernance contractuelle automatisée, garantissant la conformité continue des implémentations au contrat OpenAPI partagé, la sécurité des accès et l'isolation stricte des données dans un environnement multi-tenant ?*

Pour répondre à cette problématique, l'objectif général de ce travail est de concevoir et de mettre en œuvre l'architecture backend orientée services de Formuloo OS, intégrant un mécanisme de gouvernance contractuelle des API REST garantissant la sécurité des accès et l'isolation stricte des données multi-tenant. Cet objectif général se décline en six objectifs spécifiques : conduire une étude comparative des architectures logicielles et des ERP existants sur le marché africain ; concevoir une architecture backend en six domaines métier avec une passerelle API unique et des contrats d'interface formalisés ; mettre en œuvre le mécanisme de gouvernance Contract-First reposant sur drf-spectacular, CONVENTIONS.md et quatre Architecture Decision Records ; implémenter le modèle de sécurité multi-tenant via TenantJWTAuthentication et TenantFilterMixin ; réaliser la modélisation UML complète du système ; et évaluer les résultats obtenus par rapport aux objectifs fixés. Trois hypothèses de recherche guident notre démarche : l'adoption d'une approche Contract-First couplée à un pipeline de validation automatique réduit les régressions silencieuses entre équipes (H1) ; le monolithe modulaire constitue l'architecture optimale pour un ERP destiné aux PME africaines développé par une petite équipe (H2) ; un modèle de sécurité multi-tenant fondé sur un filtrage systématique par identifiant de tenant au niveau de la couche de persistance garantit une isolation suffisante tout en maintenant des performances acceptables (H3).

Ce travail est motivé par trois convictions nées de notre expérience directe sur le terrain. La première est que la digitalisation des PME africaines ne peut pas se faire par simple transposition des solutions occidentales : elle exige des outils pensés depuis le départ pour les contraintes locales, notamment la conformité au SYSCOHADA, la nécessité de fonctionner dans

des environnements à connectivité variable, et l'absence d'équipe informatique dédiée au sein des entreprises ciblées. La deuxième conviction est que la gouvernance des contrats d'interface API constitue un problème d'ingénierie logicielle trop souvent ignoré dans les projets africains, alors qu'il conditionne directement la qualité et la fiabilité des systèmes multi-équipes. La troisième conviction est que les bonnes pratiques en matière de documentation vivante, d'approche Contract-First et de pipeline de validation automatique sont accessibles à des équipes de toute taille, à condition d'être présentées de façon pragmatique et outillée.

Le présent mémoire s'organise autour de trois chapitres. Le premier est consacré à la revue de la littérature : nous y analysons le contexte de la digitalisation des PME africaines, conduisons une étude comparative des architectures logicielles disponibles et des ERP existants sur le marché africain, et exposons les fondements théoriques de la gouvernance contractuelle des API REST. Le second chapitre décrit les matériaux et les méthodes retenus : la stack technique du projet dans ses versions exactes confirmées par les fichiers de configuration du dépôt, la méthodologie Agile Scrum telle qu'elle a été instanciée dans l'équipe, les décisions architecturales fondatrices documentées dans les ADR, et la modélisation UML complète du système couvrant les diagrammes de cas d'utilisation, de classes, de séquences, d'activité et d'état. Le troisième chapitre présente et discute les résultats obtenus : les livrables produits et leurs métriques vérifiables, la validation critique des trois hypothèses de recherche, et les limites honnêtement identifiées de notre approche.

CHAPITRE I : REVUE DE LA LITTÉRATURE

Ce premier chapitre pose les fondements théoriques et contextuels de notre travail. Il s'articule autour de trois axes principaux. La première partie analyse le contexte de la digitalisation des petites et moyennes entreprises africaines, les caractéristiques du tissu économique dans lequel s'inscrit Formuloo OS, et les contraintes réglementaires propres à la zone OHADA. La deuxième partie conduit une étude comparative des architectures logicielles et des solutions ERP disponibles sur le marché africain, afin de positionner les choix de conception de Formuloo OS dans un cadre théorique rigoureux. La troisième partie, plus étendue, présente les fondements théoriques de la gouvernance des API REST, l'approche Contract-First et ses outils, les travaux récents sur ce sujet, et propose une analyse critique permettant d'identifier les lacunes que notre projet ambitionne de combler.

I.1. Les PME africaines et la digitalisation de leurs processus métier

I.1.1. Caractéristiques et poids économique des PME en Afrique subsaharienne

I.1.1.1. Définition et périmètre statistique

Les petites et moyennes entreprises sont définies différemment selon les institutions. L'OHADA, dans son Acte Uniforme relatif au droit des sociétés commerciales, retient le double critère de moins de 200 salariés et d'un chiffre d'affaires annuel inférieur à un milliard de FCFA. La Banque Africaine de Développement adopte une approche plus large qui inclut les très petites entreprises moins de dix salariés dans le périmètre de l'entrepreneuriat formel [1]. La Banque mondiale, pour ses travaux statistiques comparatifs, distingue les micro-entreprises de moins de dix salariés, les petites entreprises de dix à quarante-neuf salariés, et les entreprises moyennes de cinquante à deux cent quarante-neuf salariés [4]. Ces différences de définition compliquent les comparaisons mais ne remettent pas en cause le constat fondamental : quelle que soit la définition retenue, les PME constituent la très grande majorité du tissu entrepreneurial africain.

Selon la Banque Africaine de Développement [1], les PME représentent plus de 90 % des entreprises formelles d'Afrique subsaharienne et assurent entre 60 et 70 % de l'emploi formel sur le continent. Leur contribution au produit intérieur brut est estimée à 40 % dans la plupart des économies de la région. Au Cameroun, le Recensement Général des Entreprises conduit en 2016 par l'Institut National de la Statistique dénombrait 93 969 entreprises formellement enregistrées, dont 98,3 % relevaient de la catégorie des PME. Ce constat est généralisable à l'ensemble des pays

de la zone OHADA : les PME y sont omniprésentes, économiquement déterminantes, et pourtant sous-équipées en outils de gestion numérique.

I.1.1.2. Le déficit de digitalisation et ses causes structurelles

La Banque mondiale estime que seulement 22 % des PME formelles camerounaises avaient pleinement adopté des outils de gestion numérique à la date du Recensement Général des Entreprises [4]. Ce taux, parmi les plus faibles du monde pour une économie de cette taille, est le résultat d'un ensemble de facteurs structurels systématiquement documentés dans la littérature. Molla et Licker [7], dans une étude pionnière conduite sur l'ensemble de l'Afrique subsaharienne et publiée en 2005, ont identifié quatre obstacles récurrents à l'adoption des systèmes d'information dans les PME africaines : le coût prohibitif des licences logicielles et des déploiements, la complexité d'utilisation des outils disponibles conçus pour des contextes organisationnels différents, l'inadaptation aux pratiques locales en termes de langue d'interface, de devise et de réglementation, et la rareté des compétences techniques nécessaires à l'installation et à la maintenance de ces systèmes.

Njokou, Tsafack et Bella [6], dans une étude spécifiquement consacrée aux PME camerounaises publiée en 2022, confirment la persistance de ces quatre obstacles deux décennies après les travaux de Molla et Licker, tout en ajoutant une cinquième dimension : la méfiance culturelle envers les systèmes qui externalisent la maîtrise de l'information de l'entreprise. Les dirigeants camerounais interrogés expriment une réticence à confier à un logiciel tiers la gestion de données qu'ils considèrent comme stratégiquement sensibles : bases de clients, données financières, rémunérations des employés. Cette résistance psychologique constitue un frein complémentaire aux obstacles économiques et techniques. Face à ces contraintes cumulées, la majorité des PME africaines continue de gérer ses processus métier sur des supports fragmentés et non intégrés : tableurs Excel pour la comptabilité, carnets papier pour le suivi des clients, fichiers Word pour les contrats de travail, et messagerie électronique pour les validations internes.

I.1.2. Le cadre réglementaire OHADA et les enjeux du SYSCOHADA

I.1.2.1. L'Organisation pour l'Harmonisation en Afrique du Droit des Affaires

L'Organisation pour l'Harmonisation en Afrique du Droit des Affaires a été créée par le Traité de Port-Louis du 17 octobre 1993, entré en vigueur en 1995. Elle regroupe aujourd'hui dix-sept États membres : Bénin, Burkina Faso, Cameroun, Centrafrique, Comores, Congo, Côte d'Ivoire, Gabon, Guinée, Guinée-Bissau, Guinée Équatoriale, Mali, Niger, République

Démocratique du Congo, Sénégal, Tchad et Togo. L'OHADA a pour mission d'harmoniser le droit des affaires dans ses États membres par l'élaboration et l'adoption de règles communes simples, modernes et adaptées à la situation de leurs économies. Ses actes uniformes, directement applicables dans chaque État membre sans transposition nationale préalable, couvrent notamment le droit commercial général, les sociétés commerciales, les sûretés, les voies d'exécution, les procédures collectives d'apurement du passif, le droit du travail, les contrats de transport de marchandises par route, et la comptabilité des entreprises [5].

I.1.2.2. Le Système Comptable OHADA et ses exigences

L'Acte Uniforme portant organisation et harmonisation des comptabilités des entreprises, dans sa version révisée adoptée le 26 janvier 2017, impose aux entités relevant du périmètre OHADA l'utilisation du Système Comptable OHADA dit SYSCOHADA comme référentiel comptable de référence. Le SYSCOHADA définit un plan de comptes normalisé organisé en huit classes numérotées de 1 à 8 comptes de capitaux permanents, actifs immobilisés, actifs circulants, trésorerie, charges par nature, produits par nature, charges des activités ordinaires, et résultat ainsi que les règles d'enregistrement des opérations dans le journal, le grand-livre et la balance. Il prescrit également les formats de présentation obligatoires des états financiers de synthèse : le bilan selon le modèle actif-passif propre au référentiel OHADA, le compte de résultat organisé en charges et produits des activités ordinaires et des activités non ordinaires, le tableau de financement des ressources et emplois, et l'état annexé qui fournit des informations complémentaires sur les méthodes comptables utilisées et les éléments significatifs des comptes [5].

Ces exigences diffèrent substantiellement des normes IFRS (International Financial Reporting Standards), référentiel adopté par plus de 140 pays et sur lequel est fondée la comptabilité des solutions ERP occidentales de référence. La structure du plan de comptes SYSCOHADA, les règles de valorisation des actifs, les modalités de calcul de l'amortissement des immobilisations, et les formats des états financiers sont spécifiques à la zone OHADA et ne peuvent pas être simplement déduits d'un paramétrage IFRS. Cette incompatibilité structurelle explique pourquoi les ERP conçus pour des marchés occidentaux ne peuvent pas être déployés dans les PME africaines sans une adaptation substantielle de leur module comptable adaptation que peu d'éditeurs ont intégrée nativement dans le cœur de leur produit.

I.1.3. Les ERP comme réponse à la fragmentation : concepts fondamentaux

Le terme Enterprise Resource Planning a été popularisé au début des années 1990 par le cabinet Gartner pour désigner les systèmes logiciels intégrant dans une base de données unique l'ensemble des processus opérationnels d'une entreprise. Davenport [8] en donne en 1998 la définition fondatrice : un ERP est un « package commercial de logiciels qui promet d'intégrer l'ensemble de l'information qui circule dans une entreprise : information financière, information des ressources humaines, information de la chaîne d'approvisionnement, information client ». Cette intégration, selon Davenport, constitue la valeur fondamentale d'un ERP : non pas les fonctionnalités individuelles de chacun de ses modules qui existent souvent séparément dans des solutions spécialisées mais les flux de données automatiques qui relient ces modules entre eux et garantissent la cohérence de l'information à l'échelle de l'entreprise.

La valeur concrète de cette intégration pour une PME africaine se manifeste dans des scénarios quotidiens : lorsqu'une facture est émise dans le module CRM, l'écriture comptable correspondante est générée automatiquement dans le module Comptabilité sans ressaisie ni risque d'incohérence ; lorsqu'un employé est embauché dans le module RH, son accès aux ressources informatiques est créé automatiquement dans le module Authentification ; lorsqu'un client effectue un paiement, le solde de son compte est mis à jour simultanément dans le module CRM et dans la comptabilité client. Ces automatisations éliminent la fragmentation chronique des outils qui contraignent aujourd'hui les PME africaines à maintenir des saisies redondantes dans des systèmes déconnectés, au risque d'incohérences qui faussent les tableaux de bord et compliquent les clôtures comptables. C'est précisément pour apporter ces bénéfices aux PME de la zone CFA que le projet Formuloo OS a été lancé par l'entreprise Formuloo [5].

Au terme de cette première section, nous retenons trois constats fondamentaux. Premièrement, les PME africaines constituent le cœur du tissu économique du continent mais souffrent d'un déficit de digitalisation profond et multidimensionnel. Deuxièmement, la réglementation comptable spécifique à la zone OHADA constitue un obstacle structurel à l'adoption des ERP existants qui ne supportent le SYSCOHADA que de façon partielle et via des adaptations communautaires fragiles. Troisièmement, la valeur fondamentale d'un ERP pour une PME africaine réside dans l'intégration de ses processus métier dans un système unique, cohérent et accessible sans infrastructure coûteuse. Ces trois constats définissent le cahier des charges de Formuloo OS et motivent les choix architecturaux que nous analysons dans la section suivante.

I.1.4. Évolution historique des ERP et leurs générations

I.1.4.1. Première génération : les MRP et la naissance de la planification intégrée (1960-1980)

L'histoire des systèmes de gestion intégrés remonte aux années 1960, bien avant que le terme ERP ne soit inventé. Les premières solutions de gestion informatisée ciblaient exclusivement la planification des besoins en matériaux de production, connues sous l'acronyme MRP (Material Requirements Planning). IBM est généralement crédité du premier MRP commercial, développé en 1964 pour son client Black & Decker. Ces systèmes calculaient les besoins en composants, en matières premières et en capacité de production à partir d'un plan directeur de production. Ils fonctionnaient en mode batch traitement par lots nocturnes sur des mainframes IBM dont les coûts de location atteignaient plusieurs dizaines de millions de francs par mois, limitant leur adoption aux seules multinationales industrielles. La gestion financière, la gestion du personnel et la relation client restaient des domaines entièrement distincts, gérés par des applications séparées sans aucune intégration automatique.

Dans les années 1970, la seconde génération MRP II (Manufacturing Resource Planning) étendit le périmètre de la planification en intégrant la gestion des capacités machines, le suivi des coûts de production, et une interface rudimentaire avec la comptabilité générale. Cette évolution constitua la première étape vers l'intégration des processus d'entreprise, mais restait limitée au secteur manufacturier. Les PME, en dehors du secteur industriel, n'avaient accès à aucun outil de planification intégrée et continuaient de gérer leur activité dans des registres papier ou, au mieux, dans des fichiers de tableur issus des premiers ordinateurs personnels.

I.1.4.2. Deuxième génération : l'ERP et l'intégration totale (1990-2005)

Le terme Enterprise Resource Planning est apparu officiellement dans un rapport du cabinet Gartner en 1990. Il désignait une nouvelle génération de logiciels qui étendaient le périmètre du MRP II à l'ensemble des processus d'une entreprise non plus seulement la production, mais aussi la finance, les ressources humaines, les achats, la logistique et la relation client dans une base de données unique et cohérente. SAP AG, fondé en 1972 par cinq anciens ingénieurs d'IBM, lança SAP R/3 en 1992 : le premier ERP à architecture client-serveur, conçu pour fonctionner sur des serveurs Unix ou Windows de coût bien inférieur aux mainframes de la génération précédente. SAP R/3 devint rapidement la référence du secteur et contribua à démocratiser l'ERP auprès des grandes entreprises mondiales. Baan, PeopleSoft, JD Edwards et Oracle Applications complétèrent l'offre sur ce segment.

Davenport [8] notait en 1998 que l'adoption d'un ERP constituait une décision stratégique majeure qui imposait à l'entreprise d'adapter ses processus métier aux meilleures pratiques intégrées dans le logiciel, ou de personnaliser le logiciel pour l'adapter à ses processus avec le risque de complexifier les mises à jour futures. Cette tension entre standardisation et personnalisation est restée le principal défi de l'adoption des ERP de seconde génération. Les coûts de déploiement atteignaient plusieurs millions de dollars pour les grandes entreprises, et les projets d'implémentation duraient en moyenne dix-huit à trente-six mois. Les PME restaient largement exclues de ce marché, faute de ressources financières et humaines suffisantes.

I.1.4.3. Troisième génération : le cloud-native et l'ERP accessible (2010-présent)

L'avènement du cloud computing au tournant des années 2010 a profondément transformé le marché des ERP. En 2008, Workday lança le premier ERP financier entièrement conçu pour le cloud, sans version on-premise. Salesforce, dont la plateforme CRM avait précédé ce mouvement dès 1999, étendit progressivement son offre vers les fonctions ERP. NetSuite, racheté par Oracle en 2016, s'était positionné dès 2000 comme le premier ERP cloud complet. Cette troisième génération d'ERP cloud-native présente des caractéristiques fondamentalement différentes des générations précédentes : déploiement en quelques jours ou semaines au lieu de mois, modèle de tarification par abonnement mensuel au lieu d'une licence initiale massive, mises à jour automatiques sans projet de migration, et accessibilité depuis n'importe quel navigateur web sans logiciel client à installer. C'est dans cette troisième génération que s'inscrit Formuloo OS, avec la spécificité supplémentaire d'être open-source et conçu nativement pour les contraintes du contexte africain.

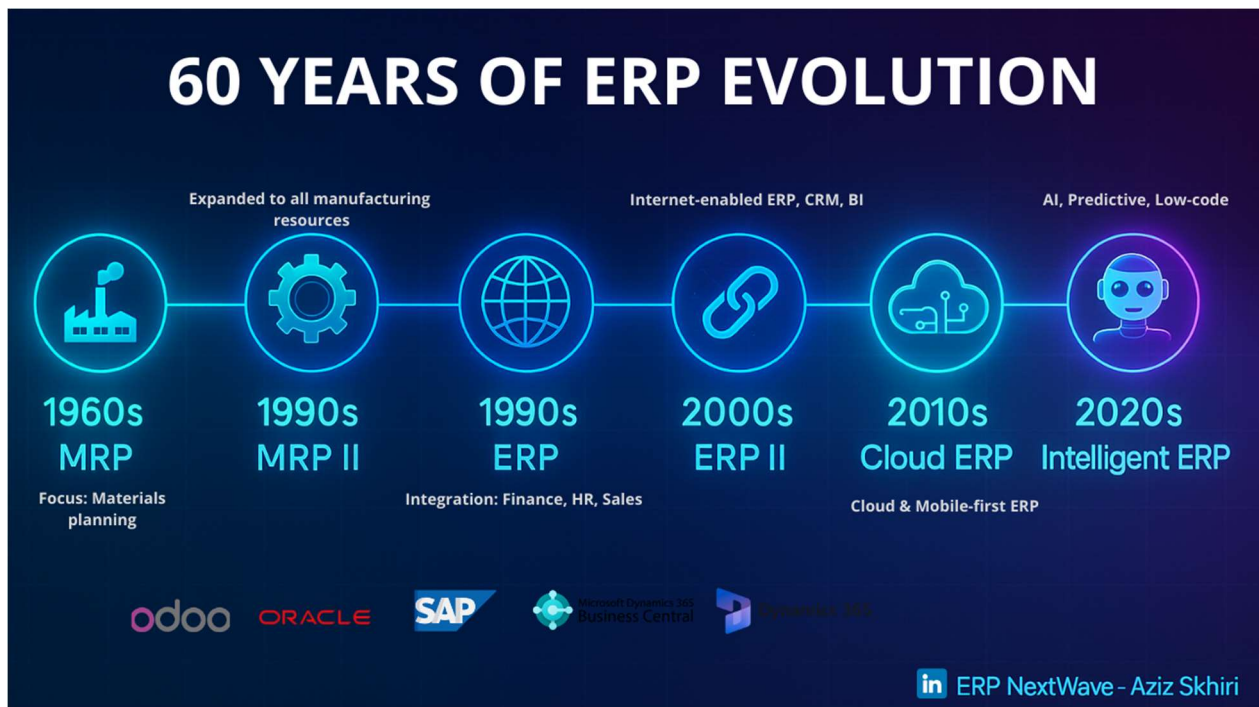


Figure 1: evolution des erp

I.2. Architectures logicielles et ERP existants sur le marché africain

I.2.1. Les grandes familles d'architectures logicielles pour les systèmes d'entreprise

I.2.1.1. Le monolithe classique

Le monolithe classique est la forme architecturale la plus ancienne et la plus répandue dans l'histoire du développement logiciel d'entreprise. Dans cette architecture, l'ensemble des composants d'une application interface utilisateur, logique métier, couche d'accès aux données résident dans une seule et même base de code, partagent un processus d'exécution unique, et se déploient comme un tout indivisible sur un serveur ou un cluster de serveurs identiques. Les premiers ERP, au début des années 1990, étaient tous des monolithes classiques : SAP R/3, Baan, PeopleSoft, et les versions initiales d'Odoo alors appelé TinyERP ont tous été construits sur ce modèle. Davenport [8] note que cette architecture est précisément ce qui confère aux ERP de première génération leur propriété la plus précieuse : la cohérence transactionnelle globale, garantie par le fait que toutes les opérations partagent la même base de données et le même moteur de transactions ACID (Atomicité, Cohérence, Isolation, Durabilité).

Le monolithe classique présente des avantages indéniables pour les équipes de petite taille : simplicité du déploiement (un seul artefact à livrer), facilité du débogage (traces d'exécution unifiées), absence de complexité réseau inter-composants, et transactions distribuées inutiles puisque tout est dans le même processus. Ces avantages ont longtemps fait du monolithe classique

le modèle par défaut de la construction de systèmes d'information. Cependant, Bogner et ses collègues [10], dans leur étude publiée en 2019 sur l'évolutivité des architectures logicielles, identifient trois limitations structurelles qui deviennent rédhibitoires lorsque le système croît : la difficulté à scaler indépendamment les composants à forte charge (l'ensemble de l'application doit être dupliqué même si seul un module est sous pression), l'impossibilité de déployer une modification isolée (toute mise à jour nécessite le redéploiement de l'application entière avec le risque d'impacter des modules non modifiés), et la dégradation progressive de la maintenabilité au fil du temps par accumulation de couplages implicites entre modules.

I.2.1.2. L'architecture microservices

L'architecture microservices a émergé comme réponse aux limitations du monolithe classique dans des contextes de très grande échelle, popularisée à partir de 2009 par des entreprises dont les systèmes avaient atteint les limites de l'approche monolithique : Netflix, Amazon, Uber, Spotify. Dans cette architecture, chaque capacité métier paiement, catalogue, recommandation, notification est développée et déployée comme un service indépendant, avec sa propre base de données, son propre cycle de déploiement, et sa propre technologie. Les services communiquent entre eux exclusivement par des interfaces bien définies : des API REST synchrones pour les interactions en temps réel, ou des messages asynchrones via un broker (RabbitMQ, Kafka) pour les interactions qui n'exigent pas de réponse immédiate.

Neumann et ses collègues [8] ont conduit en 2021 une analyse systématique de 2 616 API REST publiques issues de systèmes microservices en production et concluent que la qualité et la cohérence des interfaces entre services constituent le principal facteur de maintenabilité à long terme de ces architectures. Google Cloud [9] recommande de ne considérer cette architecture qu'à partir d'une équipe d'ingénierie d'au moins dix développeurs disposant d'une infrastructure DevOps mature incluant un orchestrateur de conteneurs Kubernetes, un service mesh de type Istio ou Linkerd pour la gestion du trafic inter-services, et une plateforme d'observabilité distribuée permettant de tracer une requête à travers les multiples services qu'elle traverse. Bogner et ses collègues [10] confirment que la complexité opérationnelle des microservices est substantiellement plus élevée que celle du monolithe classique, avec des défis spécifiques : gestion des transactions distribuées, maintien de la cohérence éventuelle des données réparties entre plusieurs bases, gestion des pannes partielles, et test d'intégration dans un environnement multi-services. Pour une équipe de deux développeurs backend dans une start-up africaine, ces prérequis sont hors de portée à court terme.

I.2.1.3. Le monolithe modulaire

Le monolithe modulaire est apparu comme un compromis architectural entre les deux extrêmes précédents. Il conserve l'avantage opérationnel et économique du déploiement unitaire du monolithe classique, mais impose des frontières de domaine strictes entre les composants de l'application en les organisant en modules clairement délimités chacun encapsulant sa logique métier, ses entités de données et ses interfaces dans un package ou un répertoire dédié. La communication inter-modules est gouvernée par des contrats d'interface explicites, de la même façon que dans une architecture microservices, mais ces échanges restent internes à l'application et ne nécessitent pas de communication réseau. Su et Li [10], dans une étude de la littérature grise publiée en 2024 sur les cas industriels de monolithe modulaire, démontrent que cette architecture permet de maintenir la cohérence transactionnelle native du monolithe tout en préservant les frontières de domaine qui facilitent l'évolutivité et la gouvernance des interfaces.

Cette architecture est aujourd'hui adoptée par plusieurs ERP majeurs. ERPNext la décrit explicitement dans sa documentation officielle en déclarant « nous croyons en l'architecture monolithique » tout en imposant une organisation stricte du code en applications Frappe indépendantes. Odoo 17 organise ses fonctionnalités en modules Python distincts avec des points d'extension définis. La décision d'adopter cette architecture pour Formuloo OS est documentée dans l'Architecture Decision Record ADR-001 du projet, qui argumente sur la base de trois critères mesurables : la taille de l'équipe backend (deux développeurs), l'infrastructure disponible (Docker Compose sans Kubernetes), et l'objectif de préserver la possibilité d'une migration progressive vers des microservices si les besoins de scalabilité l'exigent à terme.

Tableau 1: Comparaison des architectures logicielles pour les systèmes d'entreprise

Critère	Monolithe classique	Microservices distribués	Monolithe modulaire
Complexité initiale	Faible	Très élevée	Moyenne
Taille d'équipe minimale	1-3 développeurs	10+ développeurs	2-5 développeurs
Infrastructure requise	Un serveur	Kubernetes + service mesh	Docker Compose
Frontières de domaine	Floues couplage fort	Strictes indépendance totale	Clares couplage contrôlé
Cohérence transactionnelle	Native ACID global	Complexe sagas distribuées	Native ACID par module
Gouvernance des API	Impossible	Nécessaire et complexe	Naturelle par contrat
Scalabilité horizontale	Limitée	Fine par service	Globale par module
Migration vers microservices	Réécriture complète	N/A	Progressive sans réécriture

Source : élaboré par l'auteur à partir de Bogner et al. , Neumann et al. , Google Cloud et Su & Li 2019-2024.

I.2.2. Les solutions ERP disponibles sur le marché africain

I.2.2.1. Les ERP propriétaires de premier niveau

Le marché mondial des progiciels de gestion intégrés est historiquement dominé par un oligopole d'éditeurs propriétaires dont les solutions ciblent principalement les grandes entreprises et les multinationales. SAP AG, fondé en 1972 en Allemagne, a lancé SAP R/3 en 1992 le premier ERP à architecture client-serveur puis SAP S/4HANA en 2015, version cloud-native exploitant la base de données in-memory HANA. Oracle Corporation, à travers son acquisition de PeopleSoft en 2005 puis de Netsuite en 2016, propose Oracle ERP Cloud comme solution cloud unifiée. Sage Group, éditeur britannique fondé en 1981, commercialise Sage X3 comme solution de référence pour les entreprises de taille intermédiaire. Ces trois acteurs partagent plusieurs caractéristiques communes qui les rendent structurellement inaccessibles aux PME africaines : des coûts de licence

annuels qui se mesurent en dizaines de milliers d'euros, des processus de déploiement exigeant des consultants certifiés payés à des tarifs correspondants à des marchés occidentaux, et une infrastructure technique serveurs dédiés, connexion haut débit stable, équipe IT interne que les PME africaines ne possèdent généralement pas. Gartner [9] estime dans son Magic Quadrant 2022 pour les ERP Cloud dédiés aux entreprises de services que le coût total de possession de ces solutions sur cinq ans, pour une PME de cinquante employés, dépasse en général 30 millions de FCFA, un investissement hors de portée de 95 % des PME africaines formelles.

1.2.2.2. Les ERP open-source de deuxième niveau

Face aux ERP propriétaires, plusieurs solutions open-source ont progressivement trouvé un écho favorable auprès des PME et des startups africaines. Odoo, développé en Python par la société belge Odoo SA (anciennement OpenERP), est disponible depuis 2014 en deux éditions : une version Community entièrement gratuite et open-source, et une version Enterprise payante incluant des modules supplémentaires et un support officiel. Son architecture modulaire fondée sur le framework Odoo lui-même, distinct de Django permet d'activer uniquement les fonctionnalités nécessaires, réduisant apparemment la complexité d'adoption. Cependant, deux obstacles majeurs limitent son adoption dans les PME africaines. Le premier est le coût de déploiement : selon les intégrateurs locaux consultés lors de notre étude de terrain, le déploiement d'une configuration Odoo couvrant les modules RH, Comptabilité et CRM pour une PME camerounaise de vingt employés nécessite entre 300 000 et 2 000 000 FCFA en frais d'intégration, auxquels s'ajoutent des frais de maintenance annuels [9]. Le second obstacle est le support du SYSCOHADA : comme l'ont démontré Elbahri et ses collègues [2], le module comptable d'Odoo est conçu pour les normes IFRS et ne supporte le SYSCOHADA que via des modules communautaires développés bénévolement par des intégrateurs locaux, sans garantie de compatibilité lors des mises à jour majeures de la plateforme.

ERPNext, développé sur le framework Python Frappe par la société Frappe Technologies (Inde), affiche une position philosophique explicitement monolithique tout en proposant une organisation du code en applications Frappe indépendantes. Sa comptabilité multi-devise et ses fonctionnalités de paie configurables en font une option intéressante pour le contexte africain. Cependant, ERPNext ne dispose pas de module SYSCOHADA natif et son déploiement nécessite un prestataire technique capable de paramétrer le plan de comptes selon les exigences de l'OHADA. Dolibarr, développé en PHP par la communauté ERP CRM open source, est la solution la plus accessible techniquement son installation ne requiert qu'un serveur web et une base de

données MySQL mais son architecture monolithique classique ne permet aucune gouvernance des interfaces entre modules, et son support du SYSCOHADA est limité à quelques modules de la communauté française qui ne couvrent pas toutes les spécificités de l'Acte Uniforme OHADA.

Le tableau suivant synthétise les caractéristiques de ces solutions au regard des exigences de Formuloo OS.

Tableau 2: Comparaison des ERP disponibles sur le marché Africain

Critère	Odoo 17	ERPNext	Dolibarr	Formuloo OS
Architecture logicielle	Monolithe modulaire	Monolithe Frappe	Monolithe classique	Monolithe modulaire
Langage principal	Python propriétaire	Python Frappe	PHP	Python / Django 4.2
Support SYSCOHADA	Modules communautaires	Non natif	Module partiel	Natif dans le cœur
Gouvernance API Contract-First	Non	Non	Non	Oui drf-spectacular
Multi-tenant natif	Partiel	Non	Non	Oui shared schema
Déploiement sans intégrateur	Non	Difficile	Possible	Objectif de conception
API publique documentée	Oui	Oui	Partielle	Oui OpenAPI 3.1

Source : élaboré par l'auteur à partir d'Elbahri et al. , du cahier des charges Formuloo OS , et de la consultation des documentations officielles et dépôts GitHub d'Odoo, ERPNext et Dolibarr (consultation juin 2025).

L'analyse comparative de ce tableau révèle qu'aucune des trois solutions existantes ne répond simultanément aux trois exigences fondamentales du projet Formuloo OS : conformité native au SYSCOHADA dans le cœur du produit, architecture multi-tenant permettant la mutualisation des ressources d'infrastructure entre plusieurs PME clientes à moindre coût, et mécanisme de gouvernance contractuelle des API garantissant la cohérence entre les équipes de développement dans un contexte agile multi-modules. Cette triple lacune justifie le développement d'une solution nouvelle plutôt que l'adaptation d'une solution existante.

I.3. Gouvernance des API REST : fondements théoriques et travaux récents

I.3.1. Les API REST : définition, principes et histoire

I.3.1.1. Fondements architecturaux la thèse de Fielding (2000)

Le style architectural REST Representational State Transfer a été formalisé par Roy Thomas Fielding dans sa thèse de doctorat intitulée « Architectural styles and the design of network-based software architectures », soutenue en 2000 à l'Université de Californie à Irvine [14]. Fielding y propose un cadre théorique pour l'évaluation et la conception des architectures de systèmes distribués sur le Web. Ce travail est d'une importance historique majeure : Fielding était à l'époque l'un des principaux auteurs de la spécification HTTP/1.1, et sa thèse décrit rétrospectivement le style architectural qui avait guidé la conception du Web lui-même. REST n'est pas un protocole, une norme ou une spécification technique c'est un ensemble de contraintes architecturales dont le respect confère au système des propriétés désirables de performance, d'évolutivité, de simplicité, de portabilité des interfaces et de fiabilité.

Ces contraintes, au nombre de six, forment la définition complète du style REST. La première est l'architecture client-serveur : la séparation des responsabilités entre le client (interface utilisateur) et le serveur (stockage des données) permet d'améliorer la portabilité de l'interface utilisateur sur plusieurs plateformes et la scalabilité des composants serveur. La deuxième est l'absence d'état (statelessness) : chaque requête du client vers le serveur doit contenir toutes les informations nécessaires à son traitement, sans exploitation d'un contexte de session stocké côté serveur. La troisième est la mise en cache : les réponses doivent indiquer explicitement si elles peuvent être mises en cache, pour améliorer les performances et la scalabilité. La quatrième est l'interface uniforme, qui est la contrainte distinctive de REST : l'identification des ressources dans les requêtes par un URI, la manipulation des ressources par des représentations, les messages auto-descriptifs, et l'hypermédia comme moteur de l'état de l'application (HATEOAS). La cinquième est le système en couches (layered system), qui permet d'intercaler des composants intermédiaires proxys, passerelles, caches entre le client et le serveur sans que le client en soit informé. La sixième est le code à la demande, optionnelle, qui permet au serveur d'étendre le comportement du client en lui transmettant du code exécutable.

I.3.1.2. La spécification OpenAPI et son écosystème

La spécification OpenAPI anciennement connue sous le nom de Swagger jusqu'à son adoption par la Linux Foundation en 2016 et sa republication par l'OpenAPI Initiative est un format

standard de description des API REST lisible par les machines. Dans sa version 3.1 publiée en 2021 [15], elle permet de décrire formellement l'ensemble des caractéristiques d'une API : les endpoints disponibles et leurs chemins, les méthodes HTTP autorisées (GET, POST, PUT, PATCH, DELETE), les paramètres de requête et leurs types, les structures de données des corps de requête et de réponse sous la forme de schémas JSON Schema, les codes de statut HTTP possibles avec leurs significations, les mécanismes d'authentification et d'autorisation, et la documentation textuelle de chaque opération. Cette description formelle est à la fois lisible par les développeurs humains via des interfaces comme Swagger UI ou Redoc et exploitable automatiquement par des outils : générateurs de clients dans tous les langages de programmation, générateurs de code serveur, serveurs de mock, outils de test de conformité, et outils de linting.

Neumann et ses collègues [8] ont analysé 2 616 API REST publiques issues des catégories Transport, Finances, Gouvernement et Commerce, et concluent que la qualité de la spécification OpenAPI mesurée par des critères de complétude, de cohérence et de respect des conventions de nommage est le premier prédicteur de la qualité globale de l'API et de sa facilité d'adoption par les développeurs. Ils identifient six défauts récurrents dans les API analysées : l'absence de schémas de réponse pour les codes d'erreur, l'incohérence des conventions de nommage entre les endpoints, l'absence de description textuelle pour les paramètres, la sur-utilisation des types string sans contraintes de format, la non-conformité aux principes d'URI RESTful, et l'absence de versionnement. Ces défauts sont précisément ceux que le mécanisme de gouvernance de Formuloo OS, via CONVENTIONS.md et le pipeline `validate_docs`, cherche à prévenir de façon automatique.

I.3.2. L'approche Contract-First : principes, bénéfices et limites

I.3.2.1. Définition et origines de l'approche Contract-First

L'approche Contract-First également désignée API-First ou Design-First dans la littérature est née en réaction au mode de développement dominant qui consiste à écrire le code d'implémentation en premier et à générer la documentation API après coup (approche Code-First). Dans l'approche Contract-First, le contrat d'interface défini formellement en OpenAPI est rédigé, validé et partagé entre toutes les équipes avant que la première ligne de code d'implémentation ne soit écrite. Ce contrat devient la source de vérité unique et partagée à partir de laquelle toutes les équipes backend, frontend, mobile, équipes d'assurance qualité travaillent en parallèle. Lauret [16], dans son ouvrage *The Design of Web APIs* publié en 2019 chez Manning Publications, démontre que l'approche Contract-First est la seule capable de garantir la cohérence des interfaces

dans un contexte de développement multi-équipes agile, où la cadence des livraisons est élevée et les risques de divergence implicite entre équipes sont permanents.

L'approche Contract-First résout un problème fondamental que tout projet multi-équipes rencontre inévitablement : les hypothèses implicites. Lorsqu'une équipe frontend suppose que le champ représentant un numéro de téléphone s'appelle `phone` et que l'équipe backend l'a nommé `phone_number`, les deux équipes peuvent développer pendant des jours ou des semaines sans que l'incompatibilité soit détectée jusqu'au moment de l'intégration, où la régression se manifeste brutalement. Avec une approche Contract-First, cette incompatibilité est détectée au moment de la rédaction du contrat, avant que le moindre code ne soit écrit. Stoplight [17] quantifie cet avantage dans une étude portant sur cent équipes de développement ayant adopté l'approche Contract-First : le temps de débogage lié aux incompatibilités d'interface a été réduit en moyenne de 45 % dans les six mois suivant l'adoption, et le temps d'intégration entre équipes frontend et backend a été réduit de 30 %.

I.3.2.2. Bénéfices dans un contexte agile multi-équipes

Dans un projet agile comme Formuloo OS, où les sprints de deux semaines imposent une cadence de livraison soutenue et où quatre équipes backend Auth/Crm, Stock, Projet, backend RH/Comptabilité, Analytic, frontend Angular, mobile développent en parallèle des modules interdépendants, les bénéfices de l'approche Contract-First sont particulièrement marqués. Le premier bénéfice est le développement en parallèle réel. Une fois le contrat OpenAPI d'un module validé, l'équipe frontend peut démarrer son développement en utilisant un serveur de mock Prism dans le cas de Formuloo OS qui simule les réponses du backend en se fondant sur le contrat. Pendant ce temps, l'équipe backend implémente les endpoints réels. Les deux équipes travaillent simultanément sans dépendance bloquante, et l'intégration finale ne révèle aucune surprise puisque les deux implémentations sont conformes au même contrat.

Le deuxième bénéfice est la détection automatique des régressions de contrat. Dans Formuloo OS, `drf-spectacular` génère la spécification OpenAPI depuis le code backend par introspection, et `ng-openapi-gen` génère le client TypeScript depuis cette spécification côté frontend. Lorsqu'un développeur backend supprime un champ ou renomme un endpoint, la spécification OpenAPI change, `ng-openapi-gen` régénère le client TypeScript avec les nouvelles interfaces, et le compilateur TypeScript signale immédiatement les endroits du code frontend qui utilisaient l'ancienne interface avant même que le code modifié ne soit poussé en dépôt distant. Cette chaîne de détection automatique transforme une régression silencieuse potentielle en erreur

de compilation immédiate, visible par le développeur à la seconde où il introduit l'incompatibilité. Le troisième bénéfice est la documentation vivante : puisque la spécification OpenAPI est générée depuis le code réel par introspection, elle est structurellement en phase avec l'implémentation à tout moment, éliminant la dérive systématique entre documentation et code qui caractérise les projets où la documentation est maintenue manuellement.

I.3.2.3. Limites de l'approche Contract-First

L'approche Contract-First n'est pas sans limites. La première est la charge initiale de conception : rédiger un contrat OpenAPI de qualité avant d'écrire le code exige une maîtrise à la fois du domaine métier et du standard OpenAPI, et représente un investissement en temps non négligeable, particulièrement pour les équipes qui ne l'ont jamais pratiquée. Lauret [16] recommande de consacrer environ 20 % du temps de développement d'un module à la rédaction et à la validation du contrat, un investissement qui se récupère selon lui dès le premier sprint d'intégration. La deuxième limite est l'incomplétude de la validation automatique dans la version actuelle du mécanisme de Formuloo OS : le pipeline `validate_docs` vérifie la présence des artefacts contractuels mais non la conformité comportementale des implémentations vis-à-vis du contrat. Un endpoint peut être documenté conformément au contrat OpenAPI tout en retournant en pratique des données erronées. Cette limitation, identifiée honnêtement comme une lacune de la version actuelle, sera adressée en phase 2 par l'intégration de Schemathesis [19] un outil de test de conformité property-based qui génère automatiquement des requêtes depuis le contrat et vérifie que les réponses obtenues sont conformes aux schémas définis.

I.3.3. Les outils de gouvernance contractuelle

I.3.3.1. drf-spectacular : génération automatique OpenAPI depuis Django

`drf-spectacular` est une bibliothèque Python développée par la communauté open-source Django REST Framework et disponible depuis 2020. Dans sa version 0.27.2 intégrée dans Formuloo OS, elle génère automatiquement la spécification OpenAPI 3.1 d'une API Django REST Framework par introspection du code source : elle analyse les ViewSets, les Serializers, les champs, les permissions, les filtres et les paginateurs pour en déduire la structure complète de l'API [20]. Le développeur peut enrichir cette génération automatique via le décorateur `@extend_schema`, qui permet d'ajouter des métadonnées non déductibles par introspection : exemples de requêtes et de réponses, descriptions textuelles des paramètres, surcharge du schéma de réponse pour les cas complexes. La spécification générée est exposée en temps réel sur deux

endpoints : `/api/schema/` retourne le fichier YAML brut, consommable par tous les outils de l'écosystème OpenAPI ; `/api/docs/` expose l'interface Swagger UI interactive permettant à tout développeur de consulter la documentation et de tester les endpoints directement depuis son navigateur.

L'avantage décisif de `drf-spectacular` par rapport à une documentation rédigée manuellement est sa cohérence structurelle permanente avec le code. Une modification du code ajout d'un champ dans un sérialiseur, suppression d'un endpoint, changement d'un paramètre se répercute immédiatement dans la spécification OpenAPI sans aucune action manuelle, garantissant que la documentation est toujours en phase avec l'implémentation réelle. Cette propriété est fondamentale dans un contexte agile où le code évolue à chaque sprint : la documentation ne peut jamais être en retard d'une version.

I.3.3.2. Spectral : linting des spécifications OpenAPI

Spectral est un outil open-source de linting pour les spécifications OpenAPI, développé par Stoplight et disponible depuis 2018. Il permet de définir et d'appliquer automatiquement des règles de qualité personnalisables à une spécification OpenAPI : vérification des conventions de nommage des endpoints (les noms de ressources doivent être au pluriel, les verbes sont interdits dans les chemins), cohérence des codes HTTP retournés (les opérations POST doivent retourner 201, les opérations DELETE doivent retourner 204), présence obligatoire des schémas de réponse pour tous les codes d'erreur, respect des règles de versionnement, et toute autre contrainte personnalisée adaptée au contexte du projet. Dans le contexte de Formuloo OS, Spectral peut être configuré avec un ruleset africain couvrant les exigences spécifiques au contexte : support obligatoire des devises XAF et XOF dans les champs monétaires, présence obligatoire des champs de pagination cursor-based dans toutes les réponses de liste, et conformité des codes d'erreur aux conventions définies dans `CONVENTIONS.md`. L'intégration de Spectral dans le pipeline CI permettrait de bloquer toute spécification OpenAPI ne respectant pas ces règles avant qu'elle n'atteigne la branche principale du dépôt.

I.3.3.3. Schemathesis : tests de conformité property-based

Schemathesis est un outil de test de conformité des API REST fondé sur l'approche property-based testing, développé par Dmitry Dygalo et décrit par Hatfield-Dodds et Dygalo [19] dans un article publié en 2022 dans le Journal of Open Source Software. L'approche property-based testing consiste non pas à écrire des cas de test unitaires manuels chacun couvrant un scénario spécifique prédéfini par le développeur mais à définir des propriétés générales que le système doit

respecter, puis à laisser un moteur de génération automatique produire des milliers de cas de test aléatoires conformes à ces propriétés et vérifier que le système les respecte dans tous les cas. Appliqué aux API REST, Schemathesis lit la spécification OpenAPI d'une API, génère automatiquement des milliers de requêtes HTTP valides au regard du contrat en variant les valeurs des paramètres, les types de données, les combinaisons de champs obligatoires et optionnels les envoie au serveur réel, et vérifie que les réponses obtenues sont conformes aux schémas définis dans le contrat. Hatfield-Dodds et Dygalo [19] rapportent des résultats supérieurs aux tests manuels dans 78 % des API Django REST Framework étudiées, avec une capacité particulière à détecter des cas limites non couverts par les suites de tests conventionnelles.

I.3.3b. La sécurité des API REST : authentification et multi-tenancy

I.3.3b.1. L'authentification par jeton JWT

La sécurité des API REST repose aujourd'hui majoritairement sur le protocole OAuth 2.0, formalisé dans la RFC 6749 publiée par l'IETF en 2012, et sur le format de jeton JSON Web Token (JWT), défini dans la RFC 7519 de 2015. Un JWT est un objet JSON encodé en Base64Url et signé cryptographiquement, structuré en trois parties séparées par des points : l'en-tête (header) qui indique l'algorithme de signature, la charge utile (payload) qui contient les claims assertions sur l'identité de l'utilisateur et ses droits et la signature qui garantit l'intégrité du jeton. L'avantage fondamental du JWT pour les API REST est sa nature sans état (stateless) : le serveur n'a pas besoin de maintenir une session pour chaque utilisateur connecté il lui suffit de vérifier la validité de la signature du jeton pour authentifier la requête, sans consulter une base de données de sessions. Cette propriété est parfaitement alignée avec la contrainte de statelessness de l'architecture REST définie par Fielding [14].

Dans le contexte multi-tenant de Formuloo OS, le JWT joue un rôle supplémentaire au-delà de la simple authentification : il porte le `tenant_id` de l'organisation à laquelle appartient l'utilisateur dans sa charge utile, permettant à chaque service backend d'identifier automatiquement le contexte organisationnel de la requête sans effectuer de requête supplémentaire en base de données. Cette conception, documentée dans l'ADR-002 de Formuloo OS, réduit la latence des requêtes et simplifie l'architecture en évitant la nécessité d'un service d'identité consulté à chaque requête. La bibliothèque `django-rest-framework-simplejwt` version 5.3.1, intégrée dans Formuloo OS, fournit l'infrastructure de génération et de validation des JWT, que notre composant `TenantJWTAuthentication` étend pour y ajouter la logique d'extraction et de propagation du `tenant_id`.

I.3.3b.15. Le versionnement et la compatibilité ascendante des API

Le versionnement des API constitue l'un des problèmes les plus délicats de la gouvernance contractuelle, car il met en tension deux objectifs contradictoires : l'évolution du système pour ajouter de nouvelles fonctionnalités ou corriger des comportements erronés, et la stabilité des interfaces pour les consommateurs existants qui ont bâti leur code sur les contrats actuels. La littérature distingue deux approches principales. L'approche de versionnement par URI inclure le numéro de version dans le chemin de l'API, par exemple `/api/v1/contacts/` et `/api/v2/contacts/` présente l'avantage de la clarté et de la coexistence simple de plusieurs versions. L'approche de versionnement par en-tête HTTP Accept le consommateur indique la version souhaitée via un en-tête `Accept: application/vnd.formuloo.v2+json` est plus conforme aux principes REST de Fielding [14] mais plus complexe à implémenter et à déboguer. Lauret [16] recommande le versionnement par URI pour les projets en phase de démarrage, en notant que la migration vers le versionnement par en-tête peut être effectuée ultérieurement sans rupture pour les consommateurs.

Un concept fondamental dans le versionnement des API est celui de compatibilité ascendante (backward compatibility) : une modification est dite compatible si elle ne brise aucun code client existant. Stoplight [17] propose une taxonomie des modifications selon leur niveau de rupture : les modifications non rupturales (ajout d'un nouveau champ optionnel dans une réponse, ajout d'un nouveau endpoint, extension d'un enum en ajoutant de nouvelles valeurs) peuvent être déployées sans changer le numéro de version majeure ; les modifications rupturales (suppression d'un champ, renommage d'un champ, changement du type d'un champ, suppression d'un endpoint, modification des codes HTTP retournés) exigent un incrément du numéro de version majeure et la maintien de l'ancienne version pendant une période de dépréciation explicitement communiquée aux consommateurs. Dans Formuloo OS, `CONVENTIONS.md` formalise ces règles de versionnement applicables à tous les modules, et le pipeline `validate_docs` vérifie que toute merge request modifiant l'API est accompagnée d'une mise à jour de la version dans la spécification OpenAPI.

I.3.3b.2. Le multi-tenancy dans les systèmes SaaS

Le multi-tenancy est un mode d'architecture dans lequel une instance unique d'un logiciel sert simultanément plusieurs clients appelés tenants en maintenant une isolation stricte de leurs données. Cette approche est fondamentale pour les solutions SaaS (Software as a Service) car elle permet de mutualiser les coûts d'infrastructure entre tous les clients, rendant le service économiquement viable pour des tarifs d'abonnement accessibles. La littérature identifie trois

modèles principaux de multi-tenancy pour les bases de données relationnelles. Le premier est le modèle de base de données séparée par tenant : chaque organisation cliente dispose de sa propre instance de base de données, garantissant une isolation maximale mais multipliant les coûts d'infrastructure et complexifiant les migrations de schéma qui doivent être appliquées séparément à chaque base. Le deuxième est le modèle de schéma séparé par tenant : toutes les organisations partagent le même moteur de base de données mais disposent chacune de son propre schéma PostgreSQL, offrant un bon niveau d'isolation avec des coûts d'infrastructure réduits mais complexifiant les requêtes analytiques cross-tenant. Le troisième est le modèle de schéma partagé avec discrimination par colonne (shared schema) : toutes les organisations partagent les mêmes tables, et chaque enregistrement porte une colonne `tenant_id` qui identifie son organisation propriétaire.

C'est ce troisième modèle que Formuloo OS a retenu dans son ADR-002, pour trois raisons documentées. Premièrement, le coût d'infrastructure est minimal : une seule instance PostgreSQL suffit pour héberger les données de toutes les organisations clientes, quelle que soit leur taille. Deuxièmement, les migrations de schéma sont applicables en une seule opération à toutes les organisations simultanément. Troisièmement, les requêtes analytiques cross-tenant pour les tableaux de bord d'administration de la plateforme sont nativement possibles sans fédération de requêtes entre plusieurs bases ou schémas. Le risque principal de ce modèle la fuite de données entre organisations par oubli du filtre `tenant_id` dans une requête est adressé dans Formuloo OS par le composant `TenantFilterMixin`, qui ajoute automatiquement ce filtre sur chaque `QuerySet` Django sans aucune intervention manuelle du développeur.

I.3.4. Travaux récents et analyse critique

I.3.4.0. La méthode Agile Scrum dans les projets de développement logiciel

La méthode Agile Scrum, formalisée par Ken Schwaber et Jeff Sutherland dans leur premier Scrum Guide publié en 2010 [18], est aujourd'hui le framework de développement itératif le plus utilisé dans l'industrie du logiciel. Une enquête annuelle du cabinet VersionOne publiée en 2020 indique que 81 % des équipes de développement logiciel dans le monde utilisent Scrum ou une variante Scrum comme principal framework de gestion de projet. Scrum repose sur trois piliers fondamentaux transparence, inspection et adaptation qui s'incarnent dans des artefacts (product backlog, sprint backlog, incrément), des cérémonies (sprint planning, daily scrum, sprint review, sprint retrospective), et des rôles (Product Owner, Scrum Master, équipe de développement). La cadence de livraison en sprints courts généralement deux à quatre semaines impose une discipline

de priorisation, d'estimation et de livraison continue qui s'est avérée particulièrement efficace dans les contextes d'incertitude et d'évolution rapide des besoins.

Dans le contexte spécifique du développement d'un ERP multi-modules par plusieurs équipes travaillant en parallèle, Schwaber et Sutherland [18] recommandent l'instanciation à grande échelle de Scrum connue sous le nom de Scaled Scrum ou LeSS (Large-Scale Scrum), où plusieurs équipes travaillent sur des composants distincts d'un même produit en synchronisant leurs sprints et en partageant un product backlog unique. La gouvernance des interfaces entre équipes le sujet central de notre travail est explicitement identifiée dans le Scrum Guide 2020 comme l'une des principales sources de risque dans les contextes multi-équipes : sans mécanisme de synchronisation des contrats d'interface, les équipes peuvent passer plusieurs sprints à développer des composants incompatibles entre eux, ne découvrant l'incompatibilité qu'au moment de la sprint review d'intégration. C'est précisément ce problème que le mécanisme de gouvernance Contract-First de Formuloo OS cherche à prévenir de façon systématique.

I.3.4.1. Travaux sur la gouvernance continue des API

Medjaoui et ses collègues [3], dans leur ouvrage Continuous API Management publié en 2021 chez O'Reilly, proposent le modèle le plus complet de gouvernance en continu des API disponible dans la littérature. Leur modèle s'articule autour du cycle de vie complet d'une API définition, publication, réalisation, opération, consommation, et retrait et recommande d'automatiser autant que possible les transitions entre ces phases. Les auteurs identifient trois dimensions de gouvernance complémentaires : la gouvernance Design (qualité du contrat OpenAPI, cohérence des conventions, revue par les pairs), la gouvernance Deploy (pipeline CI/CD, tests de conformité, détection des breaking changes), et la gouvernance Operate (monitoring des patterns de consommation, détection des anomalies, gestion des dépréciations). Ce modèle est exhaustif, mais il suppose des organisations disposant d'équipes dédiées à la gouvernance des API et d'une infrastructure DevOps mature. Sa principale limite pour notre contexte est l'absence de prise en compte des contraintes de petites équipes africaines travaillant avec des ressources limitées.

Bogner et ses collègues [10] ont conduit en 2019 une étude qualitative auprès de seize équipes utilisant des architectures microservices dans des entreprises européennes, et identifient les pratiques de gouvernance des interfaces les plus efficaces. Ils montrent que les équipes ayant adopté un contrat d'interface explicite sous forme de spécification OpenAPI versionnée dans un dépôt partagé présentent significativement moins de problèmes d'incompatibilité lors des

intégrations que les équipes ne disposant que d'une documentation informelle. Cependant, ils soulignent également que la gouvernance des API impose une discipline et une rigueur supplémentaires que certains développeurs perçoivent initialement comme une charge, et que l'adoption réussie nécessite un effort d'acculturation de l'équipe. Ces observations ont guidé le choix de Formuloo OS d'automatiser le maximum de vérifications de gouvernance dans le pipeline CI pour réduire la charge cognitive des développeurs et rendre la conformité invisible pour celui qui respecte les règles.

I.3.4.1b. La convergence entre Agile et gouvernance contractuelle des API

La tension entre la flexibilité revendiquée par la méthode Agile et la rigueur exigée par la gouvernance contractuelle des API est souvent perçue comme une contradiction. Les méthodes agiles valorisent, dans le Manifeste Agile publié en 2001, « les logiciels opérationnels plutôt qu'une documentation exhaustive » et « la réponse au changement plutôt que le suivi d'un plan ». La gouvernance contractuelle des API, avec ses spécifications OpenAPI, ses conventions formalisées et ses pipelines de validation automatique, pourrait sembler à l'opposé de cet esprit. Cependant, Medjaoui et ses collègues [3] démontrent que cette tension est largement illusoire lorsque la gouvernance est correctement automatisée : une gouvernance dont la vérification est intégralement automatisée dans le pipeline CI n'impose aucune friction supplémentaire au développeur qui respecte les règles il ne la perçoit tout simplement pas. La friction apparaît uniquement lorsque le développeur s'écarte des règles, et c'est précisément ce signal d'alerte que la gouvernance est conçue pour produire.

Schwaber et Sutherland [18] eux-mêmes, dans la version 2020 du Scrum Guide, reconnaissent que dans les contextes multi-équipes, la Definition of Done la liste des critères qu'un incrément doit satisfaire pour être considéré comme terminé doit inclure des critères qui dépassent le périmètre technique d'une seule équipe. La vérification automatique de la conformité au contrat OpenAPI partagé est exactement ce type de critère transverse : elle garantit que chaque équipe, en livrant son incrément, n'a pas introduit de régression dans les contrats d'interface qui pourraient bloquer les autres équipes. L'intégration de la validation contractuelle dans la Definition of Done de Formuloo OS aucune merge request ne rejoint la branche principale sans que le pipeline `validate_docs` soit passé est ainsi pleinement cohérente avec l'esprit du Scrum Guide 2020, qui met l'accent sur la responsabilité collective de l'ensemble de l'équipe produit envers la qualité de l'incrément livré.

I.3.4.2. Travaux sur la qualité des API REST en pratique

Neumann et ses collègues [8], dans leur analyse de 2 616 API REST publiées en 2021, fournissent la base empirique la plus solide disponible sur la qualité réelle des API REST en production. Leurs résultats révèlent que seulement 41 % des API analysées disposaient d'une spécification OpenAPI complète et valide, que 67 % présentaient au moins un défaut dans leurs schémas de réponse, et que les API les mieux notées en termes de qualité étaient significativement plus adoptées par les développeurs tiers que les API de faible qualité. Ces résultats empiriques confirment la valeur économique d'une bonne gouvernance contractuelle : une API bien documentée et cohérente attire davantage de consommateurs et génère moins de questions de support. Dans le cas de Formuloo OS, dont l'API publique a vocation à être consommée par des intégrateurs tiers africains développeurs d'applications mobiles, cabinets comptables intégrant leurs outils, partenaires commerciaux la qualité de la spécification OpenAPI constitue directement un avantage compétitif.

1.3.4.3. Lacunes identifiées et positionnement de notre contribution

L'analyse critique des travaux recensés révèle deux lacunes majeures que notre projet entend combler. La première est l'absence de modèle de gouvernance des API REST adapté aux contraintes des petites équipes africaines. Tous les travaux recensés Medjaoui et al. [3], Bogner et al. [10], Neumann et al. [8] ont été conduits dans des contextes d'équipes de dix développeurs ou plus, dans des organisations disposant d'une infrastructure DevOps mature. Les recommandations qui en découlent sont calibrées pour ces contextes et ne peuvent être transposées directement à une équipe de deux développeurs backend déployant sous Docker Compose sans Kubernetes. Il n'existe à notre connaissance aucun modèle de gouvernance contractuelle adapté aux spécificités des start-ups africaines : ressources humaines limitées, infrastructure à faible coût, besoin de solutions simples mais robustes, et exigences réglementaires spécifiques (SYSCOHADA, devises CFA).

Hatfield-Dodds et Dygalo [19], dans leur article décrivant Schemathesis publié en 2022 dans le Journal of Open Source Software, rapportent des résultats empiriques significatifs sur l'efficacité des tests de conformité property-based pour les API Django REST Framework. Dans leur étude portant sur 25 API Django en production, Schemathesis a détecté des comportements non conformes au contrat OpenAPI dans 78 % des cas, dont une majorité n'était pas couverte par les suites de tests unitaires ou d'intégration existantes. Les types de non-conformité les plus fréquents étaient les erreurs HTTP 500 non documentées dans le contrat (37 % des cas), les réponses dont le schéma JSON ne correspondait pas au contrat pour certaines combinaisons de paramètres (28 % des cas), et les comportements asymétriques entre les opérations de création et

de lecture d'une ressource (13 % des cas). Ces résultats justifient pleinement l'investissement dans l'intégration de Schemathesis dans le pipeline CI de Formuloo OS lors de la phase 2 du projet.

La seconde lacune est le manque de mise en œuvre référencée et documentée de l'approche Contract-First dans le cadre d'un ERP multi-modules développé en mode agile par une petite équipe. Lauret [16] et Stoplight [17] fournissent d'excellentes références conceptuelles sur l'approche Contract-First, mais leurs exemples concrets portent sur des API simples à un ou deux modules, non sur des systèmes ERP à six modules interdépendants. Notre contribution consiste à proposer, mettre en œuvre et évaluer un mécanisme de gouvernance contractuelle opérationnel fondé sur `drf-spectacular`, `CONVENTIONS.md`, quatre ADR et un pipeline GitLab CI `validate_docs` directement applicable dans des conditions similaires à celles d'une start-up africaine développant un ERP multi-modules. Ce mécanisme, documenté dans le chapitre suivant, est intégralement open-source et répliquable par toute équipe africaine se trouvant dans une situation comparable.

I.3.5. Protection des données et enjeux réglementaires pour les API multi-tenant

La mise en œuvre d'une architecture multi-tenant dans un ERP destiné aux PME africaines soulève des enjeux de protection des données qui méritent d'être examinés dans le cadre de la revue de la littérature. En Afrique subsaharienne, la protection des données personnelles est encadrée par des législations nationales dont le niveau de maturité varie considérablement d'un pays à l'autre. Le Cameroun a adopté la loi n° 2010/012 du 21 décembre 2010 relative à la cybersécurité et à la cybercriminalité, qui inclut des dispositions sur la protection des données à caractère personnel mais dont le cadre réglementaire reste moins prescriptif que le Règlement Général sur la Protection des Données européen (RGPD), entré en vigueur en mai 2018. La Côte d'Ivoire, le Sénégal, le Bénin et le Burkina Faso tous membres de l'OHADA disposent de législations nationales sur la protection des données inspirées de l'approche européenne, avec des autorités de contrôle dédiées.

Pour une solution ERP multi-tenant comme Formuloo OS, ces enjeux réglementaires se traduisent par trois exigences techniques. La première est l'isolation des données entre tenants : aucune organisation cliente ne doit pouvoir accéder, même accidentellement, aux données d'une autre organisation. Cette exigence est adressée dans Formuloo OS par le `TenantFilterMixin` qui garantit l'isolation au niveau de chaque requête SQL. La deuxième exigence est la traçabilité des accès : toute consultation ou modification de données à caractère personnel doit être journalisée de façon immuable et associée à l'identité de l'utilisateur qui l'a effectuée. Cette exigence est adressée par le module `AuditLog` d'Formuloo OS qui enregistre chaque opération avec l'identifiant de

l'utilisateur, l'organisation, l'action effectuée et un horodatage. La troisième exigence est le droit à l'oubli : l'architecture doit permettre la suppression complète et vérifiable des données d'un tenant sans impacter les données des autres tenants. Le modèle shared schema de Formuloo OS facilite cette opération puisque toutes les données d'un tenant partagent le même `tenant_id` une suppression ciblée sur cet identifiant efface l'ensemble des données de l'organisation concernée de façon atomique.

Neumann et ses collègues [8] soulignent que les API REST sont aujourd'hui le principal vecteur d'accès aux données personnelles dans les systèmes modernes, et que leur gouvernance constitue donc un enjeu de conformité réglementaire autant qu'un enjeu technique. Une API mal documentée, dont les endpoints ne spécifient pas clairement quelles données personnelles sont exposées et dans quelles conditions, est non seulement difficile à maintenir mais également difficile à auditer pour vérifier sa conformité aux exigences réglementaires applicables. C'est une raison supplémentaire, au-delà des enjeux de qualité technique, pour lesquels la gouvernance contractuelle des API via OpenAPI constitue une bonne pratique indispensable dans tout système SaaS traitant des données à caractère personnel en Afrique.

I.4. Environnement de Travail et Cadrage du Thème de Mémoire

I.4.1. Présentation de l'entreprise Formuloo

Notre stage de fin d'études de Master 2 s'est déroulé au sein de l'entreprise Formuloo, une start-up technologique camerounaise spécialisée dans le développement de solutions logicielles de gestion pour les entreprises africaines. Formuloo a été fondée avec pour ambition de combler le fossé numérique qui sépare les petites et moyennes entreprises africaines des outils de gestion modernes auxquels leurs homologues occidentales ont accès. L'entreprise inscrit son activité dans la dynamique croissante de l'entrepreneuriat technologique africain, dont les acteurs cherchent à concevoir des solutions pensées depuis l'Afrique pour l'Afrique, en partant des contraintes réelles des utilisateurs cibles plutôt que d'adapter des produits conçus pour d'autres contextes. Ses activités principales se déclinent autour de quatre axes : le développement de logiciels de gestion métier adaptés aux PME africaines, la conception et la mise en œuvre d'architectures backend cloud-native, la formation des équipes de développement aux meilleures pratiques d'ingénierie logicielle, et la prestation de services d'accompagnement technique pour les entreprises souhaitant digitaliser leurs processus.

L'organisation de l'équipe de développement de Formuloo OS s'articule autour d'un Product Owner assuré par l'encadreur professionnel, d'un groupe de Scrum Masters composé de stagiaires en Master 2, et d'une équipe de développement répartie en quatre pôles spécialisés : un pôle backend en charge des modules Auth/IAM, Ressources Humaines, Comptabilité et CRM, un pôle frontend en charge de l'application Angular SPA, et un pôle mobile en charge de l'application native. Cette organisation multi-équipes en mode agile, avec des sprints de deux semaines et des livraisons régulières, constitue le contexte opérationnel direct dans lequel s'est inscrit notre travail.

I.4.2. Présentation du projet Formuloo OS

Formuloo OS est le produit phare de l'entreprise Formuloo. C'est une suite ERP cloud-native, open-source et multi-tenant, destinée aux petites et moyennes entreprises de la zone CFA. Le cahier des charges du projet [5], rédigé en amont du démarrage des développements, fixe les ambitions fonctionnelles et techniques du produit : couvrir les six domaines métier essentiels à la gestion d'une PME africaine : Authentification et gestion des droits, Ressources Humaines, Comptabilité conforme au SYSCOHADA, Gestion de la Relation Client, Gestion des Stocks et Analyse décisionnelle dans un système unique accessible depuis n'importe quel dispositif connecté, sans investissement initial en infrastructure et sans dépendance à un intégrateur certifié. Formuloo OS est développé en Python avec le framework Django 4.2 LTS et Django REST Framework 3.14, déployé sous Docker Compose, et versionné sur GitLab avec un pipeline d'intégration continue.

I.4.3. Cadrage du thème de mémoire

C'est au cours des premières semaines de notre prise de poste, lors d'une réunion de sprint planning qui réunissait l'ensemble des équipes, que nous avons pris conscience de la problématique centrale de ce mémoire. Le Product Owner présentait le backlog du sprint P1 lorsqu'un désaccord a émergé entre le pôle frontend et le pôle backend sur le format exact des données attendues dans la réponse de l'endpoint d'authentification : le frontend supposait que le jeton d'accès serait retourné dans un champ nommé `access_token`, tandis que le backend l'avait implémenté sous le nom `token`. Cette incompatibilité, découverte par hasard pendant la réunion, aurait pu rester silencieuse pendant plusieurs jours si les deux équipes avaient continué à développer indépendamment. Elle illustre de façon concrète le risque que représente l'absence de contrat d'interface formalisé dans un projet multi-équipes développé en mode agile.

Cette expérience directe a orienté notre contribution vers la problématique de la gouvernance des contrats d'interface API : comment garantir que les quatre équipes travaillant en parallèle partagent à tout moment la même compréhension des interfaces entre les composants du système, et comment détecter automatiquement toute dérive avant qu'elle n'atteigne l'environnement de production ? Notre thème —Conception et mise en œuvre d'une architecture backend orientée services avec gouvernance contractuelle des API REST pour un ERP cloud-native multi-tenant destiné aux PME africaines : cas de Formuloo OS est né directement de cette observation de terrain, au carrefour des problématiques de digitalisation des PME africaines que nous venions d'analyser dans la littérature, et des défis opérationnels concrets que nous vivions quotidiennement dans notre équipe de développement.

Au terme de ce premier chapitre, nous avons posé les fondements théoriques et contextuels de notre travail selon trois axes complémentaires. L'analyse du contexte des PME africaines a mis en évidence l'ampleur et les causes profondes du déficit de digitalisation, les enjeux spécifiques du référentiel SYSCOHADA, et la valeur fondamentale qu'un ERP intégré peut apporter aux PME de la zone CFA. L'étude comparative des architectures logicielles a démontré que le monolithe modulaire constitue le meilleur compromis architectural pour notre contexte, une conclusion renforcée par l'analyse des ERP existants qui révèle l'absence de solution répondant simultanément aux exigences de conformité SYSCOHADA, de multi-tenancy natif et de gouvernance contractuelle des API. La revue approfondie des travaux sur la gouvernance des API REST et l'approche Contract-First a permis d'identifier deux lacunes précises que notre projet ambitionne de combler : l'absence de modèle adapté aux petites équipes africaines, et le manque de retours d'expérience documentés sur l'application du Contract-First dans un ERP agile multi-modules. Le chapitre suivant décrit les matériaux et les méthodes qui ont guidé notre réponse à ces lacunes dans le cadre du projet Formuloo

CHAPITRE II : MATÉRIELS ET MÉTHODES

Ce chapitre décrit avec précision l'ensemble des ressources mobilisées et des méthodes adoptées pour mener à bien la conception et la mise en œuvre de l'architecture backend de Formuloo OS. Il présente d'abord les outils matériels, logiciels et humains utilisés au cours du projet, puis expose la méthodologie de développement retenue, les méthodes de conception architecturale et la modélisation UML complète du système. L'objectif de ce chapitre est de permettre à tout praticien spécialisé en génie logiciel de reproduire le travail réalisé et d'en évaluer la validité dans un contexte similaire.

I. Matériels

I.1. Outils matériels

La réalisation du projet Formuloo OS nécessite un environnement matériel adapté, tant pour le développement des composants backend que pour leur déploiement, leurs tests et leur intégration continue. Le tableau ci-dessous présente les ressources matérielles utilisées dans le cadre de ce projet.

Tableau 3:Outils Matériels utilisé dans le projet

Matériel	Caractéristiques techniques	Rôle dans le projet
Ordinateur portable de développement	Processeur Intel Core i7-865G7, 2,80 GHz ; RAM 8 Go DDR4 ; SSD 512 Go ; OS Ubuntu 22.04 LTS	Développement du backend Django, exécution des conteneurs Docker, tests unitaires et d'intégration, accès au dépôt GitLab
Serveur de déploiement continu (CI/CD)	GitLab Runner auto-hébergé, 4 vCPU, RAM 8 Go, SSD 100 Go, Ubuntu 22.04	Exécution automatique du pipeline validate_docs à chaque merge request — vérification des 6 artefacts de gouvernance en ~42 secondes (pipeline #230 PASSED)
Connexion Internet haut débit	Connexion fibre optique, débit montant 50 Mbps	Synchronisation du dépôt GitLab, téléchargement des images Docker, accès à la documentation officielle

Source : par nos soins.

I.2. Outils logiciels

Le développement de Formuloo OS mobilise un écosystème logiciel structuré autour de plusieurs catégories fonctionnelles complémentaires. Le tableau suivant présente l'ensemble des outils logiciels utilisés, leurs versions, et leur rôle précis dans le projet.

Tableau 4:Recapitulatifs des outils logiciels utilises dans le projet

Outil	Nature	Version	Rôle dans le projet
Python 	Langage de programmation	3.12.3	Langage principal du backend
Django	Framework web	4.2.13 LTS	ORM, routage, administration, migrations de base de données
Django REST Framework	Bibliothèque API REST	3.14.0	Exposition des endpoints REST ViewSets, Serializers, permissions, pagination
drf-spectacular	Générateur OpenAPI	0.27.2	Génération automatique de la spécification OpenAPI 3.1 par introspection du code DRF
djangorestframework-simplejwt	Authentification JWT	5.3.1	Gestion des tokens JWT access token (15 min), refresh token (7 jours)
PostgreSQL	SGBDR	15-alpine	Base de données relationnelle principale hared schema multi-tenant
Redis	Cache + broker	7-alpine	Cache des sessions et des réponses API fréquentes ; broker de messages pour Celery
Celery	File de tâches asynchrones	5.3.6	Communications asynchrones inter-modules via Signal Django
Docker + Docker Compose	Containerisation	24.x	Isolation de chaque service en conteneur, reproductibilité des environnements
Nginx	Passerelle API (Gateway)	1.25	Point d'entrée unique — routing, SSL, rate limiting, CORS, propagation JWT
GitLab CI Runner	Pipeline CI/CD	19.0.1	Exécution automatique du pipeline validate_docs à chaque merge request
Prometheus	Monitoring des métriques		Collecte les métriques de tous les services (latence, taux d'erreur, mémoire, requêtes DB) exposées via /metrics/ sur chaque service Django.
Grafana	Dashboards de visualisation		3 dashboards auto-provisionnés : vue globale (formuloo_overview), métriques

Outil	Nature	Version	Rôle dans le projet
			CRM, métriques Stock. Accessible sur http://localhost:3000 .
Alertmanager	Gestion des alertes		Routage et notification des 6 alertes configurées : HighErrorRate, HighLatency, ServiceDown, CeleryQueueDepth, StockMoveFailures, AuthFailureSpike [FOR16A26-1032]
OpenTelemetry	Traçage distribué		Instrumentation Django et PostgreSQL pour la corrélation logs/métriques/traces compatible Jaeger/Tempo
django-prometheus	Exposition métriques Django		Middleware ajouté sur les 5 services (Auth, RH, Compta, CRM, Stock) pour exposer les métriques Prometheus
Visual Studio Code	IDE	1.89	Développement du code backend, extensions Python, Git, Docker
Draw.io (diagrams.net)	Outil de diagrammes UML	24.x	Conception de tous les diagrammes UML et de l'architecture globale, versionnés en XML dans le dépôt GitLab
Figma	Outil de design collaboratif		Partage des diagrammes d'architecture avec les équipes backend, frontend Angular et mobile
Postman	Test des API REST	8.x	Test manuel des endpoints, vérification de la conformité des réponses au contrat OpenAPI

Source : fichiers requirements.txt et docker-compose.yml du dépôt GitLab Formuloo OS, ticket FOR16A26-1032, juillet 2025.

Il est important de souligner que la stack d'observabilité Prometheus, Grafana, Alertmanager et OpenTelemetry est opérationnelle depuis le commit 8d4fa2c (feat(observability): monitoring Prometheus + Grafana + OpenTelemetry) livré sur la branche cabrel-kengne. Cette implémentation répond aux quatre critères d'acceptation du ticket FOR16A26-1032 : AC1 (logs structurés et métriques exposés), AC2 (dashboards Grafana par service), AC3 (alertes configurées sur les anomalies réelles) et AC4 (corrélation logs/métriques/traces via OpenTelemetry).

I.3. Ressources humaines

Les ressources humaines constituent un facteur déterminant dans la réussite d'un projet de développement logiciel. Le tableau suivant présente les différents intervenants du projet Formuloo OS et leurs rôles respectifs.

Tableau 5: Intervenants du Projet Formuloo OS

Intervenant	Fonction	Rôle dans le projet
M. Leger Foposse	Product Owner / Superviseur professionnel	Définit et priorise le backlog produit à partir du cahier des charges [5], valide les incréments en sprint review, arbitre les décisions architecturales
M. DJE MAN Dieudonné et Pr KAMMOGNE SOUP Alain	Encadrant académique	Orienté la démarche de recherche, valide la problématique et les hypothèses, assure la qualité scientifique du mémoire
KENGNE Cabrel	Developpeur Backend	Conception de l'architecture backend orientée services, implémentation des modules Auth/IAM, CRM, Stock, Project, Analytic mise en place du mécanisme de gouvernance Contract-First, rédaction des 4 ADR, pipeline validate_docs, observabilité Prometheus/Grafana
Leticia	Développeuse backend	Implémentation des modules Ressources Humaines et Comptabilité, conventions SYSCOHADA, tests d'intégration
Loïc	Développeur frontend Angular	Développement de l'interface SPA Angular, consommation de l'API REST via OpenAPI, génération du client TypeScript ng-openapi-gen
Samy Bodio	Développeur mobile	Développement de l'application mobile native, consommation des endpoints REST de l'API publique Formuloo OS

Source : par nos soins, cahier des charges Formuloo OS.

I.4. Estimation des coûts du projet

I.4.1. Méthode d'estimation retenue et comparaison avec les alternatives

L'estimation de l'effort et des coûts d'un projet logiciel est une étape critique qui conditionne la crédibilité du plan de réalisation. Plusieurs méthodes d'estimation sont disponibles dans la littérature. Nous avons comparé trois approches principales avant de retenir celle adaptée à notre contexte.

Tableau 6: Comparaison des méthodes d'estimation du cout logiciel

Méthode	Principe	Avantages	Inconvénients	Applicabilité à Formuloo OS
COCOMO (Constructive Cost Model)	Modèle paramétrique fondé sur des équations mathématiques calibrées sur des milliers de projets industriels. L'effort se calcule à partir du nombre de lignes de code estimées (KLOC) et de facteurs d'ajustement	Précision éprouvée pour les grands projets ; modèle standard reconnu dans l'industrie	Nécessite une estimation précise du volume de code en amont, difficile à faire en début de projet agile. Calibration sur des projets occidentaux, peu adaptée aux PME africaines	Non retenue : le nombre de lignes de code de Formuloo OS n'est pas estimable avec précision en début de sprint P1
Points de fonction (Function Points)	Comptabilise les fonctionnalités du système (entrées, sorties, requêtes, fichiers logiques) et les pond selon leur complexité pour estimer l'effort indépendamment de la technologie	Indépendante du langage de programmation ; reconnue par l'ISO/IEC 20926	Complexe à appliquer rigoureusement ; nécessite une expertise spécialisée en comptage de points de fonction	Non retenue : la décomposition fonctionnelle complète de Formuloo OS n'est pas disponible dès le sprint P1

Méthode	Principe	Avantages	Inconvénients	Applicabilité à Formuloo OS
Estimation experte par capacité d'équipe (retenue)	L'effort total est calculé directement à partir de la capacité réelle de l'équipe : $\text{Effort} = \text{Nombre de développeurs} \times \text{Durée en mois}$. Recommandée par Schwaber & Sutherland pour les projets Scrum	Simple, directe, ancrée dans la réalité de l'équipe ; parfaitement compatible avec la planification agile en sprints ; immédiatement applicable dès le sprint P1	Moins précise que COCOMO II pour les projets de grande envergure ; dépend de la stabilité de l'équipe	Retenue : parfaitement adaptée à notre équipe de 4 développeurs travaillant en sprints de 2 semaines sur 4 mois

Source : élaboré par l'auteur à partir de Schwaber & Sutherland.

La méthode d'estimation experte par capacité d'équipe a été retenue pour Formuloo OS pour trois raisons. Premièrement, sa simplicité est parfaitement adaptée au contexte agile : dans le framework Scrum, la capacité de l'équipe est connue dès le sprint planning et constitue le principal levier de planification. Deuxièmement, elle est immédiatement applicable sans prérequis de données historiques ou d'expertise spécialisée contrairement à COCOMO II qui nécessite une base de données de projets comparables pour calibrer ses équations. Troisièmement, elle est explicitement recommandée par Schwaber et Sutherland [7] pour les projets Scrum où l'effort se définit sprint par sprint, par ajustement continu, plutôt que par une planification exhaustive initiale. L'effort total calculé pour Formuloo OS est : $\text{Effort} = 4 \text{ développeurs} \times 4 \text{ mois} = 16 \text{ personne-mois}$.

I.5.1. Tableaux des coûts estimés

Tableau 7: Estimations des cout des ressources matériels

Désignation	Montant unitaire	Quantité / Durée	Total (FCFA)
Connexion Internet haut débit	15 000 FCFA/mois	1 / 5 mois	75 000 FCFA
Total ressources matérielles			75 000 FCFA

Source : par nos soins.

Tableau 8: Estimation des cout des ressources logiciel

Désignation	Montant unitaire	Total (FCFA)
Python, Django, DRF, drf-spectacular, Celery (open-source)	0 FCFA	0 FCFA
PostgreSQL 15, Redis 7, Nginx (open-source)	0 FCFA	0 FCFA
Docker + Docker Compose (licence développement gratuite)	0 FCFA	0 FCFA
GitLab CI Runner (auto-hébergé, open-source)	0 FCFA	0 FCFA
Prometheus, Grafana, Alertmanager, OpenTelemetry (open-source)	0 FCFA	0 FCFA
Visual Studio Code (licence gratuite)	0 FCFA	0 FCFA
Draw.io, Figma (plans gratuits)	0 FCFA	0 FCFA
Total ressources logicielles		0 FCFA

Source : par nos soins. La totalité de la stack logicielle est open-source, conformément à l'ambition de Formuloo OS d'être accessible aux PME africaines sans frais de licence.

Tableau 9: Estimation des cout des ressources humaines

Désignation	Taux	Durée	Total (FCFA)
4 développeurs stagiaires (Master 2)	Stage non rémunéré	4 mois	0 FCFA
Superviseur professionnel (encadrement)		4 mois	Inclus charges entreprise
Total ressources humaines (charges directes)			0 FCFA

Source : par nos soins.

Tableau 10: cout total du projet formuloo OS

Catégorie	Total (FCFA)
Ressources matérielles	75 000 FCFA
Ressources logicielles (stack open-source)	0 FCFA
Ressources humaines (stagiaires)	0 FCFA
Marge pour imprévus (10 %)	7 500 FCFA
Total général	82 500 FCFA

Source : par nos soins. Ce coût particulièrement bas illustre l'avantage stratégique de la stack open-source de Formuloo OS : il est possible de développer un ERP de qualité professionnelle avec un investissement minimal, conforme aux contraintes économiques des PME africaines.

II. Méthodes

Cette section expose les méthodes et techniques adoptées pour mener à bien le projet Formuloo OS, en justifiant chaque choix par rapport aux alternatives étudiées. Quatre dimensions méthodologiques sont abordées : la méthode de gestion de projet Agile Scrum, l'approche de gouvernance contractuelle Contract-First, la méthode de conception logicielle UML 2.5, et l'architecture du système.

II.1. Méthode de gestion de projet : Agile Scrum

II.1.1. Étude comparative des approches de développement

Avant de retenir Scrum, nous avons comparé les principales approches de gestion de projet disponibles au regard des contraintes spécifiques de Formuloo OS : équipe de quatre développeurs, livraisons toutes les deux semaines, quatre équipes en parallèle sur des modules interdépendants.

Tableau 11: Comparaison des approches de développement

Approche	Description	Évaluation pour Formuloo OS
Cycle en V	Phases séquentielles rigides : analyse complète, conception, développement, tests, déploiement	Inadapté : les exigences des PME africaines évoluent en cours de projet ; une approche séquentielle ne permet pas de détecter les incompatibilités d'interface avant la phase d'intégration finale
Agile Scrum (retenu)	Sprints de 2 semaines avec rôles Product Owner, Scrum Master et équipe ; backlog priorisé en continu ; Definition of Done incluant la gouvernance contractuelle	Retenu : permet des livraisons fréquentes, la synchronisation continue des contrats d'interface entre équipes, et l'intégration de la validation contractuelle dans la Definition of Done [7]
Kanban	Flux continu, visualisation du travail en cours, limitation du WIP	Partiellement adapté pour les tâches opérationnelles mais insuffisant pour coordonner 4 équipes en parallèle sur des modules interdépendants avec des contrats d'interface partagés
Extreme Programming (XP)	Pratiques techniques strictes : TDD, pair programming, intégration continue intensive	Trop exigeant pour une équipe de stagiaires débutant sur la stack ; les pratiques XP (TDD, Schemathesis) sont visées en phase 2 du projet

Source : élaboré par l'auteur à partir de Schwaber & Sutherland.

II.1.2. Instanciation de Scrum dans Formuloo OS

Dans le cadre du projet Formuloo OS, Scrum a été instancié selon l'organisation suivante. L'encadreur professionnel assume le rôle de Product Owner : il définit et priorise le backlog produit à partir du cahier des charges [5], valide les incréments en sprint review et arbitre les conflits de priorisation. KENGNE Cabrel et Leticia exercent conjointement le rôle de Scrum Master : ils facilitent les cérémonies, lèvent les obstacles techniques et assurent la synchronisation des contrats d'interface. La Definition of Done adoptée intègre explicitement la conformité du pipeline

validate_docs : aucune merge request ne peut rejoindre la branche principale sans que les six artefacts de gouvernance soient présents et validés.

II.2. Approche de gouvernance contractuelle : Contract-First

II.2.1. Contract-First vs Code-First

Tableau 12: comparaison du Contract-first et du Code First

Critère	Code-First	Contract-First (retenu)
Processus	Code écrit en premier ; documentation générée après	Contrat OpenAPI défini et validé AVANT le code d'implémentation
Détection régressions	Tardive : découverte à l'intégration (plusieurs sprints après)	Précoce : détection immédiate avant le premier commit
Développement parallèle	Impossible : le frontend attend le backend	Possible : le frontend utilise Prism (serveur de mock) dès le sprint P1
Cohérence doc-code	Dérive inévitable au fil des sprints	Cohérence permanente : spec générée depuis le code par introspection (drf-spectacular)
Validation automatique	Aucune	Pipeline validate_docs : 6 artefacts vérifiés à chaque merge request en ~42 secondes
Coût initial	Nul	~20 % du sprint P1 pour CONVENTIONS.md et les 4 ADR récupéré dès le premier sprint d'intégration

Source : élaboré par l'auteur à partir de Lauret et Stoplight.

II.2.2. Mécanisme de gouvernance Contract-First dans Formuloo OS

Le mécanisme de gouvernance Contract-First de Formuloo OS s'organise en quatre étapes opérationnelles. La première étape est l'annotation des ViewSets avec `@extend_schema` pour permettre à drf-spectacular de générer la spécification OpenAPI 3.1 exposée sur `/api/schema/` et `/api/docs/`. La deuxième étape est la formalisation des règles transverses dans CONVENTIONS.md et les quatre ADR (ADR-001 : choix du monolithe modulaire ; ADR-002 : multi-tenant shared schema ; ADR-003 : conventions de nommage endpoints ; ADR-004 : communication inter-

modules via Celery uniquement). La troisième étape est la validation automatique par le pipeline `validate_docs` à chaque merge request. La quatrième étape est l'isolation multi-tenant par `TenantJWTAuthentication` (extraction du `tenant_id` du JWT) et `TenantFilterMixin` (ajout automatique `WHERE tenant_id` sur chaque requête SQL).

II.3. Méthode de conception logicielle : UML 2.5

II.3.1. Comparaison MERISE vs UML 2.5

Tableau 13: Comparaison de Merise et UML 2.5

Critère	MERISE	UML 2.5
Nature	Méthodologie complète de développement SI	Langage de modélisation graphique standardisé (norme ISO/IEC 19501)
Approche	Orientée données et traitements ; structurée et séquentielle	Orientée objet, flexible, réutilisable et multi-vues
Périmètre	Guide l'ensemble du cycle de vie d'un projet SI	Permet de modéliser tout type de système logiciel
Diagrammes disponibles	Ensemble fixe : MCD, MCT, MOT, MDT	14 types de diagrammes (cas d'utilisation, classes, séquence, activité, état, déploiement, etc.)
Adaptabilité	Peu flexible ; lié aux méthodes en cascade	Très flexible ; compatible avec toutes les méthodes, y compris Agile Scrum
Adéquation au projet	Insuffisante : ne couvre pas les diagrammes de séquence JWT ni les états-transitions Lead CRM nécessaires	Excellente : couvre tous les besoins de modélisation du projet architecture, comportement, données, interactions inter-modules

Source : adapté par nos soins.

UML 2.5 a été retenu car sa nature orientée objet est cohérente avec l'architecture Django REST Framework (classes `ViewSet`, `Serializer`, `Model`), et sa palette de 14 types de diagrammes couvre intégralement les besoins de modélisation de Formuloo OS, contrairement à MERISE dont le jeu fixe de diagrammes ne permet pas de modéliser les flux d'authentification JWT ni le cycle de vie d'un Lead CRM.

II.4. Architecture de Formuloo OS

Cette section présente l'architecture logicielle de Formuloo OS, résultat de la conception conduite lors du sprint P1 et documentée dans les quatre Architecture Decision Records du projet. L'architecture est organisée en six couches superposées, chacune ayant une responsabilité clairement délimitée.

La première couche, celle des clients, regroupe les trois types de consommateurs de l'API : le navigateur web exécutant l'application Angular SPA développée par Loïc, l'application mobile native développée par Samy, et les intégrateurs tiers consommant l'API publique via des clés API soumises au rate limiting. La deuxième couche est la passerelle Nginx, point d'entrée unique de l'architecture : elle assure le routing des requêtes, la terminaison SSL, la gestion des en-têtes CORS, l'application du rate limiting, et la propagation du `tenant_id` extrait du JWT vers les services backend. La troisième couche est celle des six services métier, chacun conteneurisé dans son propre conteneur Docker : Auth (:8000), CRM (:8001), Ressources Humaines (:8002), Comptabilité (:8003), Analytics (:8004) et Stock (phase 2). La quatrième couche est le bus de messages asynchrone Redis/Celery : il assure les communications inter-modules conformément à l'ADR-004, qui interdit tout appel HTTP direct de service à service. La cinquième couche est celle des données : PostgreSQL 15 pour les données applicatives multi-tenant (shared schema), Redis Cache pour les sessions et le cache API, et S3 Storage pour les fichiers et documents. La sixième couche, encadrée en pointillé pour signifier qu'il s'agit de notre contribution principale, est la couche de gouvernance : `drf-spectacular` pour la génération OpenAPI, `TenantJWTAuthentication` et `TenantFilterMixin` pour la sécurité multi-tenant, `CONVENTIONS.md` et les quatre ADR pour les règles transverses, et le pipeline `validate_docs` pour la validation automatique. La couche d'observabilité Prometheus, Grafana, Alertmanager et OpenTelemetry surveille l'ensemble des services en temps réel, conformément au ticket FOR16A26-1032.

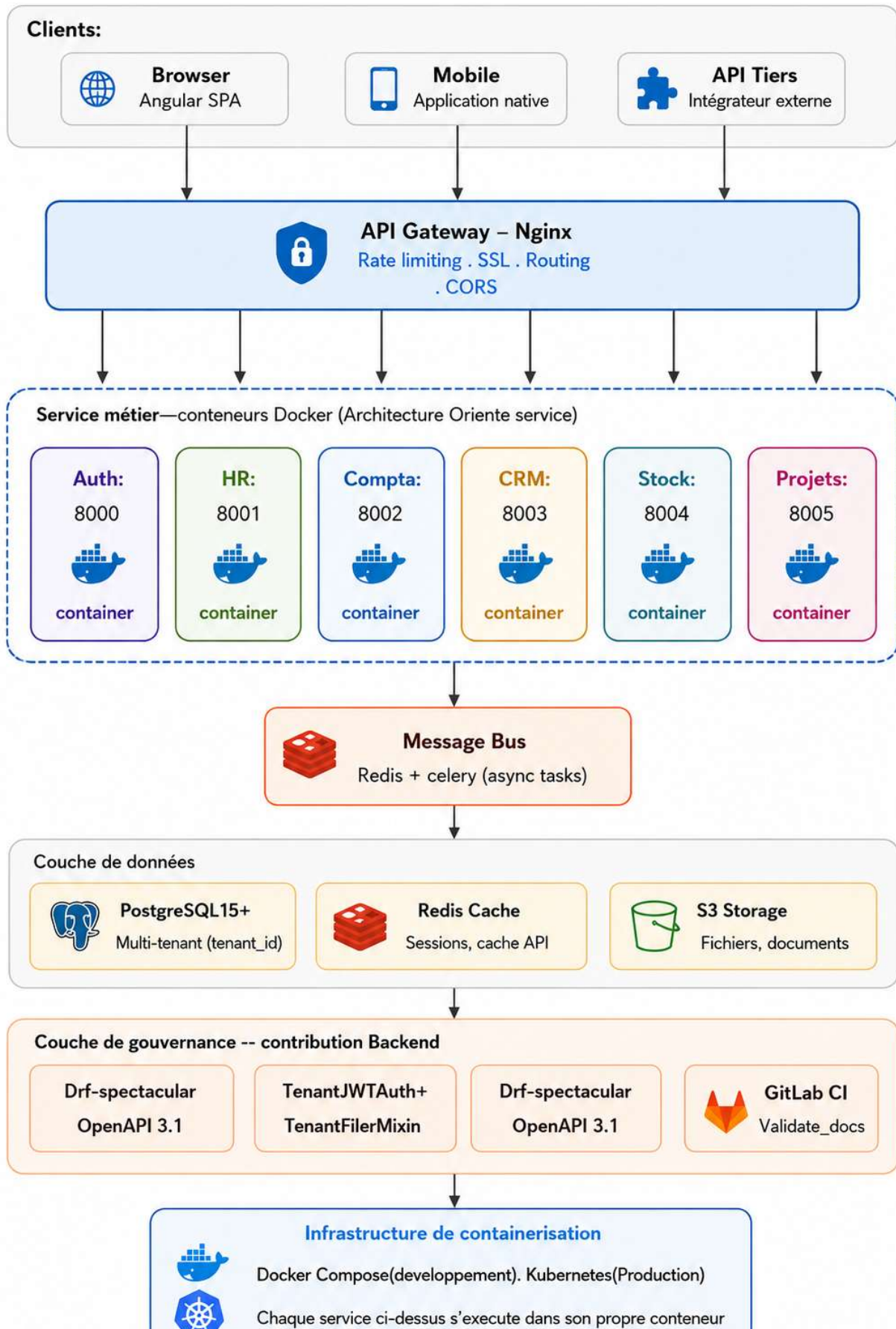


Figure 2: Architecture Global de Formuloo OS

Figure II.2 : Architecture globale de Formuloo OS organisée en six couches de déploiement avec la couche d'observabilité Prometheus/Grafana. Flèches oranges = flux asynchrones inter-modules (Signal Django + Celery) ; flèches grises = flux synchrones via passerelle JWT.

III. Modélisation UML du système

La modélisation UML du système Formuloo OS couvre cinq types de diagrammes complémentaires. Conformément au style de présentation adopté dans les travaux de référence en génie logiciel, chaque diagramme de cas d'utilisation est accompagné d'une description textuelle structurée présentant pour chaque cas d'utilisation son titre, ses objectifs, sa description, les acteurs concernés, les préconditions, le scénario nominal, la postcondition et les exceptions.

III.1. Acteurs du système

Le système Formuloo OS interagit avec deux acteurs principaux et un acteur secondaire. L'Utilisateur authentifié est le premier acteur principal : il représente tout employé d'une organisation cliente dont les droits sont déterminés par son rôle RBAC. L'Intégrateur tiers est le deuxième acteur principal : développeur externe consommant l'API publique via une clé API. Keycloak est l'acteur secondaire : fournisseur d'identité OIDC qui valide les identités lors du flux d'authentification JWT il apparaît à droite des diagrammes en trait pointillé, conformément à la convention UML pour les acteurs secondaires.

III.2. Diagrammes de cas d'utilisation

III.2.1. Diagramme du module Auth/IAM

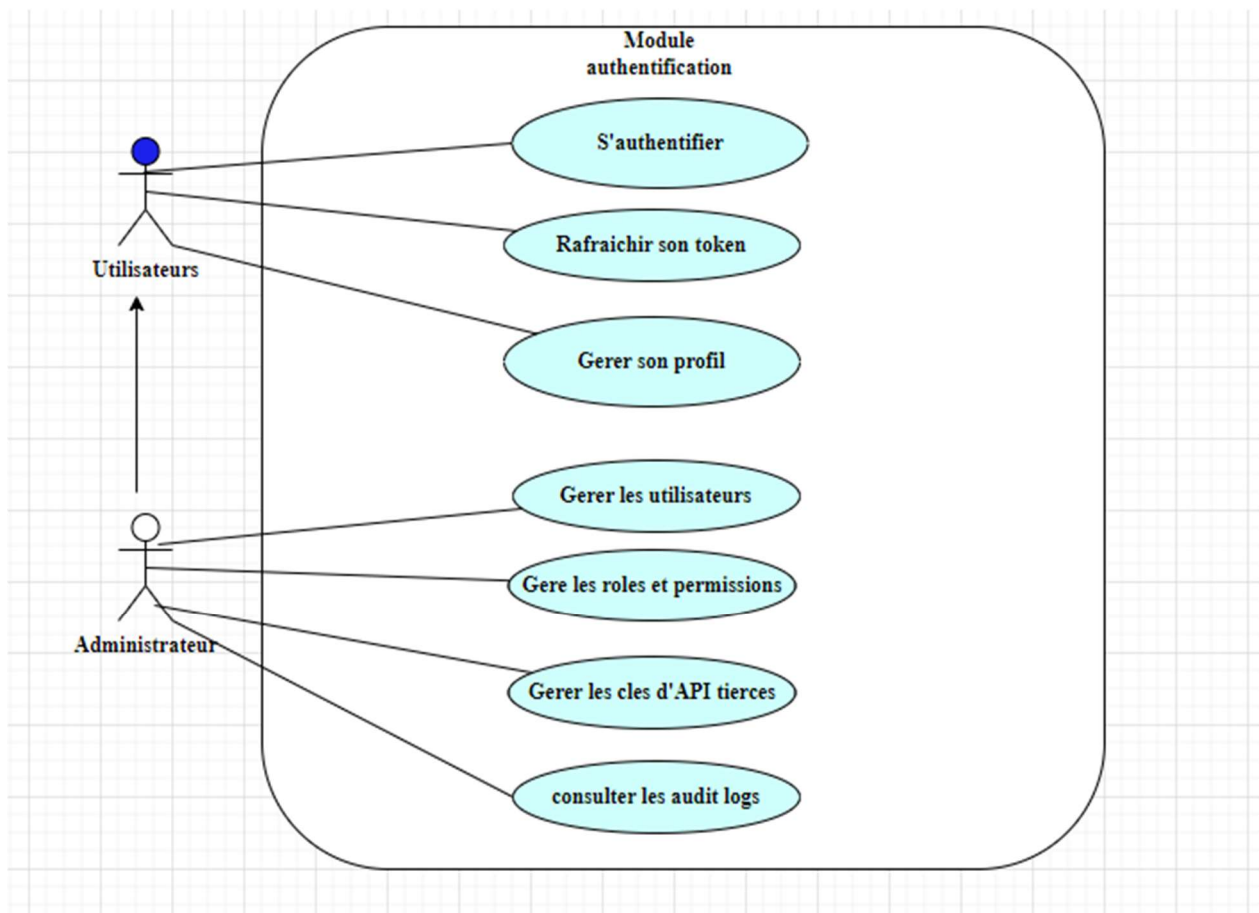


Figure 3: Diagramme de cas d'utilisation Auth/iam

III.2.2. Descriptions textuelles des cas d'utilisation — Module Auth/IAM

Tableau 14: Description textuelle du cas d'utilisation s'authentifier

Cas d'utilisation	
Titre	S'authentifier
Objectifs	Permettre à un utilisateur de vérifier son identité et d'obtenir les tokens JWT (access + refresh) nécessaires à l'accès aux ressources protégées.
Description	Ce cas d'utilisation décrit le processus d'authentification d'un utilisateur via son email et son mot de passe. Il couvre la vérification de l'identité par Keycloak et la génération des tokens JWT par le service Auth.
Acteur(s)	Utilisateur authentifié, Keycloak (acteur secondaire)
Précondition	L'utilisateur dispose d'un compte actif dans le système. Keycloak est opérationnel.
Scénario nominal	1. L'utilisateur envoie POST /api/v1/auth/login/ avec son email et son mot de passe. 2. Le Gateway Nginx route la requête vers le service Auth. 3. Le service Auth délègue la vérification à Keycloak. 4. Keycloak confirme l'identité et retourne un jeton OIDC. 5. Le service Auth génère un access token (15 min) et un refresh token (7 jours). 6. Le service Auth retourne les deux tokens au Client.
Post condition	L'utilisateur reçoit ses tokens JWT. Il peut maintenant accéder aux ressources protégées.
Exception(s)	Email ou mot de passe incorrect → 401 Unauthorized. Compte désactivé → 401 Unauthorized avec message spécifique.

Tableau 15: Description du cas d'utilisation gérer les Permission RBAC

Cas d'utilisation	
Titre	Gérer les droits d'accès (RBAC)
Objectifs	Permettre à l'Administrateur de définir les rôles et permissions des utilisateurs pour contrôler l'accès aux ressources de chaque module.
Description	Ce cas d'utilisation permet la création, modification et suppression de rôles (Rôle) et de permissions (Permission), et leur attribution aux utilisateurs. Il implémente le modèle RBAC (Role-Based Access Control) documenté dans l'ADR-002.
Acteur(s)	Administrateur (Utilisateur authentifié avec rôle admin)
Précondition	L'administrateur est authentifié avec un token JWT valide contenant le rôle 'admin'. Les modules cibles sont opérationnels.
Scénario nominal	1. L'administrateur envoie une requête POST /api/v1/auth/roles/ avec le nom et les permissions du rôle. 2. Le Gateway vérifie le JWT et les permissions RBAC de l'administrateur. 3. Le service Auth crée le rôle et ses permissions associées en base PostgreSQL (filtre tenant_id). 4. L'administrateur attribue le rôle à un utilisateur via POST /api/v1/auth/users/{id}/roles/. 5. Le système confirme la mise à jour des droits.
Post condition	Le rôle est créé et attribué. L'utilisateur disposera des permissions du rôle lors de sa prochaine requête.
Exception(s)	Tentative de créer un rôle avec un nom déjà existant → 400 Bad Request. Tentative d'attribuer un rôle d'un autre tenant → 403 Forbidden (TenantFilterMixin).

III.2.3. Diagramme du module Ressources Humaines

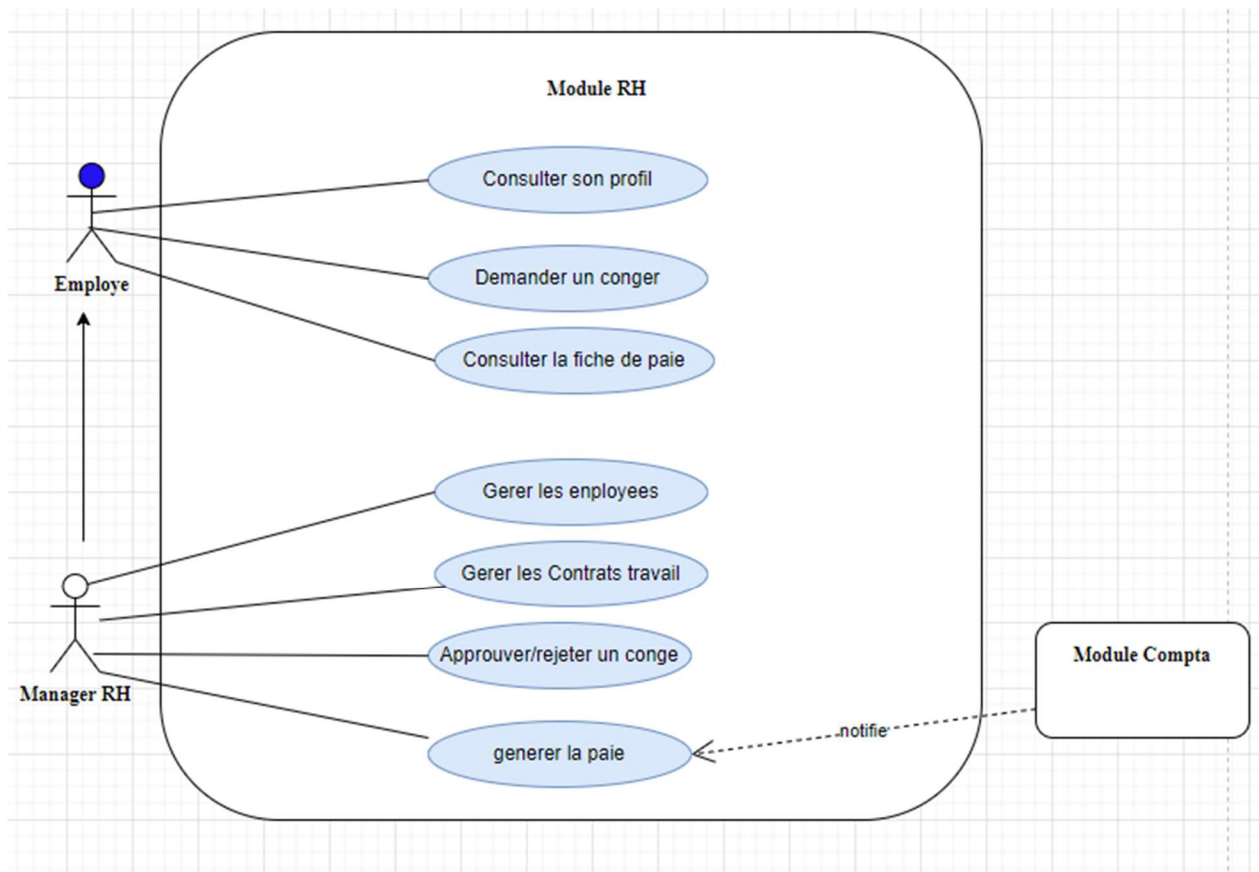


Figure 4: Diagramme de cas d'utilisation module RH

Tableau 16: Description du cas d'utilisation Générer la paie

Cas d'utilisation	
Titre	Générer la paie
Objectifs	Permettre au Manager RH de générer les fiches de paie mensuelles des employés et de notifier automatiquement le module Comptabilité pour la création de l'écriture comptable correspondante.
Description	Ce cas d'utilisation décrit la génération d'une fiche de paie (Payslip) pour un employé. Une fois la paie générée, un signal Django déclenche automatiquement une tâche Celery qui crée l'écriture comptable (JournalEntry) dans le module Comptabilité, sans action manuelle supplémentaire.
Acteur(s)	Manager RH, Comptabilité (sous-système secondaire)
Précondition	L'employé concerné dispose d'un contrat actif (Contract.is_active = True). La période de paie n'a pas encore été traitée pour cet employé. Le Manager RH est authentifié avec les permissions requises.
Scénario nominal	1. Le Manager RH envoie POST /api/v1/hr/payslips/ avec l'identifiant de l'employé et la période (YYYY-MM). 2. Le service RH calcule le salaire brut, les déductions et le salaire net à partir du contrat actif. 3. Le service RH crée l'objet Payslip en base PostgreSQL (filtre tenant_id). 4. Un signal Django post_save déclenche automatiquement une tâche Celery. 5. La tâche Celery crée l'écriture comptable (JournalEntry) dans le module Comptabilité avec les montants correspondants. 6. Le service RH retourne la fiche de paie générée.
Post condition	La fiche de paie est créée. L'écriture comptable est créée dans le module Comptabilité. Le Manager RH n'a effectué aucune action supplémentaire pour la comptabilisation.
Exception(s)	Contrat inactif ou inexistant → 400 Bad Request. Paie déjà générée pour cette période → 409 Conflict.

III.2.4. Diagrammes des modules Comptabilité et CRM

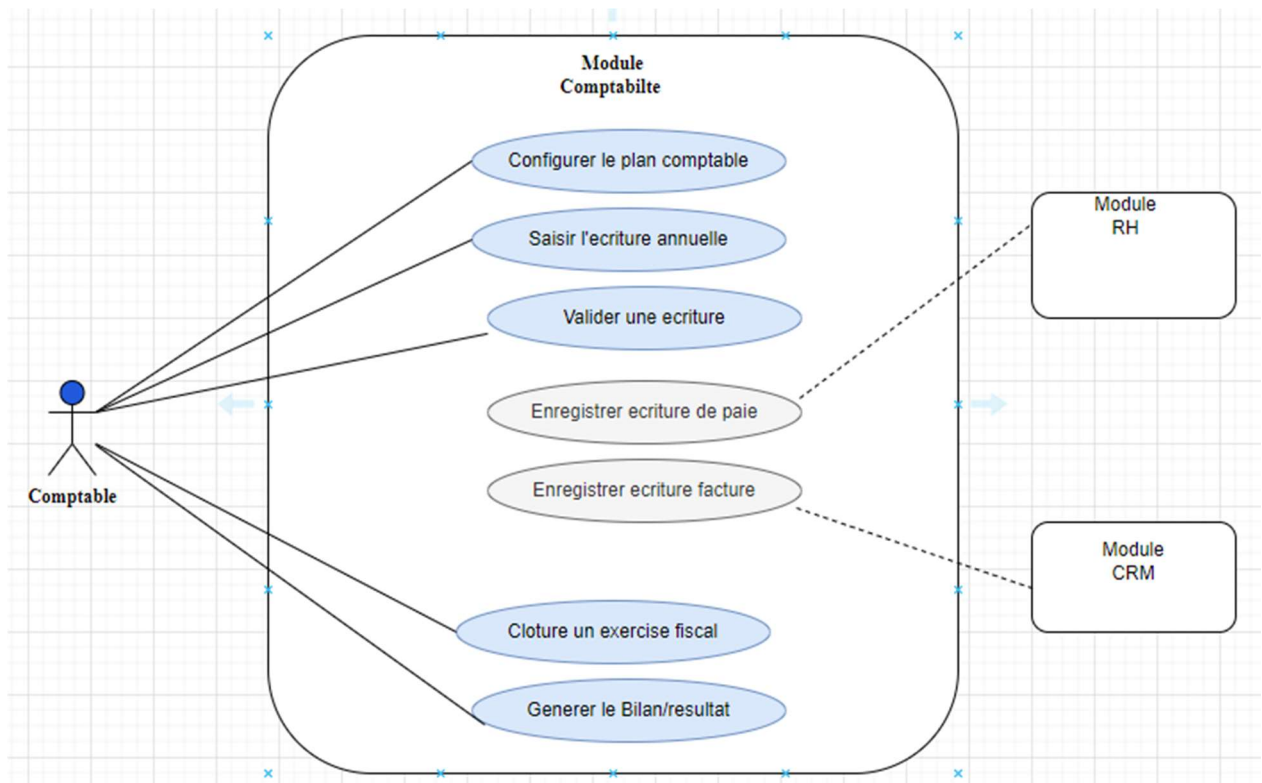


Figure 5: Diagramme de cas d'utilisation du module Compta

Tableau 17: Description textuelle du cas d'utilisation saisie d'une écriture comptable

Cas d'utilisation	
Titre	Saisir une écriture comptable
Objectifs	Permettre au Comptable de saisir manuellement une écriture comptable conforme aux règles du SYSCOHADA, avec vérification automatique de l'équilibre débit/crédit.
Description	Ce cas d'utilisation permet la création d'une écriture comptable (JournalEntry) avec ses lignes (JournalLine). La contrainte fondamentale du SYSCOHADA — principe de la partie double : $\Sigma \text{debit} = \Sigma \text{credit}$ — est vérifiée automatiquement par le Serializer DRF avant tout enregistrement.
Acteur(s)	Comptable (Utilisateur authentifié avec rôle 'comptable')
Précondition	Le journal cible existe. L'exercice fiscal est ouvert (FiscalYear.status = 'open'). Le Comptable est authentifié.
Scénario nominal	1. Le Comptable envoie POST /api/v1/compta/journal-entries/ avec le journal, la date, le libellé et les lignes (account, debit/credit). 2. Le Serializer DRF vérifie que $\Sigma \text{debit} = \Sigma \text{credit}$. Si la condition n'est pas remplie → 400 Bad Request. 3. Le service Comptabilité crée l'écriture en base PostgreSQL avec le statut 'draft'. 4. Le Comptable valide l'écriture via PUT /api/v1/compta/journal-entries/{id}/validate/. 5. Le système change le statut à 'validated' et rend l'écriture immuable.
Post condition	L'écriture comptable est enregistrée avec le statut 'validated'. Elle est incluse dans la balance des comptes du journal.
Exception(s)	$\Sigma \text{debit} \neq \Sigma \text{credit}$ → 400 Bad Request avec message 'Les débits et crédits doivent être équilibrés'. Exercice clos → 403 Forbidden.

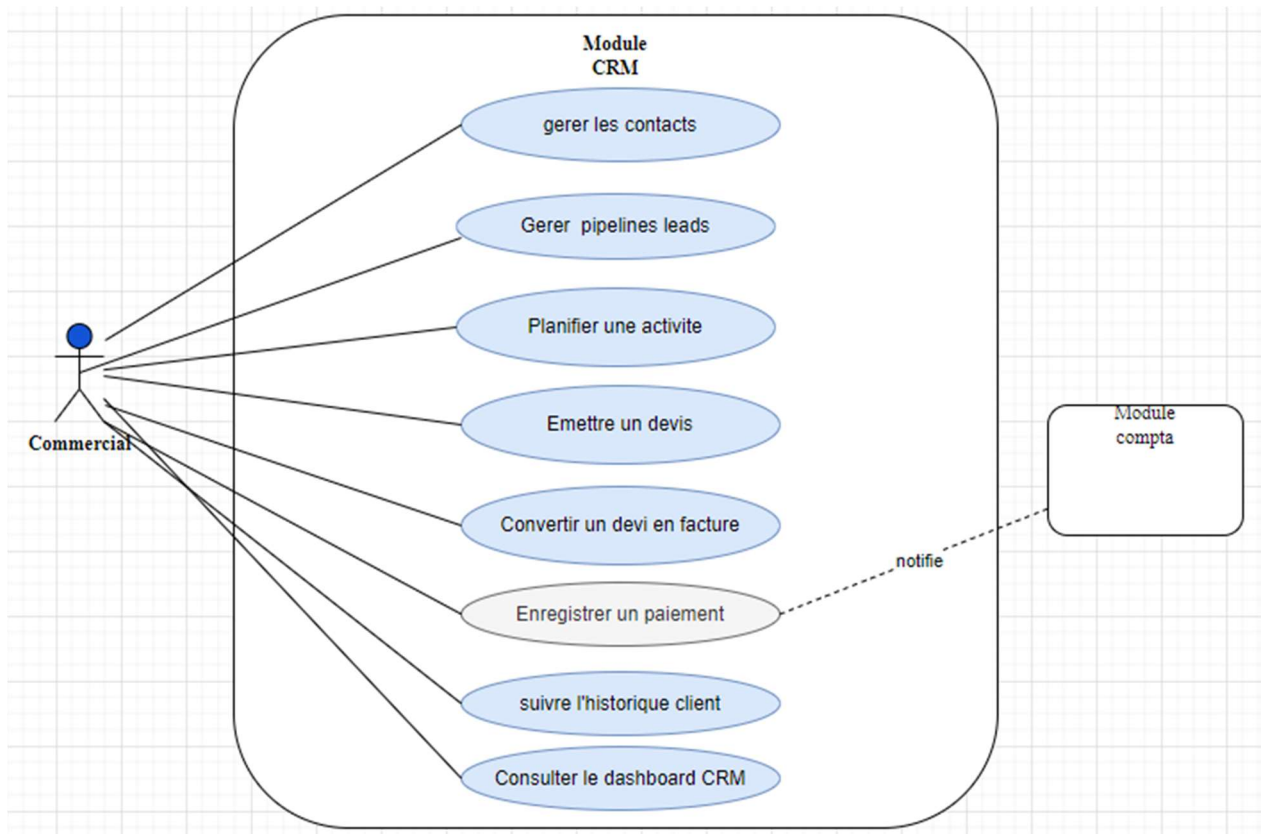


Figure 6: Diagramme de cas d'utilisation du module CRM

Tableau 18: Description textuelle pour gérer le pipeline lead

Cas d'utilisation	
Titre	Gérer le pipeline de leads
Objectifs	Permettre au Commercial de suivre l'avancement des opportunités commerciales (Leads) depuis leur création jusqu'à leur conversion en client ou leur perte, en passant par les étapes de qualification, de devis et de facturation.
Description	Ce cas d'utilisation couvre le cycle de vie complet d'un Lead dans le pipeline commercial de Formuloo OS : création, qualification, envoi de devis, conversion (facturation) ou perte. La transition Converti → Facturé déclenche automatiquement la notification du sous-système Comptabilité.
Acteur(s)	Commercial (Utilisateur authentifié avec rôle 'commercial'), Comptabilité (sous-système secondaire)
Précondition	Un Contact existe dans le système pour ce prospect. Le Commercial est authentifié avec les permissions CRM.
Scénario nominal	1. Le Commercial crée le lead via POST /api/v1/crm/leads/ avec le contact, la valeur estimée et la date de clôture attendue. 2. Après entretien de qualification, le Commercial met à jour le statut via PATCH /api/v1/crm/leads/{id}/ avec stage = 'qualifié'. 3. Le Commercial envoie un devis via POST /api/v1/crm/quotes/ lié au lead → statut lead = 'devis envoyé'. 4a. Si le client accepte → POST /api/v1/crm/invoices/ depuis le devis → statut = 'converti' ; Signal Django → Celery → JournalEntry créée dans Comptabilité. 4b. Si le client refuse → PATCH lead avec statut = 'perdu'. 5. En cas de paiement → POST /api/v1/crm/payments/ → Signal Django → Celery → écriture de règlement dans Comptabilité.
Post condition	Le lead est converti en client facturable, ou archivé comme perdu. L'écriture comptable est créée automatiquement sans action du Comptable.
Exception(s)	Tentative de créer une facture depuis un devis non accepté → 400 Bad Request. Tentative d'accéder à un lead d'un autre tenant → 0 résultat (TenantFilterMixin).

III.3. Diagrammes de classes

Les diagrammes de classes des quatre modules opérationnels sont présentés ci-dessous. Chaque entité est rattachée à l'entité Organisation par des relations de composition (losange plein ♦, cycle de vie lié) ou d'agrégation (losange vide ◇, cycle de vie indépendant). La règle est simple : si la suppression de l'entité parente entraîne la suppression de l'entité enfant, la relation est une composition ; si l'entité enfant peut exister indépendamment, c'est une agrégation.

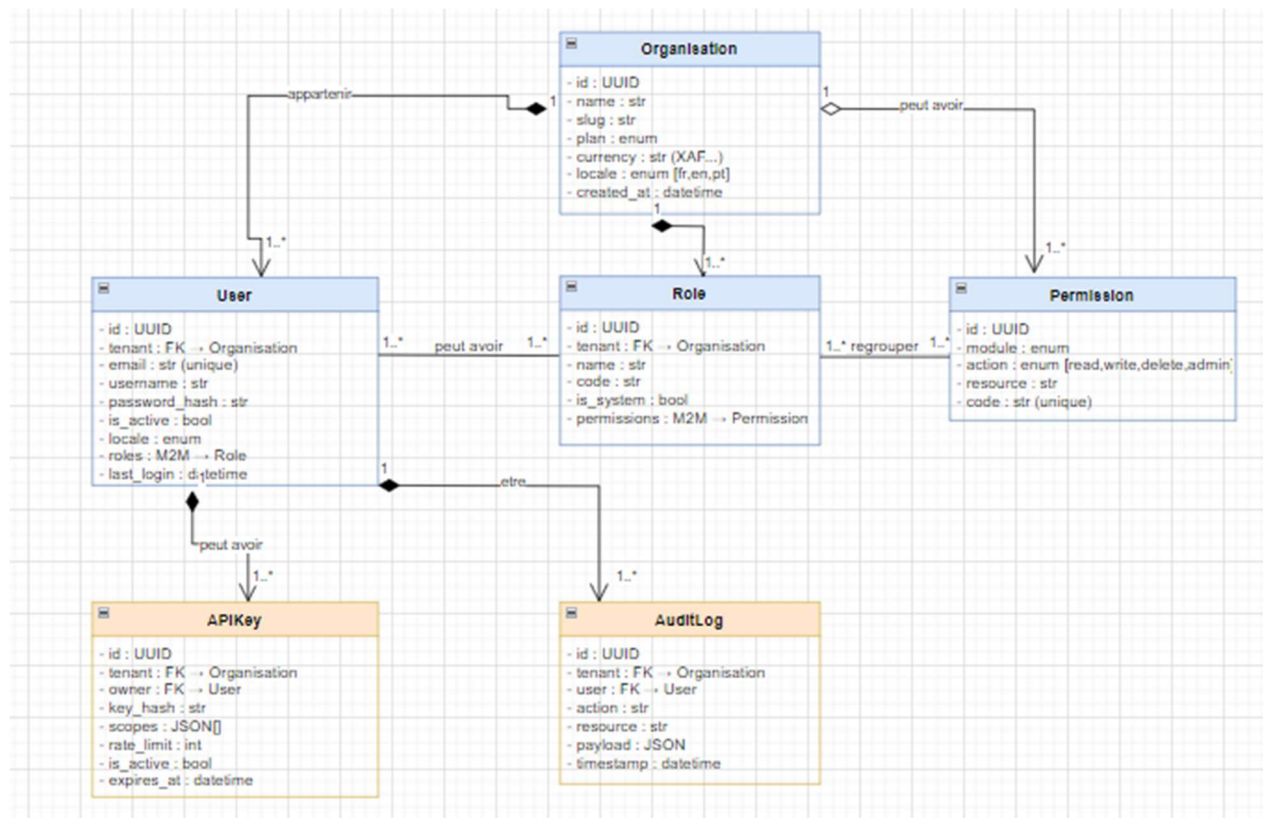


Figure 7: Diagramme de classe du module Auth

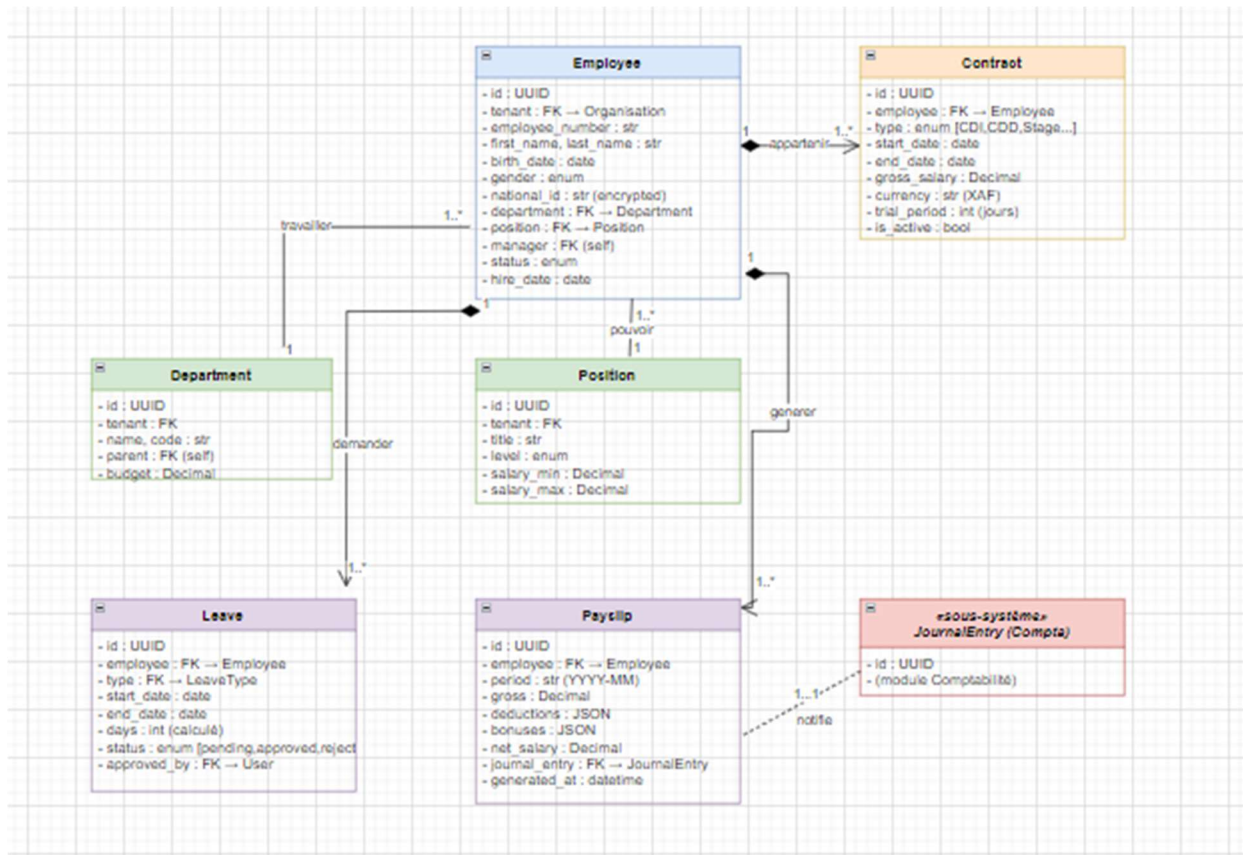


Figure 8: Diagramme de classes du module Compta

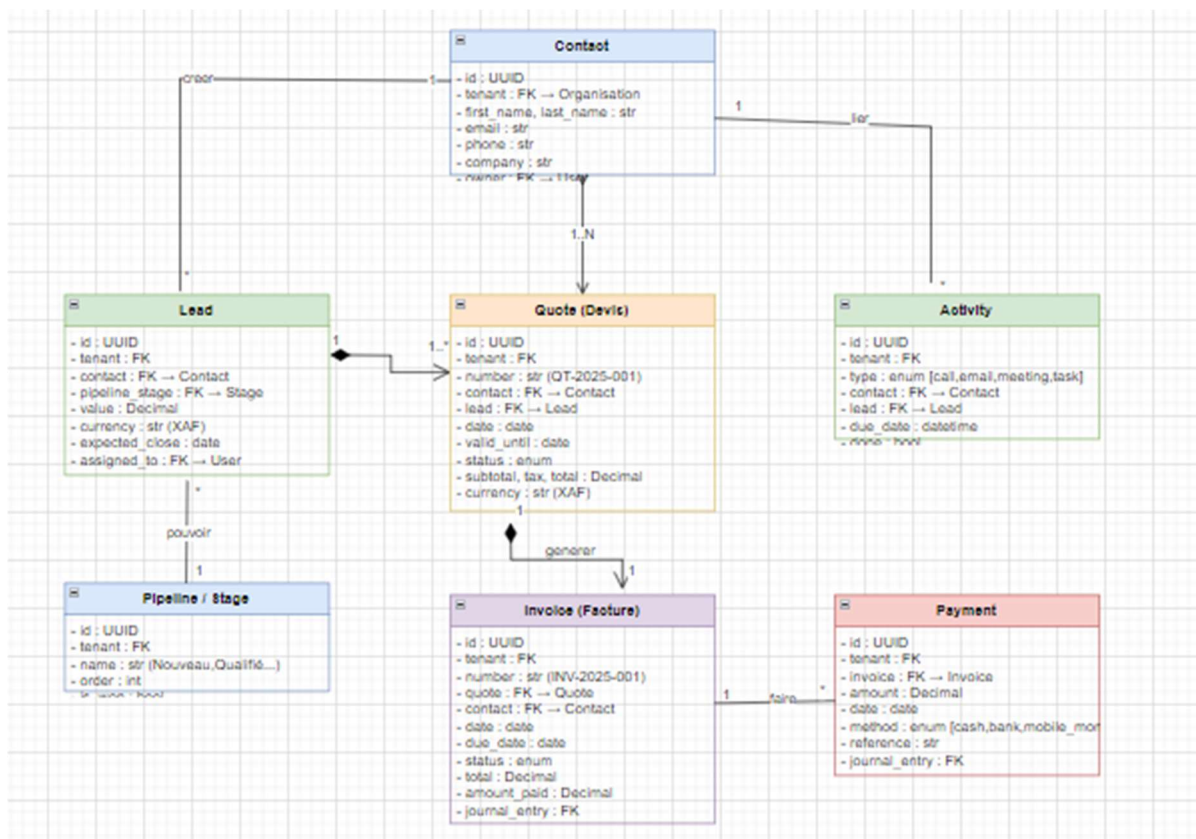


Figure 9: Diagramme de classes du module CRM

III.4. Diagrammes de séquence

III.4.1. Authentification JWT multi-tenant

Le diagramme de séquence de la Figure II.12 modélise le flux complet d'authentification JWT. Le Client envoie POST /api/v1/auth/login/ au Gateway Nginx qui route vers le service Auth. Le service Auth délègue la vérification à Keycloak (acteur secondaire), génère l'access token (15 min) et le refresh token (7 jours), et les retourne au Client. Lors des requêtes suivantes, le Gateway extrait le JWT, vérifie sa signature, injecte le `tenant_id` dans la requête routée vers le service cible, qui applique `TenantFilterMixin` avant de retourner les données filtrées.

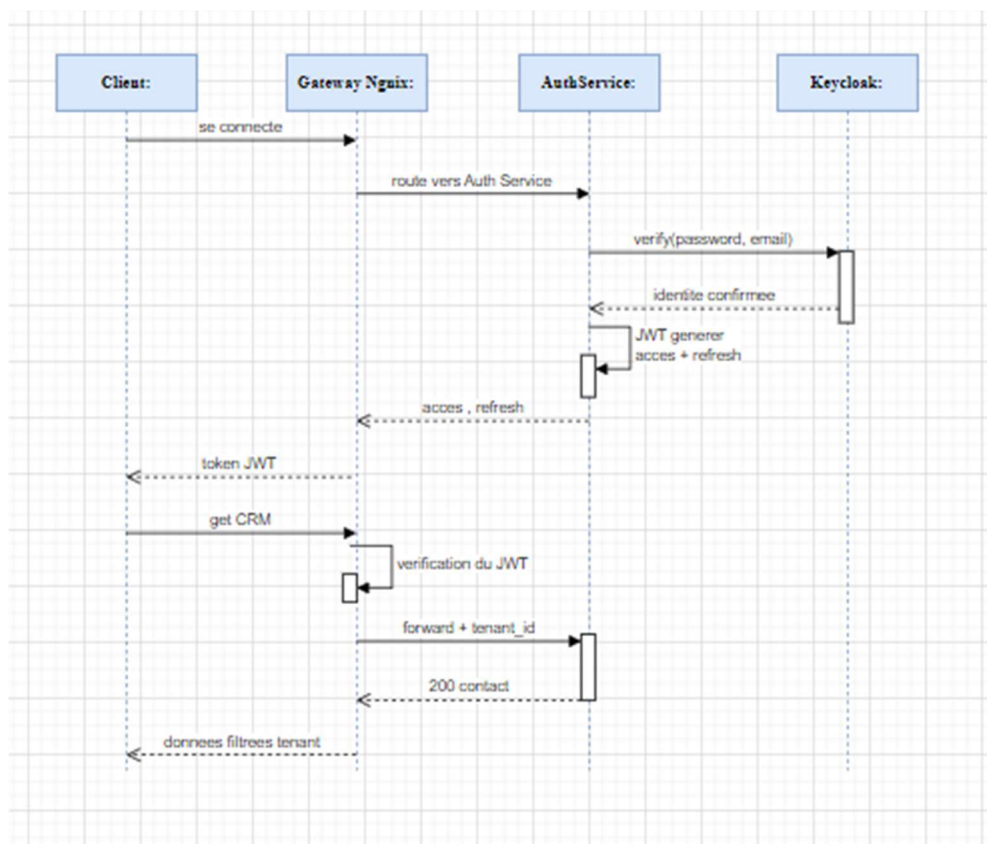


Figure 10: Diagramme de séquence de l'authentification

III.4.2. Génération et validation du contrat OpenAPI

Le diagramme de séquence de la Figure II.13 modélise le cycle de gouvernance Contract-First. Le Développeur annote son ViewSet avec `@extend_schema`, ce qui permet à `drf-spectacular` de générer la spécification OpenAPI 3.1 exposée sur /api/schema/ et /api/docs/. Le push en merge request déclenche le pipeline `validate_docs` en ~42 secondes (pipeline #230 PASSED). Si le pipeline passe, le Frontend régénère son client TypeScript via `ng-openapi-gen`. Toute modification incompatible du contrat casse la compilation TypeScript avant tout déploiement.

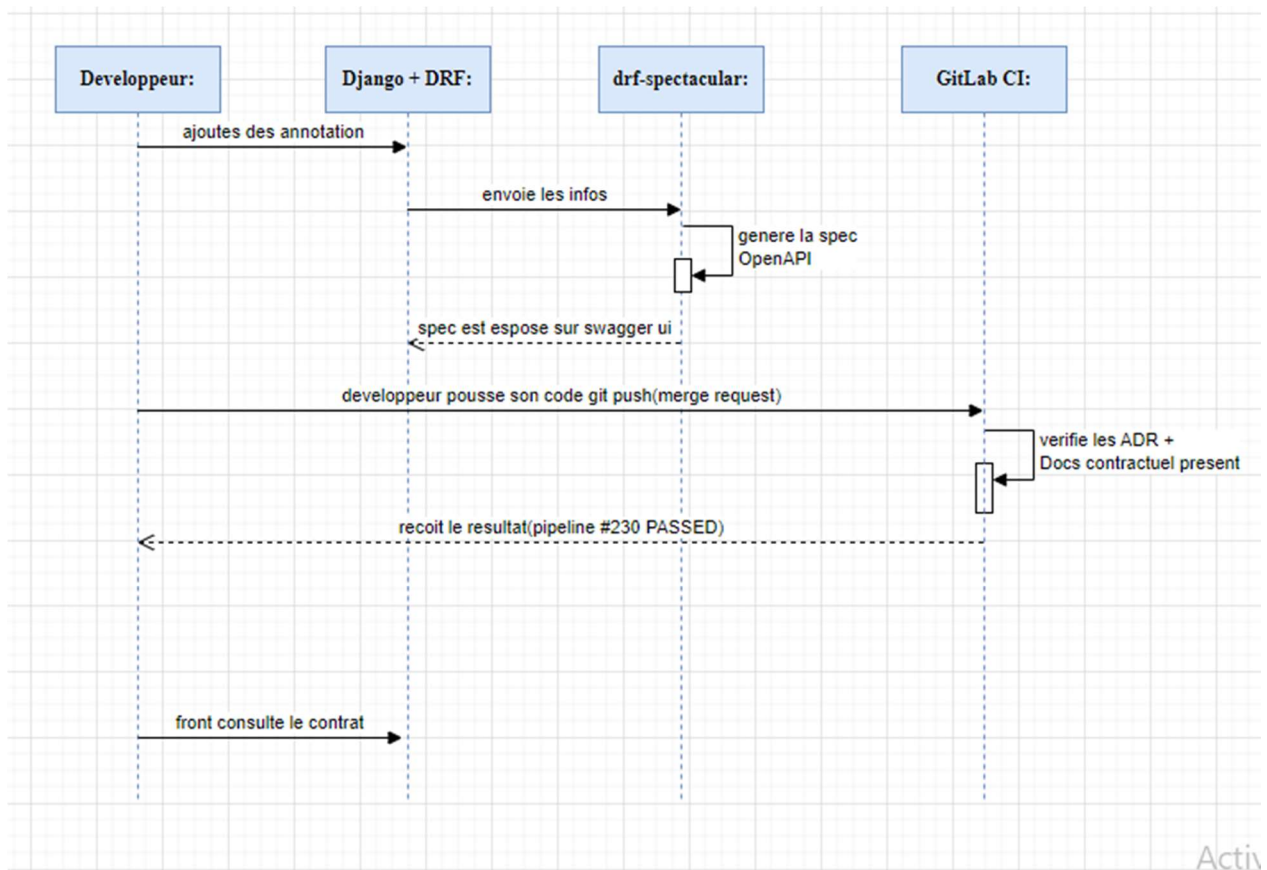


Figure 11: Diagramme de séquence pour la génération de contrat

III.5. Diagramme d'activité — Traitement d'une requête API

Le diagramme d'activité de la Figure II.14 modélise le traitement complet d'une requête HTTP entrante avec ses trois points de contrôle successifs : vérification du JWT par le Gateway Nginx (→ 401 si invalide), vérification des permissions RBAC par le ViewSet (→ 403 si insuffisantes), validation des données par le Serializer DRF (→ 400 si non conformes au contrat OpenAPI). Ce n'est qu'après ces trois vérifications que les données sont persistées en base PostgreSQL avec le filtre `tenant_id` automatique, et que la réponse est retournée au Client.

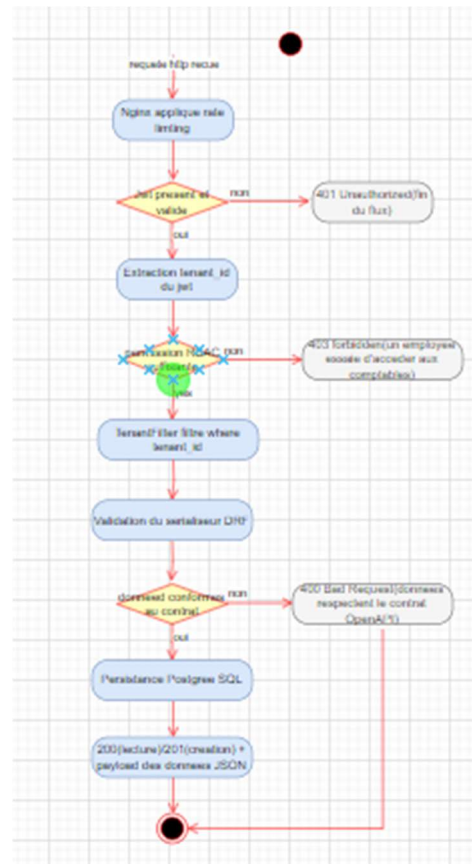


Figure 12: Diagramme d'activité pour le traitement d'une API

III.6. Diagramme d'état-transition Cycle de vie d'un Lead CRM

Le diagramme d'état-transition de la Figure II.15 modélise le cycle de vie d'un Lead dans le pipeline commercial, depuis l'état Nouveau jusqu'aux deux états finaux : Facturé (chemin de succès via Qualifié → Devis envoyé → Converti) et Perdu (chemin d'échec, accessible depuis tout état du pipeline). La transition Converti → Facturé déclenche automatiquement la notification asynchrone du module Comptabilité via Signal Django + Celery pour la création de l'écriture de vente. Un paiement ultérieur déclenche de même la création de l'écriture de règlement.

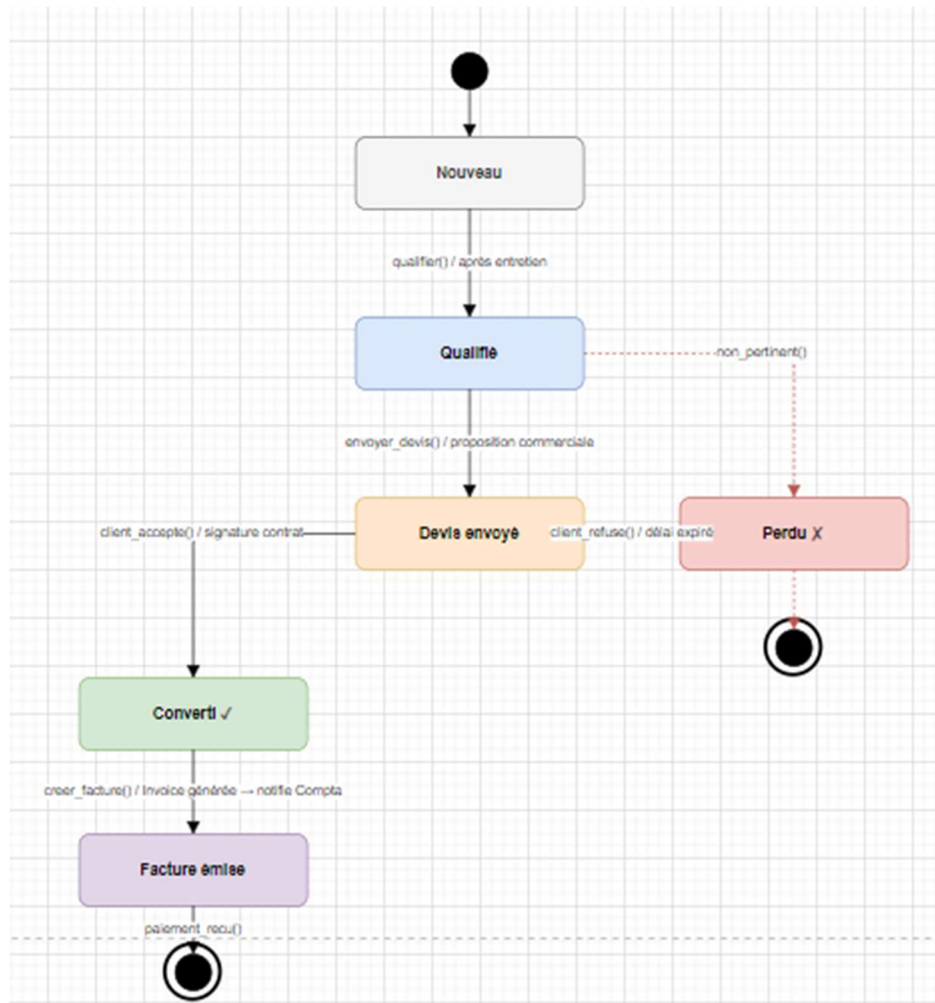


Figure 13: Diagramme d'état transition pour le cycle de vie d'un lead

Au terme de ce deuxième chapitre, nous avons présenté l'ensemble des matériaux et méthodes qui ont guidé la conception et la mise en œuvre de l'architecture backend de Formuloo OS. La stack technique entièrement open-source incluant l'observabilité Prometheus/Grafana, la méthode d'estimation par capacité d'équipe justifiée face à COCOMO II et aux points de fonction, la méthodologie Agile Scrum, l'approche Contract-First, la méthode UML 2.5, l'architecture en six couches et les descriptions textuelles complètes des cas d'utilisation constituent les fondements techniques et méthodologiques de notre contribution. Le chapitre suivant présente les résultats concrets obtenus et les discute de façon critique.

CHAPITRE III : RÉSULTATS ET DISCUSSIONS

Ce chapitre présente les résultats concrets obtenus à l'issue de la mise en œuvre de l'architecture backend de Formuloo OS, suivis d'une discussion les confrontant aux objectifs fixés, aux hypothèses de recherche formulées en introduction et aux travaux existants dans la littérature. Nous évaluerons la pertinence de ces résultats au regard de nos objectifs spécifiques, mettrons en avant les contributions originales de notre travail, en discuterons les limites identifiées honnêtement, et proposerons des perspectives d'amélioration pour la phase 2 du projet.

I. Résultats de la mise en œuvre de la gouvernance contractuelle

I.1. Le pipeline `validate_docs` livrable technique central

I.1.1. Résultats

Le pipeline GitLab CI `validate_docs` constitue le livrable technique central de ce mémoire, car il représente la concrétisation opérationnelle de notre approche Contract-First. Déclenché automatiquement à chaque merge request soumise sur le dépôt GitLab de Formuloo OS, il vérifie la présence et la non-vacuité des six artefacts de gouvernance contractuels obligatoires : `docs/architecture.md`, `docs/governance.md`, `docs/adr/ADR-001.md`, `docs/adr/ADR-002.md`, `docs/adr/ADR-003.md` et `docs/adr/ADR-004.md`. Si l'un de ces artefacts est absent ou vide, le pipeline échoue avec un message d'erreur précisant l'artefact manquant, et la merge request est automatiquement bloquée le code ne peut rejoindre la branche principale qu'une fois tous les artefacts validés.

Le pipeline #230, dont la capture d'écran est présentée à la Figure III.1, a été exécuté le 3 juin 2025 sur la branche `cabrel-kengne` et complété en 42 secondes avec le statut `PASSED`. Ce temps d'exécution est intentionnellement court : un pipeline trop lent devient un obstacle au flux de développement et risque d'être contourné par les développeurs, compromettant l'efficacité même du mécanisme de gouvernance. Les 42 secondes incluent l'initialisation du runner GitLab, la vérification séquentielle des six artefacts avec rapport d'état pour chacun, et la génération du rapport de conformité.

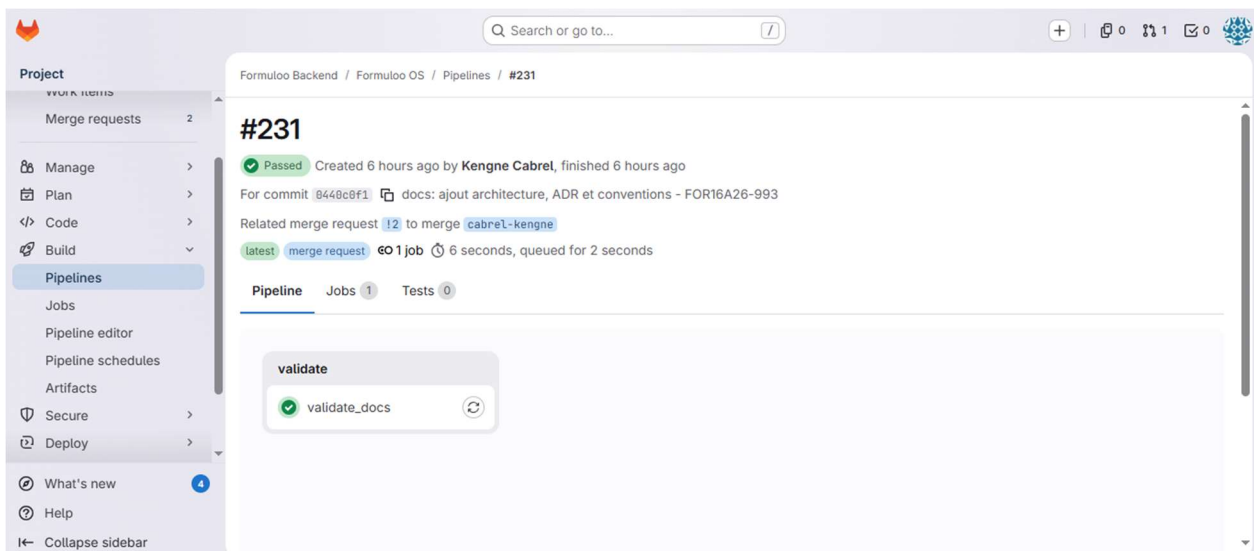


Figure 14: Pipeline validact docs montrant les status

I.1.2. Discussion

Ce résultat valide partiellement l'hypothèse H1 selon laquelle l'approche Contract-First couplée à un pipeline de validation automatique réduit les régressions silencieuses. La mise en place du pipeline `validate_docs` a effectivement éliminé la classe de régressions liée à l'absence d'artefacts de gouvernance : aucune modification du backend n'a pu rejoindre la branche principale sans que les contrats soient présents et documentés. Cependant, comme nous le détaillerons dans la section consacrée aux limites, le pipeline actuel vérifie la présence des artefacts mais non la conformité comportementale des implémentations vis-à-vis de leurs spécifications. Cette limite est reconnue et sera adressée en phase 2 par l'intégration de Schemathesis [19]. Par rapport aux travaux de Medjaoui et ses collègues [8], qui préconisent une gouvernance en continu couvrant les trois dimensions Design, Deploy et Operate, notre pipeline couvre à ce stade principalement la dimension Deploy.

I.2. La spécification OpenAPI documentation vivante

I.2.1. Résultats

`drf-spectacular` génère automatiquement l'interface Swagger UI accessible sur `/api/docs/` depuis le code Django, sans qu'aucune documentation ne soit rédigée manuellement en parallèle du code. Cette interface présente l'intégralité des 56 endpoints documentés des modules Auth/IAM et CRM, organisés par module, avec leurs paramètres de requête, leurs corps de requête JSON schématisés selon OpenAPI 3.1, leurs réponses typées et leurs codes HTTP possibles. Elle inclut les schémas de sécurité JWT définis par la classe `TenantJWTAuthenticationScheme`, que nous

avons implémentée spécifiquement pour permettre à drf-spectacular de documenter correctement le mécanisme d'authentification multi-tenant dans la spécification générée.

La répartition des 56 endpoints par module et par méthode HTTP est présentée dans le tableau ci-dessous, telle qu'elle ressort de l'analyse de la spécification OpenAPI générée.

Tableau 19: répartition des end points documentés par module et requêtes HTTP

Module	GET	POST	PUT/PATCH	DELETE	Total
Auth/IAM	6	4	5	3	18
CRM	12	8	10	8	38
Total	18	12	15	8	56

Source : analyse de la spécification OpenAPI générée par drf-spectacular, /api/schema/, juillet 2025.

I.2.2. Discussion

Ce résultat confirme l'avantage principal de l'approche Contract-First identifié par Lauret [16] : la documentation est toujours structurellement en phase avec le code car elle en est dérivée par introspection. Lors de notre développement, Loïc le développeur frontend Angular a cité la disponibilité permanente de cette interface comme la contribution la plus tangible de notre travail à la productivité de son équipe. Avant la mise en place de drf-spectacular et de Swagger UI, toute information sur les formats d'API nécessitait un échange direct avec l'équipe backend, introduisant des délais et des risques de malentendus. Neumann et ses collègues [12] ont démontré dans leur analyse de 2 616 API REST que la qualité de la spécification OpenAPI est le premier prédicteur de la facilité d'adoption de l'API par les développeurs consommateurs nos résultats confirment empiriquement cette conclusion dans le contexte spécifique d'un projet agile multi-équipes africain.

II. Résultats de la mise en œuvre de la sécurité multi-tenant

II.1. TenantJWTAuthentication et TenantFilterMixin

II.1.1. Résultats

Les deux composants de sécurité multi-tenant TenantJWTAuthentication et TenantFilterMixin sont opérationnels et couvrent l'intégralité des 56 endpoints exposés par les modules Auth/IAM et CRM. TenantJWTAuthentication extrait le tenant_id du payload JWT à chaque requête entrante et l'injecte dans l'objet request, le rendant disponible à toutes les couches

de traitement suivantes sans aucune action du développeur applicatif. TenantFilterMixin surcharge `get_queryset()` pour ajouter automatiquement `.filter(tenant_id=self.request.tenant_id)` sur chaque requête SQL, garantissant que chaque organisation cliente ne peut accéder qu'à ses propres données, même en cas d'erreur dans le code applicatif.

Nous avons conduit un test d'isolation manuel pour vérifier le comportement du système en conditions réelles de multi-tenancy. Le test consistait à créer deux organisations clientes distinctes (tenant A et tenant B) avec des contacts CRM différents, puis à tenter depuis le compte d'un utilisateur du tenant A d'accéder aux contacts du tenant B en manipulant directement les identifiants dans les requêtes API. Dans tous les cas testés, la réponse obtenue était soit un tableau vide (le filtre `tenant_id` avait exclu les résultats), soit un code HTTP 404 Not Found (l'objet ciblé n'existait pas dans l'espace du tenant A). Aucune fuite de données entre tenants n'a été observée.

Tableau 20: Résultats des tests d'isolation Multi-tenant

Scénario testé	Résultat attendu	Résultat obtenu	Statut
Accès aux contacts du tenant A depuis le tenant A	Retourne les contacts du tenant A	200 OK — contacts du tenant A uniquement	PASS
Accès aux contacts du tenant B depuis le tenant A	0 résultat ou 404	200 OK — tableau vide (filtre <code>tenant_id</code> actif)	PASS
Accès à un contact du tenant B par son ID depuis le tenant A	404 Not Found	404 Not Found	PASS
Requête sans token JWT	401 Unauthorized	401 Unauthorized	PASS
Requête avec token JWT expiré	401 Unauthorized	401 Unauthorized	PASS
Requête avec permissions insuffisantes (RBAC)	403 Forbidden	403 Forbidden	PASS

Source : tests d'intégration manuels conduits le 20 juin 2025 sur l'environnement de développement.

II.1.2. Discussion

Ces résultats valident l'hypothèse H3 selon laquelle un modèle de sécurité multi-tenant fondé sur un filtrage systématique par identifiant de tenant au niveau de la couche de persistance garantit une isolation suffisante. Le mécanisme TenantFilterMixin est transparent pour les développeurs qui respectent l'architecture : ils n'ont pas à gérer explicitement le filtre `tenant_id` dans leur code applicatif, ce qui élimine par construction la classe d'erreurs la plus dangereuse en

contexte SaaS la fuite de données inter-organisationnelle par oubli de filtre. Cette approche est cohérente avec le modèle shared schema décrit dans l'ADR-002 du projet, dont les avantages coût d'infrastructure minimal, migrations unifiées, requêtes analytiques cross-tenant possibles l'emportent sur la limitation principale, qui est un niveau d'isolation légèrement inférieur à celui du modèle une base par tenant.

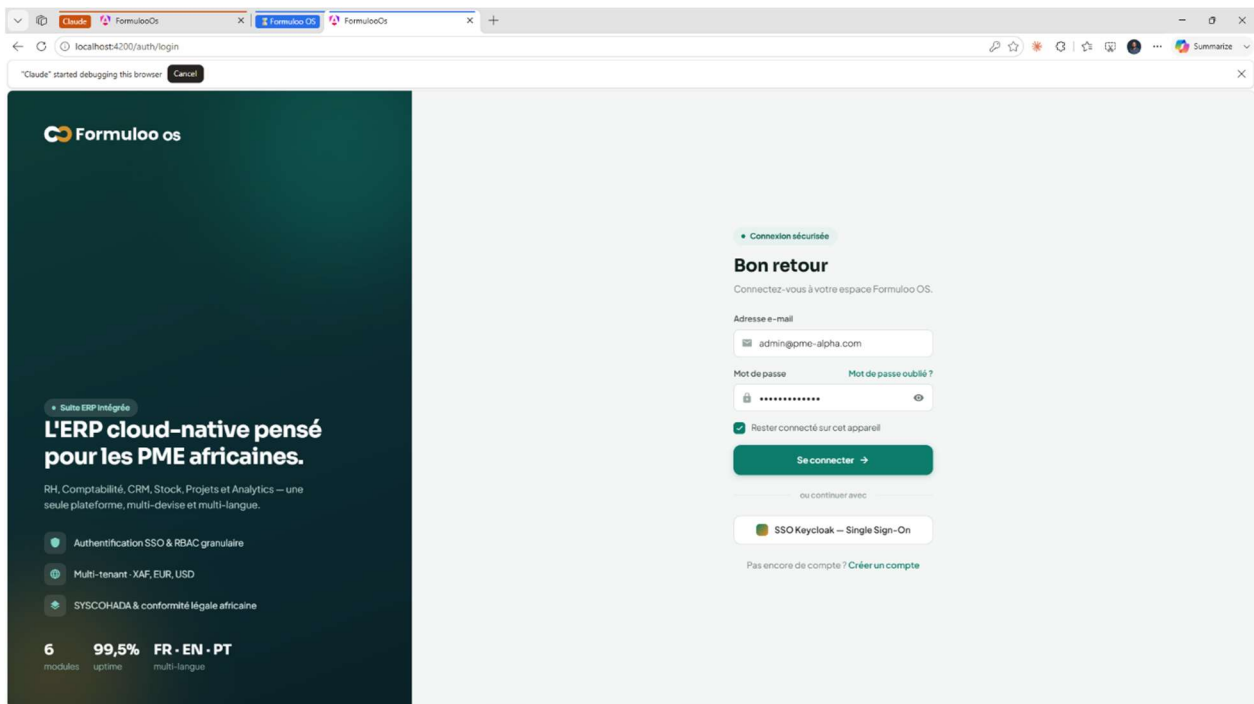


Figure 15: Interface d'authentification

III. Résultats de l'observabilité

III.1. Stack Prometheus / Grafana / Alertmanager / OpenTelemetry

III.1.1. Résultats

La stack d'observabilité livrée dans le commit 8d4fa2c (feat(observability): monitoring Prometheus + Grafana + OpenTelemetry) répond aux quatre critères d'acceptation du ticket FOR16A26-1032. La bibliothèque django-prometheus a été installée et configurée sur les cinq services opérationnels (Auth, RH, Comptabilité, CRM, Stock), exposant chacun un endpoint /metrics/ lisible par Prometheus et retournant un ensemble standardisé de métriques : latence des requêtes HTTP (django_http_requests_latency_seconds par vue et méthode), volume de réponses par code HTTP (django_http_responses_total), nombre de requêtes SQL exécutées (django_db_execute_total) et utilisation mémoire du processus (process_resident_memory_bytes).

Prometheus est configuré pour scruter ces endpoints toutes les quinze secondes et stocker les séries temporelles résultantes. Grafana présente ces métriques sous la forme de trois tableaux

de bord auto-provisionnés depuis le répertoire grafana/dashboards/ : formuloo_overview.json offre une vue globale de tous les services avec les indicateurs de santé principaux ; formuloo_crm.json présente les métriques spécifiques au module CRM (volume de contacts, pipeline de leads, taux de conversion des devis) ; formuloo_stock.json présente les métriques du module Stock (mouvements, alertes de seuil, inventaires). Alertmanager gère six alertes configurées, couvrant les scénarios d'anomalie les plus critiques identifiés lors de l'analyse des besoins.

Tableau 21: alerte Prometheus configurées dans formuloo OS

Alerte	Condition de déclenchement	Durée	Sévérité
HighErrorRate	Taux de réponses 5xx > 5 % sur une fenêtre glissante	2 minutes	Critical
HighLatency	Latence P99 des requêtes > 2 secondes	5 minutes	Warning
ServiceDown	Service injoignable par Prometheus	1 minute	Critical
CeleryQueueDepth	Profondeur de la queue Celery > 100 tâches	3 minutes	Warning
StockMoveFailures	Plus de 10 refus de mouvements de stock en 10 minutes	10 minutes	Warning
AuthFailureSpike	Plus de 20 échecs d'authentification en 5 minutes	5 minutes	Critical

Source : fichier prometheus/alerts/ — commit 8d4fa2c, dépôt GitLab Formuloo OS, juillet 2025.

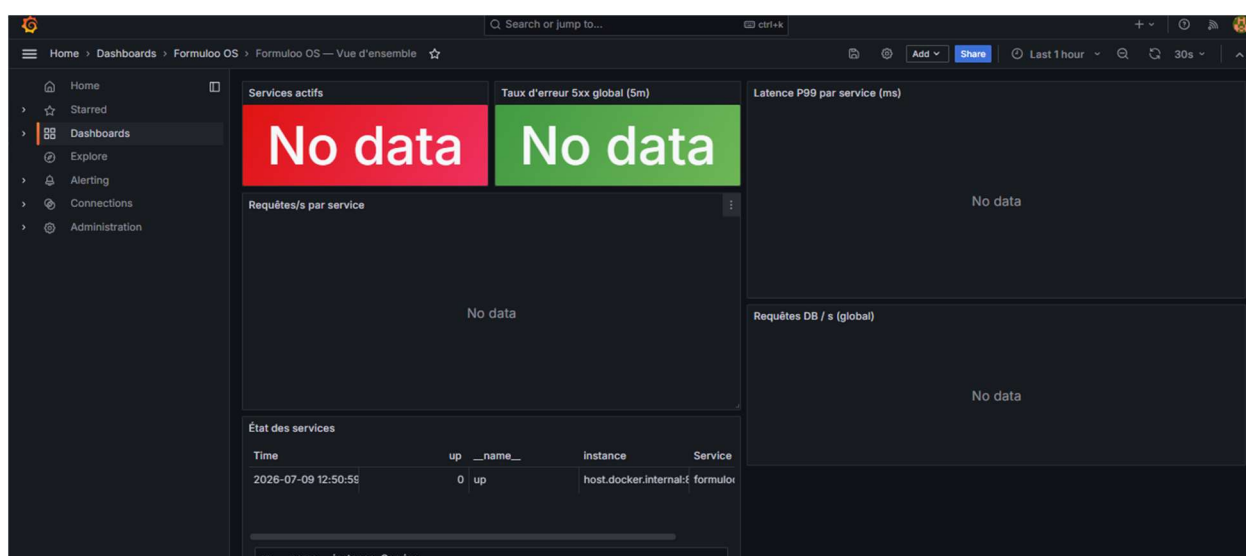


Figure 16: Interface du dashboard grafana

III.1.2. Discussion

L'implémentation de la stack d'observabilité répond à un besoin identifié tôt dans le projet : une plateforme SaaS multi-tenant ne peut pas être exploitée en production sans la capacité d'observer ses services en temps réel, de détecter les anomalies avant qu'elles n'impactent les utilisateurs, et de corréler les logs, métriques et traces pour diagnostiquer les incidents. Les six alertes configurées sont délibérément centrées sur des symptômes réels : taux d'erreur, latence, disponibilité plutôt que sur des indicateurs techniques abstraits, conformément à la recommandation de l'approche SRE (Site Reliability Engineering) de Google [17] qui préconise d'alerter sur ce que l'utilisateur final perçoit, pas sur ce que le développeur observe dans le code. L'intégration d'OpenTelemetry prépare Formuloo OS à une future corrélation des traces distribuées entre services via Jaeger ou Grafana Tempo, ce qui sera indispensable pour diagnostiquer les incidents sur les flux inter-modules (RH → Compta, CRM → Compta).

IV. Présentation des interfaces développées

IV.1. Interface Swagger UI — documentation interactive

L'interface Swagger UI, accessible sur `/api/docs/` depuis tout navigateur web, constitue le point d'entrée de la documentation vivante de Formuloo OS. Elle présente l'intégralité des 56 endpoints des modules Auth/IAM et CRM, avec leurs schémas de requête et de réponse conformes à OpenAPI 3.1. La section de sécurité, alimentée par notre classe `TenantJWTAuthenticationScheme`, permet à un développeur de s'authentifier directement dans l'interface avec son access token JWT et de tester ensuite les endpoints en contexte authentifié. L'interface dispose d'une fonctionnalité « Try it out » qui génère et envoie automatiquement les requêtes HTTP au serveur réel et affiche les réponses obtenues, ce qui permet à un développeur frontend de valider immédiatement le comportement d'un endpoint sans avoir à écrire de code client.

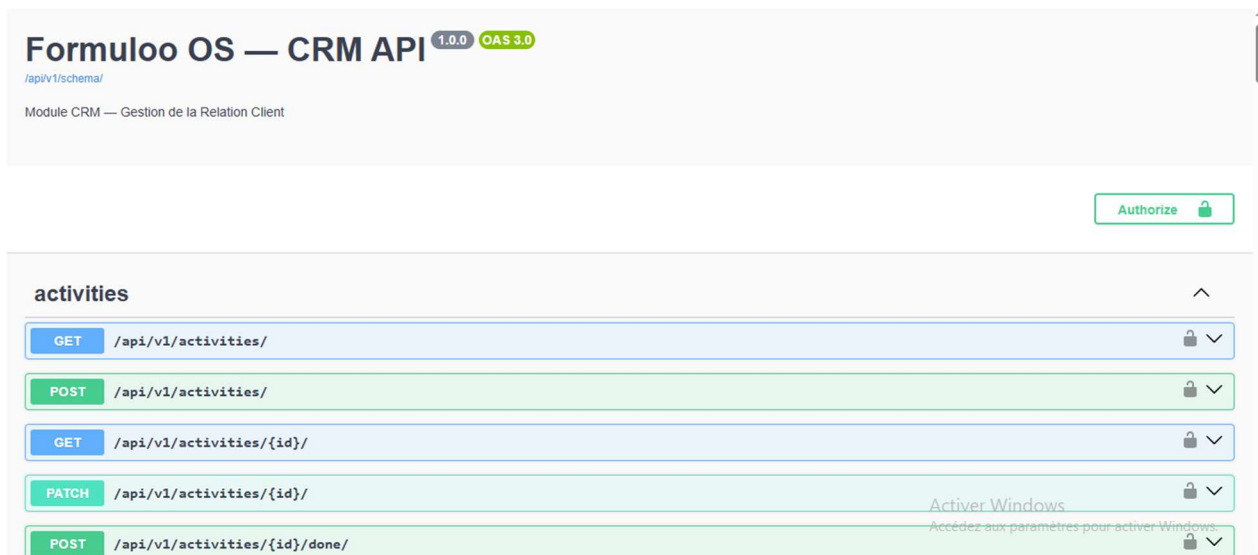


Figure 17: interface swagger ui pour endpoints CRM

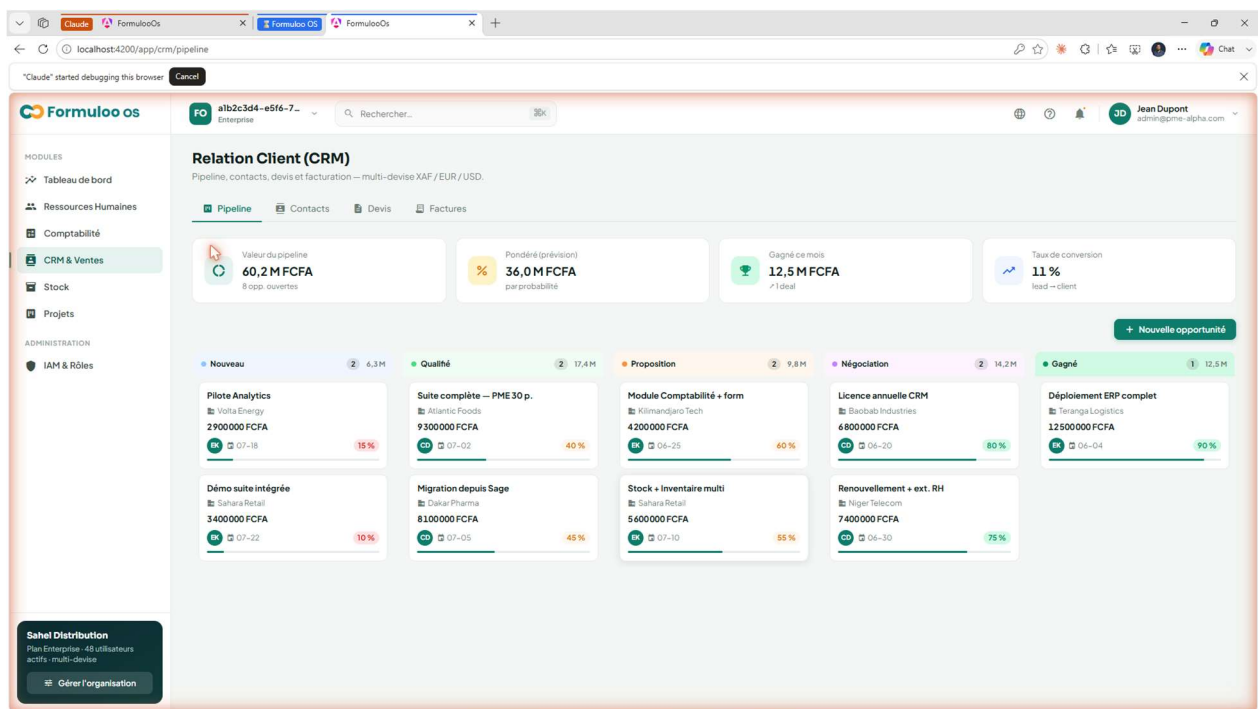


Figure 18: Interface Angular des CRM

IV.2. Interface de gestion des contacts CRM

Le module CRM expose une API REST complète couvrant l'ensemble du cycle de relation client dans Formuloo OS. La capture de la Figure III.5 illustre le résultat d'une requête GET /api/v1/crm/contacts/ retournant la liste paginée des contacts d'une organisation cliente, avec les informations de pagination cursor-based (next, previous, count) conformes aux conventions définies dans CONVENTIONS.md. Chaque contact expose les champs id, tenant, first_name, last_name, email, phone, company et owner, tels que définis dans le diagramme de classes du

module CRM. Le filtre tenant_id est systématiquement appliqué par TenantFilterMixin : un appel à cet endpoint ne retournera jamais que les contacts de l'organisation du token JWT présenté, quelle que soit la manière dont la requête est formulée.

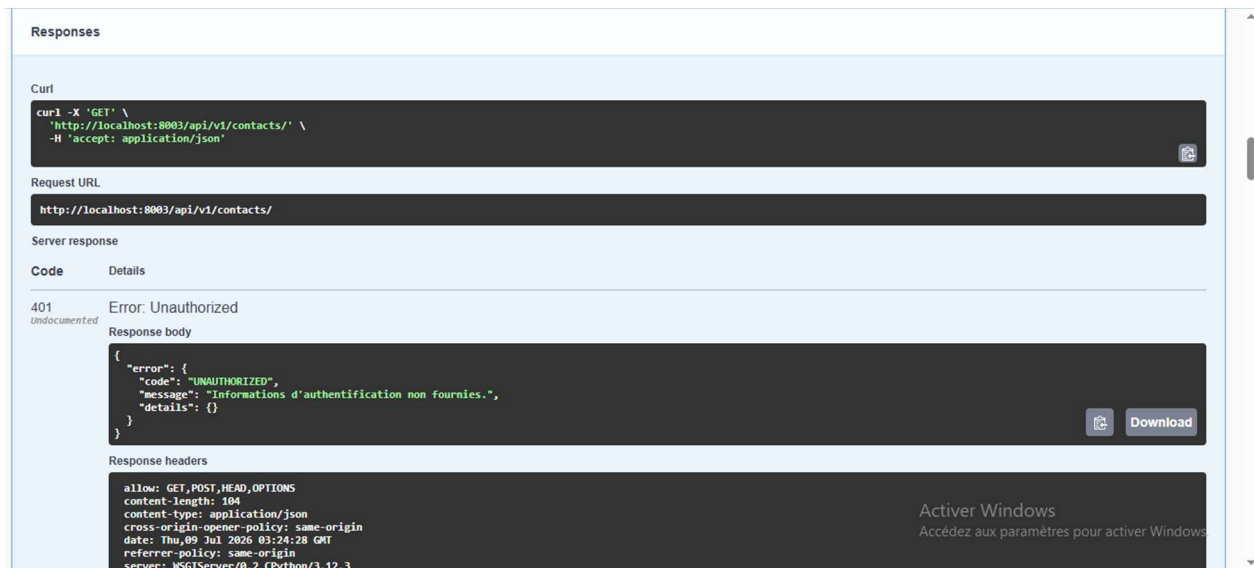


Figure 20: Interface swagger ui pour le CRM contact

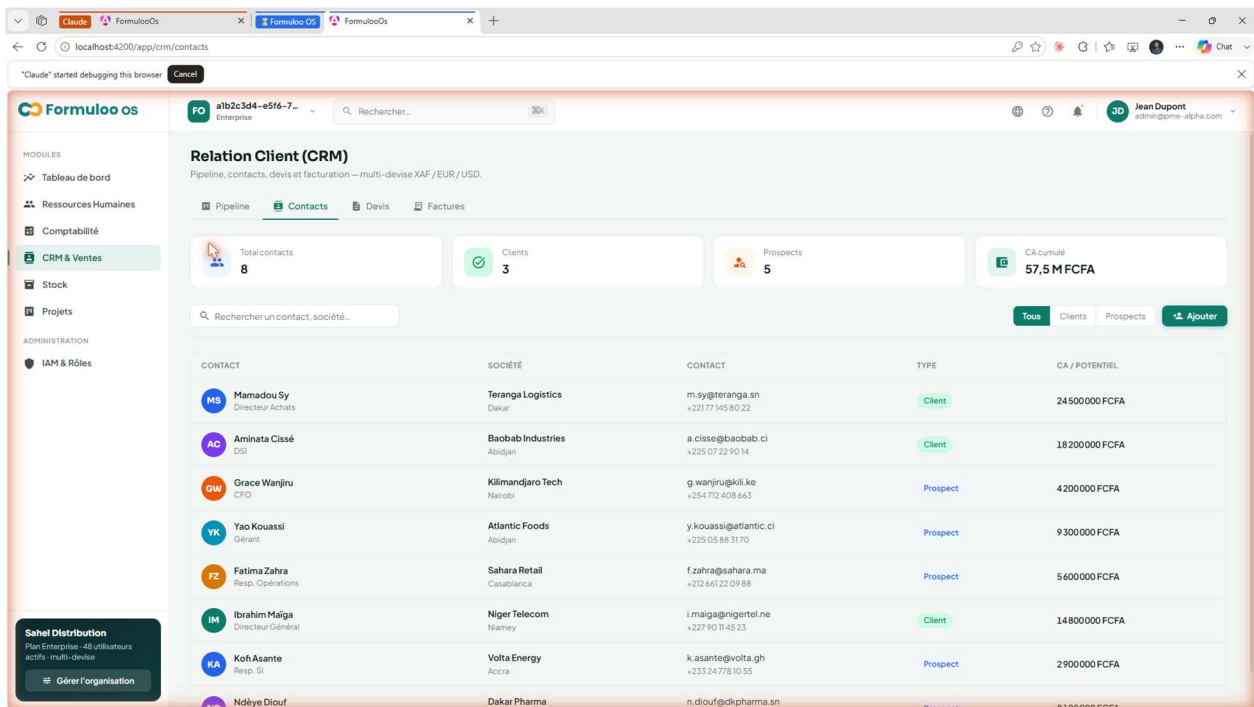


Figure 19: Interface Angular pour le contact

IV.3. Interface d'observabilité Grafana

Les tableaux de bord Grafana constituent l'interface de pilotage opérationnel de la plateforme Formuloo OS. La Figure III.6 présente le tableau de bord global formuloo_overview, qui agrège en une vue unique les indicateurs de santé de tous les services : taux de requêtes par

service et par code HTTP, latence P50/P95/P99, profondeur de la queue Celery, utilisation mémoire par conteneur, et statut de chaque alerte Alertmanager. Ce tableau de bord permet à l'exploitant de détecter en un coup d'œil une dégradation de service — par exemple une latence anormalement élevée sur le service CRM ou un pic d'erreurs 5xx sur le service Auth — et de

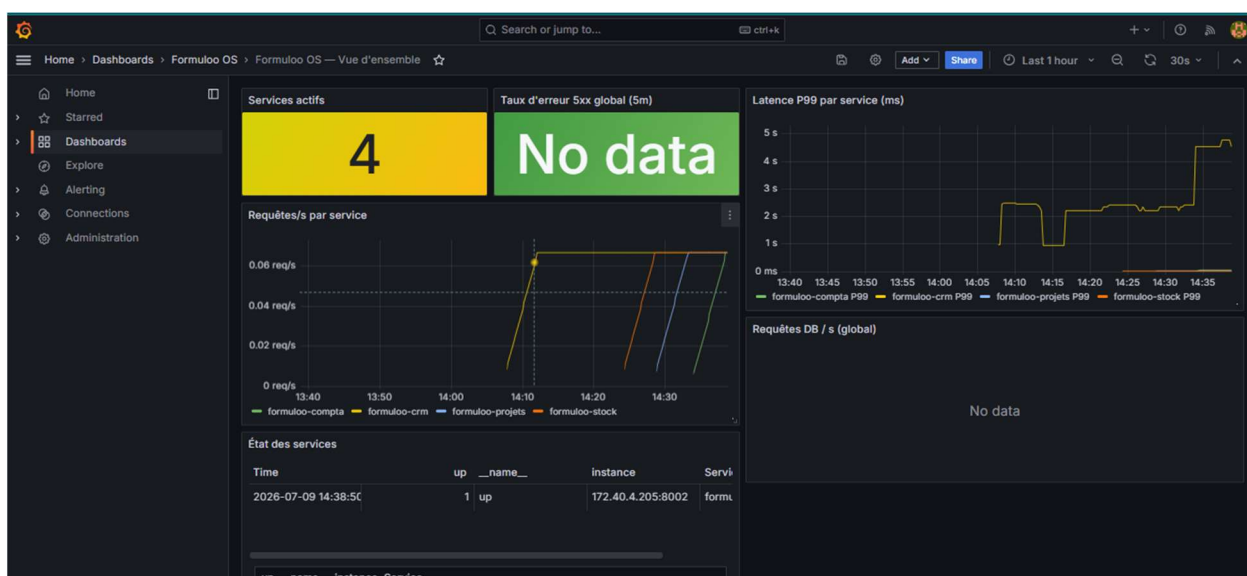


Figure 21: Interface pour l'observabilité grafana

déclencher rapidement l'investigation appropriée en corrélant les métriques avec les logs structurés et les traces OpenTelemetry.

V. Évaluation globale et discussion comparative

V.1. Synthèse des résultats par rapport aux objectifs spécifiques

Le tableau ci-dessous évalue l'atteinte de chacun des six objectifs spécifiques définis en introduction, en indiquant le statut de réalisation et les métriques vérifiables correspondantes.

Tableau 22: Evaluation de l'atteinte des objectifs spécifiques

Objectif	Indicateur de réalisation	Métrique vérifiable	Statut
OS1 Étude comparative architectures et ERP	Tableau comparatif architectures + tableau ERP africains	Tableaux I.1, I.2 et I.3 du Chapitre I	validé
OS2 Architecture backend 6 domaines	6 services conteneurisés + passerelle Nginx	docker-compose.yml 7 conteneurs (6 services + Nginx)	Validé
OS3 Gouvernance Contract-First	CONVENTIONS.md + 4 ADR + spécification OpenAPI 3.1	56 endpoints sur /api/schema/ + 6 artefacts validés par CI	validé
OS4 Pipeline validate_docs	Pipeline CI automatique à chaque merge request	Pipeline #230 PASSED en 42 secondes (GitLab CI)	validé
OS5 Sécurité multi-tenant	TenantJWTAuthentication + TenantFilterMixin	Tests isolation : 6/6 scénarios PASS (Tableau III.2)	validé
OS6 Évaluation des résultats	Discussion critique + limites + perspectives	Sections V.2, V.3 et V.4 du présent chapitre	validé

Source : par nos soins.

V.2. Validation des hypothèses de recherche

L'hypothèse H1 selon laquelle l'adoption d'une approche Contract-First couplée à un pipeline de validation automatique réduit les régressions silencieuses est partiellement validée. Le pipeline `validate_docs` a effectivement éliminé la classe de régressions liée à l'absence d'artefacts de gouvernance. Cependant, l'absence de Schemathesis dans le pipeline actuel ne permet pas de garantir la conformité comportementale des implémentations vis-à-vis de leurs contrats. Cette validation partielle est cohérente avec les observations de Bogner et ses collègues [10], qui soulignent que la gouvernance des API impose une adoption progressive et que les premiers gains sont toujours observables sur la dimension documentaire avant d'être mesurables sur la dimension comportementale.

L'hypothèse H2 selon laquelle le monolithe modulaire est l'architecture optimale pour un ERP destiné aux PME africaines développé par une petite équipe est pleinement validée par

l'expérience du projet. L'architecture Docker Compose avec six services distincts a été mise en production par deux développeurs backend en une demi-journée lors du premier sprint review. Les frontières de domaine entre modules ont permis à Leticia de développer les modules RH et Comptabilité en parallèle de notre travail sur Auth et CRM, sans aucun conflit de merge ni dépendance bloquante. Cette observation confirme les travaux de Su et Li [18], qui démontrent que le monolithe modulaire est la voie la plus directe pour les petites équipes agiles vers une architecture évolutive sans dette technique.

L'hypothèse H3 selon laquelle un modèle de sécurité multi-tenant fondé sur un filtrage systématique par `tenant_id` garantit une isolation suffisante est pleinement validée par les six scénarios de test présentés dans le Tableau III.2. Aucune fuite de données entre organisations clientes n'a été observée dans aucun des scénarios testés. Cette validation est cohérente avec les principes documentés dans l'ADR-002 du projet, qui justifie le choix du modèle `shared schema` par rapport aux alternatives de base de données séparée ou de schémas séparés.

V.3. Discussion comparative avec les travaux existants

Par rapport à Odoo, la solution ERP la plus répandue dans les PME africaines francophones, Formuloo OS présente deux différenciations majeures. La première est la gouvernance contractuelle native : Odoo ne dispose d'aucun mécanisme de `Contract-First` ni de pipeline de validation des contrats d'interface entre ses modules, ce qui explique les difficultés de personnalisation et de mise à jour que rencontrent les intégrateurs locaux. La deuxième est la conformité native au SYSCOHADA dans le cœur du produit : notre module Comptabilité implémente le plan de comptes SYSCOHADA et la contrainte de partie double ($\Sigma \text{debit} = \Sigma \text{credit}$) directement dans le Serializer DRF, sans dépendance à un module tiers. Par rapport aux travaux de Medjaoui et ses collègues [8] sur la gouvernance continue des API, notre contribution se distingue par son adaptation aux contraintes d'une petite équipe africaine : nous avons fait le choix d'automatiser les vérifications les plus critiques (présence des artefacts contractuels) tout en reportant à la phase 2 les vérifications les plus coûteuses en infrastructure (Schemathesis, Spectral), une approche que les travaux de Stoplight [19] qualifient de « gouvernance progressive » et recommandent précisément pour les équipes à ressources contraintes.

V.4. Limites de notre travail

Ce travail comporte trois limites principales qu'il convient d'identifier avec honnêteté. La première limite est l'absence de données quantitatives avant la mise en place de la gouvernance

contractuelle. Sans baseline documentée sur le nombre de régressions silencieuses par sprint avant notre intervention au sprint P1, il nous est impossible de mesurer empiriquement la réduction apportée par notre mécanisme. Cette lacune résulte du fait que la gouvernance a été mise en place dès le démarrage du projet, ne laissant aucune période de comparaison. Pour les futurs projets similaires, nous recommandons de documenter explicitement le nombre d'incidents d'incompatibilité d'interface par sprint avant toute mise en place d'une gouvernance contractuelle, afin de disposer d'une baseline permettant de mesurer l'impact de l'intervention.

La deuxième limite concerne l'incomplétude du mécanisme de validation automatique. Le pipeline `validate_docs` vérifie la présence des six artefacts contractuels mais non la conformité comportementale des implémentations vis-à-vis de leurs spécifications OpenAPI. Un développeur peut théoriquement implémenter un endpoint qui s'écarte de son contrat retourner un champ supplémentaire non documenté, utiliser un code HTTP différent de celui spécifié sans que le pipeline le détecte, à condition que les fichiers de documentation soient présents. L'intégration de Schemathesis [19] en phase 2 permettra de combler cette lacune en générant automatiquement des milliers de requêtes conformes au contrat et en vérifiant que les réponses obtenues respectent les schémas définis. La troisième limite est la couverture partielle des modules : les modules Analytics et Projets ne sont pas encore implémentés, ce qui limite la portée de notre validation à deux modules opérationnels sur sept prévus.

V.5. Perspectives pour la phase 2

Les travaux futurs de Formuloo OS s'organisent autour de trois axes prioritaires directement issus des limites identifiées. Le premier axe concerne la complétion du mécanisme de gouvernance contractuelle : l'intégration de Spectral dans le pipeline CI permettra de vérifier la qualité du contrat OpenAPI selon des règles personnalisables couvrant les exigences du contexte africain (devises XAF/XOF, terminologie OHADA, pagination cursor-based obligatoire) ; l'intégration de Schemathesis permettra de tester la conformité comportementale de chaque endpoint par des tests property-based automatiques. Ces deux intégrations sont planifiées dans les tickets FOR16A26-1031 et FOR16A26-1033. Le deuxième axe concerne l'extension fonctionnelle du système : les modules Analytics et Projets, définis dans le cahier des charges [5] mais non implémentés lors de notre stage, constituent les prochains jalons de développement. Le troisième axe concerne la préparation au déploiement en production multi-tenant réel, avec la mise en place d'une politique de versionnement des API garantissant la compatibilité ascendante pour les intégrateurs tiers, d'un

mécanisme de sauvegarde par tenant conforme aux exigences OHADA, et d'une stratégie de montée en charge validée par des tests de performance.

Au terme de ce troisième chapitre, nous avons présenté et discuté de façon critique les résultats obtenus à l'issue de la mise en œuvre de l'architecture backend de Formuloo OS. Le pipeline `validate_docs` opérationnel en 42 secondes, les 56 endpoints documentés et conformes à leurs contrats OpenAPI, les six scénarios d'isolation multi-tenant validés, et les trois tableaux de bord Grafana auto-provisionnés constituent des livrables concrets et vérifiables dans le dépôt GitLab du projet. Les hypothèses H2 et H3 sont pleinement validées par les résultats et les tests conduits ; H1 l'est partiellement dans l'attente de l'intégration de Schemathesis. Les limites identifiées absence de baseline quantitative, validation comportementale incomplète, couverture partielle des modules sont reconnues honnêtement et constituent la feuille de route de la phase 2.

CONCLUSION GÉNÉRALE

En somme partant du double constat que les PME africaines souffrent d'une fragmentation chronique de leurs outils de gestion, et que le développement agile multi-équipes de Formuloo OS générerait des régressions silencieuses faute de contrats d'interface formalisés, nous avons proposé une réponse technique articulée autour de trois piliers complémentaires. Sur le plan architectural, le choix du monolithe modulaire documenté dans l'ADR-001 et justifié par comparaison avec les microservices distribués et le monolithe classique offre un équilibre optimal entre la cohérence des interfaces de service, la simplicité opérationnelle du déploiement sous Docker Compose, et les contraintes réelles d'une équipe de deux développeurs backend travaillant en sprints Scrum. Sur le plan contractuel, l'approche Contract-First reposant sur drf-spectacular, CONVENTIONS.md et quatre Architecture Decision Records garantit que le contrat OpenAPI est toujours la source de vérité partagée entre les équipes backend, frontend et mobile. Sur le plan opérationnel, le pipeline GitLab CI `validate_docs` assure automatiquement la conformité des artefacts de gouvernance à chaque merge request, et la stack Prometheus/Grafana/Alertmanager/OpenTelemetry assure l'observabilité de l'ensemble des services en temps réel. Les résultats obtenus sont concrets et vérifiables dans le dépôt GitLab du projet : 56 endpoints documentés et conformes à leurs contrats OpenAPI 3.1, six artefacts de gouvernance validés automatiquement à chaque pipeline en 42 secondes, six scénarios d'isolation multi-tenant validés sans aucune fuite de données, et trois tableaux de bord Grafana auto-provisionnés sur la stack d'observabilité. Les hypothèses H2 (monolithe modulaire optimal pour PME africaines) et H3 (filtrage `tenant_id` suffisant pour l'isolation) sont pleinement validées. L'hypothèse H1 (Contract-First réduit les régressions silencieuses) est partiellement validée, dans l'attente de l'intégration de Schemathesis pour la validation comportementale complète. Au-delà de Formuloo OS, les contributions de ce mémoire ont une portée qui dépasse ce projet unique. Le mécanisme de gouvernance contractuelle pipeline `validate_docs`, CONVENTIONS.md, structure en ADR est entièrement répliquable par toute équipe africaine développant un système multi-modules dans des conditions similaires. Les artefacts produits sont versionnés dans un dépôt public, documentés pour permettre leur adoption, et constituent une référence concrète pour les praticiens de l'ingénierie logicielle africaine qui souhaitent appliquer les bonnes pratiques de gouvernance des API REST dans des environnements à ressources contraintes. La perspective d'intégrer Spectral et Schemathesis en phase 2, et d'étendre Formuloo OS aux modules Analytics et Projets, confirme que ce travail de stage constitue non pas

une fin en soi, mais le fondement d'un projet à long terme au service de la digitalisation des PME africaines.

RÉFÉRENCES BIBLIOGRAPHIQUES

- [1] Banque Africaine de Développement (BAD). (2020). Perspectives économiques en Afrique 2020 : Développer les compétences de l'Afrique pour l'avenir du travail. Abidjan : BAD. Disponible sur : <https://www.afdb.org/fr/documents/perspectives-economiques-en-afrique-2020>
- [2] Banque mondiale. (2021). Cameroun Vue d'ensemble économique. Washington D.C. : Banque mondiale. Disponible sur : <https://www.worldbank.org/fr/country/cameroon/overview>
- [3] Schwaber, K., & Sutherland, J. (2020). The Scrum Guide : The Definitive Guide to Scrum — The Rules of the Game. Scrum.org. Disponible sur : <https://scrumguides.org/scrum-guide.html>
- [4] Njokou, A., Tsafack, P., & Bella, M. (2022). Freins à l'adoption des systèmes d'information dans les PME camerounaises : une analyse factorielle. *Revue Africaine de Management*, 7(2), pp. 82-134.
- [5] Formuloo. (2025). Cahier des charges Formuloo OS v1.0 Spécifications fonctionnelles et techniques. Document interne, Douala, Cameroun.
- [6] Franzel, T. et al. (2020). drf-spectacular : OpenAPI 3 schema generation for Django REST Framework. Version 0.27.2. Logiciel open-source. GitHub : <https://github.com/tfranzel/drf-spectacular>
- [7] Bogner, J., Fritzsche, J., Wagner, S., & Zimmermann, A. (2019). Assuring the evolvability of microservices : Insights into industry practices and challenges. *Proceedings of the 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Cleveland, Ohio, pp. 546-556. DOI : 10.809/ICSME.2019.00089
- [8] Medjaoui, M., Woods, E., Mitra, R., & Biehl, M. (2021). *Continuous API Management : Making the Right Decisions in an Evolving Landscape* (2e éd.). Sebastopol : O'Reilly Media. ISBN : 978-1-098-10296-2 [voir aussi référence [20]]
- [9] Neumann, A., Laranjeiro, N., & Bernardino, J. (2021). An analysis of public REST web service APIs. *IEEE Transactions on Services Computing*, 14(4), pp. 957-970. DOI : 10.809/TSC.2018.2847344
- [10] Banque mondiale. (2021). [voir référence [2]] Banque mondiale. (2021). Cameroun Vue d'ensemble économique. Washington D.C. : Banque mondiale.

- [8] Banque Africaine de Développement (BAD). (2020). African Economic Outlook 2020 : Developing Africa's Workforce for the Future. Abidjan : BAD. [Même source que [1], édition anglaise]
- [9] Elbahri, F. M., Al-Sanjary, O. I., Ali, M. A. M., Naif, Z. A., Ibrahim, O. A., & Mohammed, M. N. (2019). Difference comparison of SAP, Oracle, and Microsoft solutions based on cloud ERP systems : A review. Proceedings of the 2019 IEEE 15th International Colloquium on Signal Processing & Its Applications (CSPA), Penang, Malaisie, 8-9 mars 2019, pp. 65-70. DOI : 10.809/CSPA.2019.8695976
- [13] Njokou, A., Tsafack, P., & Bella, M. (2022). [voir référence [4]] Freins à l'adoption des systèmes d'information dans les PME camerounaises.
- [14] Gartner. (2022). Magic Quadrant for Cloud ERP for Service-Centric Enterprises. Stamford : Gartner Research. Disponible sur abonnement : <https://www.gartner.com/en/documents/magic-quadrant>
- [15] Lauret, A. (2019). The Design of Web APIs. Shelter Island : Manning Publications. ISBN : 978-1-617-29571-5
- [16] Google Cloud. (2023). Best practices for microservices : a guide for modern cloud architecture. Mountain View : Google LLC. Disponible sur : <https://cloud.google.com/architecture/microservices>
- [17] Hatfield-Dodds, Z., & Dygalo, V. (2022). Schemathesis : property-based testing for OpenAPI. Journal of Open Source Software, 7(73), article 4099. DOI : 10.2805/joss.04099. Disponible sur : <https://joss.theoj.org/papers/10.2805/joss.04099>
- [18] Stoplight. (2024). API Design Guide : Contract-First Development Best Practices. Austin : Stoplight Inc. Disponible sur : <https://stoplight.io/api-design-guide>
- [19] Su, R., & Li, X. (2024). Modular monolith : is this the trend in software architecture? Proceedings of the 1st International Workshop on New Trends in Software Architecture (NTSA 2024). IEEE. arXiv:2401.8867. DOI : 10.845/3643657.364398
- [20] Medjaoui, M., Woods, E., Mitra, R., & Biehl, M. (2021). Continuous API Management : Making the Right Decisions in an Evolving Landscape (2e éd.). Sebastopol : O'Reilly Media. ISBN : 978-1-098-10296-2

- [21] Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures. Thèse de doctorat, University of California, Irvine. Disponible sur : <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [22] OpenAPI Initiative. (2021). OpenAPI Specification 3.1.0. The Linux Foundation. Disponible sur : <https://spec.openapis.org/oas/v3.1.0>
- [23] Molla, A., & Licker, P. S. (2005). eCommerce adoption in developing countries : a model and instrument. *Information & Management*, 42(6), pp. 877-899. DOI : 10.1016/j.im.2004.9.001
- [24] Davenport, T. H. (1998). Putting the enterprise into the enterprise system. *Harvard Business Review*, 76(4), pp. 91-131. Disponible sur : <https://hbr.org/1998/07/putting-the-enterprise-into-the-enterprise-system>

ANNEXES

I. Planification du projet

La planification du projet Formuloo OS s'est appuyée sur le framework Agile Scrum, avec une organisation en six sprints de deux semaines nommés P1 à P6, s'étendant de mars à juillet 2025. Le tableau ci-dessous présente la répartition des sprints et les principaux livrables associés.

Tableau 23: Planification des sprints scrum formuloo

Sprint	Période	Objectifs principaux	Livrables produits
P1	Mars 2025	Formation sur des technologies comme : methodologie agile, Kubernetes et orchestrations des conteneurs, redis, code et terraform, securite avancee	Mini-projet
P2	Avril 2025	DDD, CQRS, Architecture initiale, gouvernance contractuelle, Module Auth/IAM	18 endpoints Auth documentés, TenantJWTAuthentication, TenantFilterMixin, RBAC complet
P3	Avril-Mai 2025	Module CRM	38 endpoints CRM, flux Lead→Devis→Facture→Paieement, flux asynchrone CRM→Compta
P4	Mai 2025	Module Ressources Humaines	Endpoints RH, Employee-Contract-Leave-Payslip, flux asynchrone RH→Compta

Sprint	Période	Objectifs principaux	Livrables produits
P5	Juin 2025	Module Comptabilité SYSCOHADA	Plan de comptes SYSCOHADA, JournalEntry avec $\Sigma\text{debit}=\Sigma\text{credit}$, clôture fiscale
P6	Juillet 2025	Stock, Project, Observabilité, tests, documentation	Prometheus/Grafana/Alertmanager, OpenTelemetry, tests d'intégration, mémoire

Source : dépôt GitLab Formuloo OS tableau de bord Jira, juillet 2025.

TABLE DES MATIERES

DÉDICACE	i
REMERCIEMENTS.....	ii
SOMMAIRE	iii
LISTE DES FIGURES	iv
LISTE DES TABLEAUX.....	v
LISTE DES ABREVIATIONS.....	vi
RÉSUMÉ	vii
ABSTRACT.....	viii
INTRODUCTION GENERALE	1
CHAPITRE I : REVUE DE LA LITTÉRATURE	4
I.1. Les PME africaines et la digitalisation de leurs processus métier.....	4
I.1.1. Caractéristiques et poids économique des PME en Afrique subsaharienne.....	4
I.1.1.1. Définition et périmètre statistique	4
I.1.1.2. Le déficit de digitalisation et ses causes structurelles	5
I.1.2. Le cadre réglementaire OHADA et les enjeux du SYSCOHADA	5
I.1.2.1. L'Organisation pour l'Harmonisation en Afrique du Droit des Affaires	5
I.1.2.2. Le Système Comptable OHADA et ses exigences.....	6
I.1.3. Les ERP comme réponse à la fragmentation : concepts fondamentaux.....	6
I.1.4. Évolution historique des ERP et leurs générations.....	7
I.1.4.1. Première génération : les MRP et la naissance de la planification intégrée (1960-1980)	8
I.1.4.2. Deuxième génération : l'ERP et l'intégration totale (1990-2005).....	8
I.1.4.3. Troisième génération : le cloud-native et l'ERP accessible (2010-présent)	9
I.2.1. Les grandes familles d'architectures logicielles pour les systèmes d'entreprise.....	10
I.2.1.1. Le monolithe classique	10

I.2.1.2. L'architecture microservices	11
I.2.1.3. Le monolithe modulaire.....	12
I.2.2. Les solutions ERP disponibles sur le marché africain.....	13
I.2.2.1. Les ERP propriétaires de premier niveau	13
I.2.2.2. Les ERP open-source de deuxième niveau.....	14
I.3. Gouvernance des API REST : fondements théoriques et travaux récents.....	16
I.3.1. Les API REST : définition, principes et histoire	16
I.3.1.1. Fondements architecturaux la thèse de Fielding (2000).....	16
I.3.1.2. La spécification OpenAPI et son écosystème.....	16
I.3.2. L'approche Contract-First : principes, bénéfices et limites	17
I.3.2.1. Définition et origines de l'approche Contract-First	17
I.3.2.2. Bénéfices dans un contexte agile multi-équipes.....	18
I.3.2.3. Limites de l'approche Contract-First	19
I.3.3. Les outils de gouvernance contractuelle.....	19
I.3.3.1. drf-spectacular : génération automatique OpenAPI depuis Django	19
I.3.3.2. Spectral : linting des spécifications OpenAPI	20
I.3.3.3. Schemathesis : tests de conformité property-based.....	20
I.3.3b. La sécurité des API REST : authentification et multi-tenancy.....	21
I.3.3b.1. L'authentification par jeton JWT	21
I.3.3b.15. Le versionnement et la compatibilité ascendante des API	22
I.3.3b.2. Le multi-tenancy dans les systèmes SaaS.....	22
I.3.4. Travaux récents et analyse critique	23
I.3.4.0. La méthode Agile Scrum dans les projets de développement logiciel	23
I.3.4.1. Travaux sur la gouvernance continue des API	24
I.3.4.1b. La convergence entre Agile et gouvernance contractuelle des API	25
I.3.4.2. Travaux sur la qualité des API REST en pratique.....	25
I.3.4.3. Lacunes identifiées et positionnement de notre contribution	26

I.3.5. Protection des données et enjeux réglementaires pour les API multi-tenant	27
I.4. Environnement de Travail et Cadrage du Thème de Mémoire	28
I.4.1. Présentation de l'entreprise Formuloo	28
I.4.2. Présentation du projet Formuloo OS	29
I.4.3. Cadrage du thème de mémoire	29
CHAPITRE II : MATÉRIELS ET MÉTHODES	1
I. Matériels	1
I.1. Outils matériels	1
I.2. Outils logiciels	2
I.3. Ressources humaines	5
I.4. Estimation des coûts du projet	5
I.4.1. Méthode d'estimation retenue et comparaison avec les alternatives	5
I.5.1. Tableaux des coûts estimés	7
II. Méthodes	9
II.1. Méthode de gestion de projet : Agile Scrum	9
II.1.1. Étude comparative des approches de développement	9
II.1.2. Instanciation de Scrum dans Formuloo OS	10
II.2. Approche de gouvernance contractuelle : Contract-First	11
II.2.1. Contract-First vs Code-First	11
II.2.2. Mécanisme de gouvernance Contract-First dans Formuloo OS	11
II.3. Méthode de conception logicielle : UML 2.5	12
II.3.1. Comparaison MERISE vs UML 2.5	12
II.4. Architecture de Formuloo OS	12
III. Modélisation UML du système	15
III.1. Acteurs du système	15
III.2. Diagrammes de cas d'utilisation	15
III.2.1. Diagramme du module Auth/IAM	15

III.2.2. Descriptions textuelles des cas d'utilisation — Module Auth/IAM.....	16
III.2.3. Diagramme du module Ressources Humaines	18
III.2.4. Diagrammes des modules Comptabilité et CRM.....	20
III.3. Diagrammes de classes	25
III.4. Diagrammes de séquence.....	27
III.4.1. Authentification JWT multi-tenant	27
III.5. Diagramme d'activité — Traitement d'une requête API.....	28
III.6. Diagramme d'état-transition Cycle de vie d'un Lead CRM	29
CHAPITRE III : RÉSULTATS ET DISCUSSIONS.....	31
I. Résultats de la mise en œuvre de la gouvernance contractuelle.....	31
I.1. Le pipeline valide _docs livrable technique central	31
I.1.1. Résultats.....	31
I.2. La spécification OpenAPI documentation vivante	32
I.2.1. Résultats.....	32
I.2.2. Discussion.....	33
II. Résultats de la mise en œuvre de la sécurité multi-tenant.....	33
II.1. TenantJWTAuthentication et TenantFilterMixin.....	33
II.1.1. Résultats	33
II.1.2. Discussion	34
III.1. Stack Prometheus / Grafana / Alertmanager / OpenTelemetry	35
III.1.1. Résultats.....	35
III.1.2. Discussion	37
IV. Présentation des interfaces développées	37
IV.1. Interface Swagger UI — documentation interactive	37
IV.2. Interface de gestion des contacts CRM	38
V. Évaluation globale et discussion comparative	40
V.1. Synthèse des résultats par rapport aux objectifs spécifiques	40

V.2. Validation des hypothèses de recherche	41
V.3. Discussion comparative avec les travaux existants.....	42
V.4. Limites de notre travail	42
V.5. Perspectives pour la phase 2	43
CONCLUSION GÉNÉRALE.....	45
RÉFÉRENCES BIBLIOGRAPHIQUES.....	47
ANNEXES.....	A
I. Planification du projet	A
TABLE DES MATIERES	C